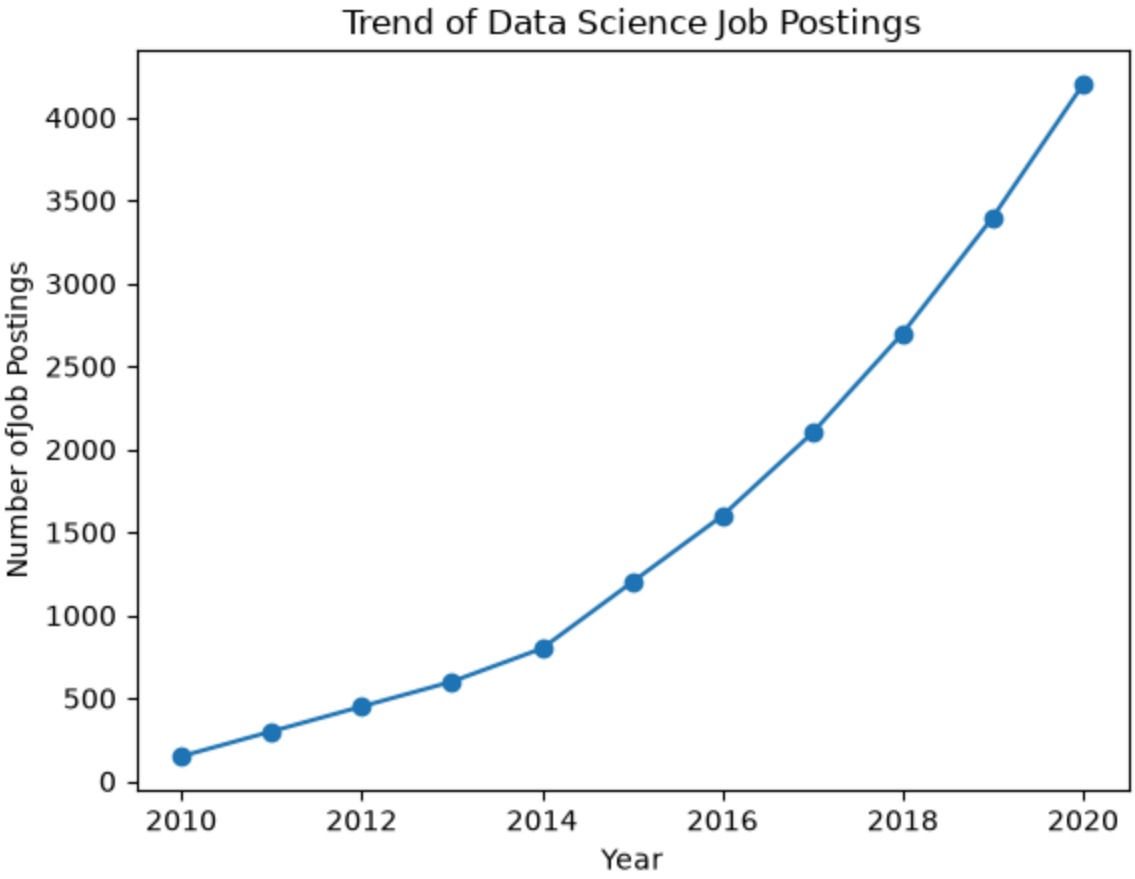
Exp No: 1.a Analyze the trend of data science job postings over the last decade
Description: Use web scraping (e.g., BeautifulSoup) or APIs (e.g., LinkedIn API) to gather data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib/seaborn for visualization.

In [5]:
```
import pandas as pd
import matplotlib.pyplot as plt
data = {'Year': list(range(2010, 2021)),
'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200

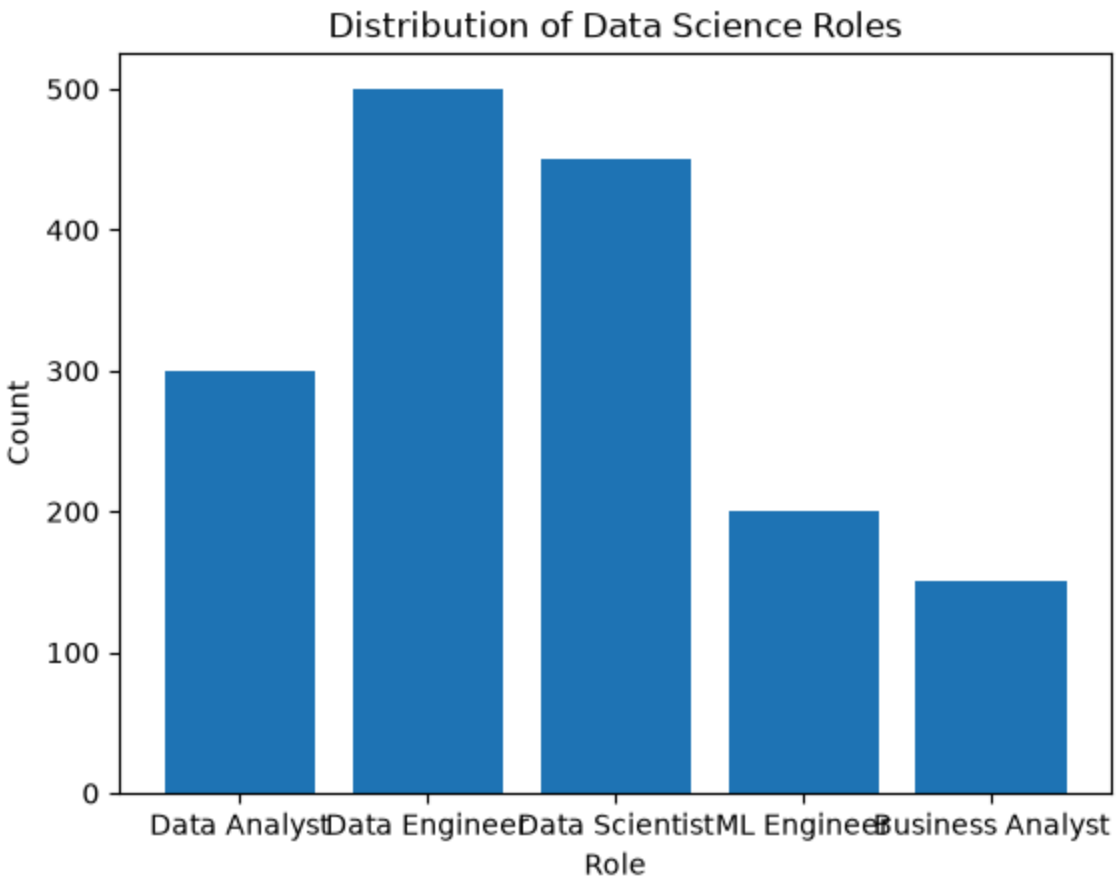
df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number ofJob Postings')
plt.show()
```



Exp No:1.b Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset. Description: Use a dataset of job postings and categorize them into different roles. Visualize the distribution using pie charts or bar plots.

In [6]:
```
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer',
'Business Analyst']
counts = [300, 500, 450, 200, 150]
```

```
plt.bar(roles, counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.show()
```



Exp No:1.c Conduct an experiment to differentiate Structured , Un-structured and Semi structured data based on data sets given. Description: Create small datasets for each type and explain their characteristics.

In [8]:
```
# Structured data example
structured_data = pd.DataFrame({
'ID': [1, 2, 3],
'Name': ['Alice', 'Bob', 'Charlie'],
'Age': [25, 30, 35]
})
print("Structured Data:\n", structured_data)

# Unstructured data example
unstructured_data = "This is an example of unstructured data. It can be a pi
print("\nUnstructured Data:\n", unstructured_data)

# Semi-structured data example (JSON)
semi_structured_data = {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 1
print("\nSemi-structured Data:\n", semi_structured_data)
```

```
Structured Data:
    ID     Name  Age
0   1    Alice   25
1   2      Bob   30
2   3  Charlie   35

Unstructured Data:
 This is an example of unstructured data. It can be a piece of text, an imag
e,or a video file.

Semi-structured Data:
 {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
```

Exp No:1.d Conduct an experiment to encrypt and decrypt given sensitive data.

Description: Use the cryptography library to encrypt and decrypt a piece of data.

In [10]:

```python
# Generate key and encrypt data
from cryptography.fernet import Fernet
key = Fernet.generate_key()
f = Fernet(key)
token = f.encrypt(b"Rajalakshmi Engineering College")
token
b'...'
f.decrypt(token)
b'Rajalakshmi Engineering College'
key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Rajalakshmi Engineering College."
cipher_text = cipher_suite.encrypt(plain_text)
# Decrypt data
decrypted_text = cipher_suite.decrypt(cipher_text)
print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

```
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data: b'gAAAAABqxlJFm49-lidsmx5hqtYK_uvFjDj7KOoXxmR4D_l6O_FVzO_UUS-
a1rI8rfFV4oIpfdWVnoKo-dcstuvjQOrJnyGGpP4DKeuSwYi_BlGyZ_P2g7G-5gU-FvFPKbzevK7I
vlJd'
Decrypted Data: b'Rajalakshmi Engineering College.'
```

Exp No:2 Upload and Analyze the data set given in csv format and perform data preprocessing and visualization. Description: Use sample data set sales-data.csv.

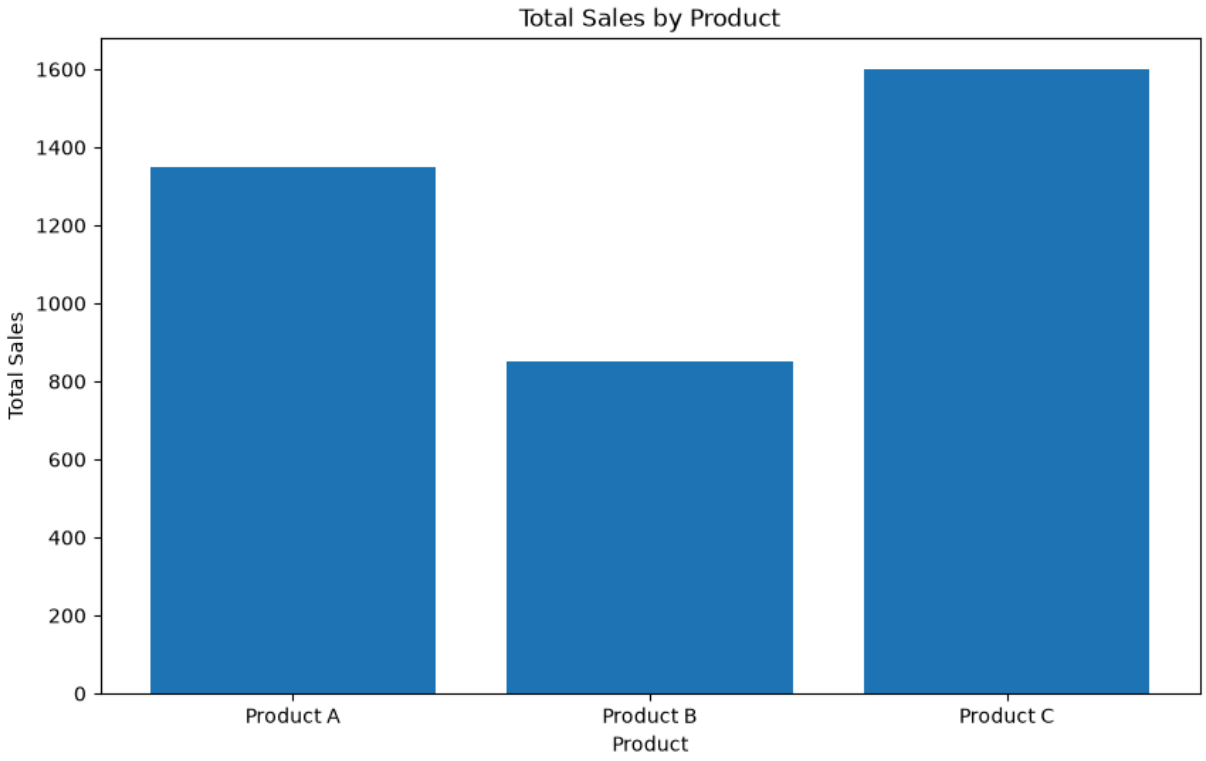
In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Load the data into a pandas DataFrame
file_path=r"C:\Users\jaiwa\Downloads\sales_data - sales_data.csv"
df = pd.read_csv(file_path)
# Display the first few rows of the DataFrame
print(df.head())
# Check for missing values
print(df.isnull().sum())
# Fill or drop missing values if necessary
#df['Sales'].fillna(df['Sales'].mean(), inplace=True)
df['Sales'] = df['Sales'].fillna(df['Sales'].mean())
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
# Summary statistics
print(df.describe())
# Group by product and calculate the total sales and quantity
product_summary = df.groupby('Product').agg({
'Sales': 'sum',
'Quantity': 'sum'
}).reset_index()
print(product_summary)

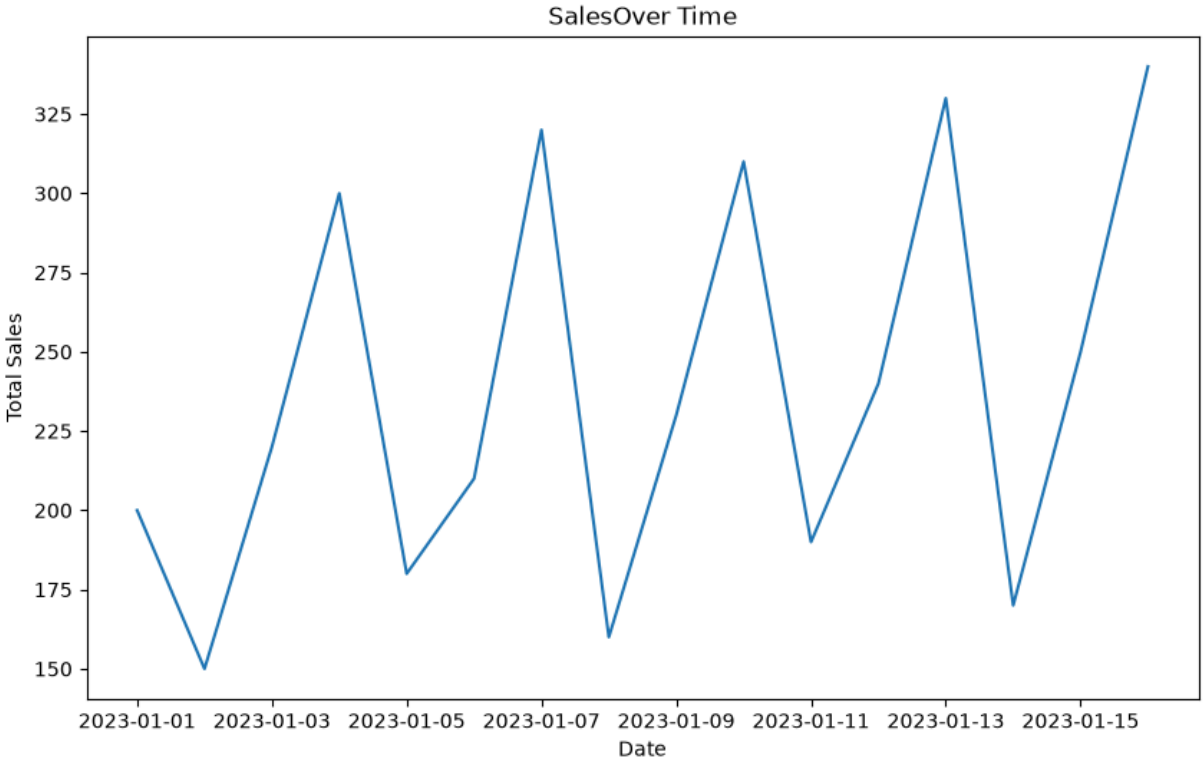
11

# Bar plot of total sales by product
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
# Line plot of sales over time
#df['Date'] = pd.to_datetime(df['Date'])
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('SalesOver Time')
plt.show()
# Pivot table to analyze sales by region and product
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Produc
aggfunc=np.sum, fill_value=0)
print(pivot_table)
# Correlation matrix
#correlation_matrix = df.corr()
correlation_matrix = df.corr(numeric_only=True)
print(correlation_matrix)
```

```python
# Heatmap of the correlation matrix
import seaborn as sns
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

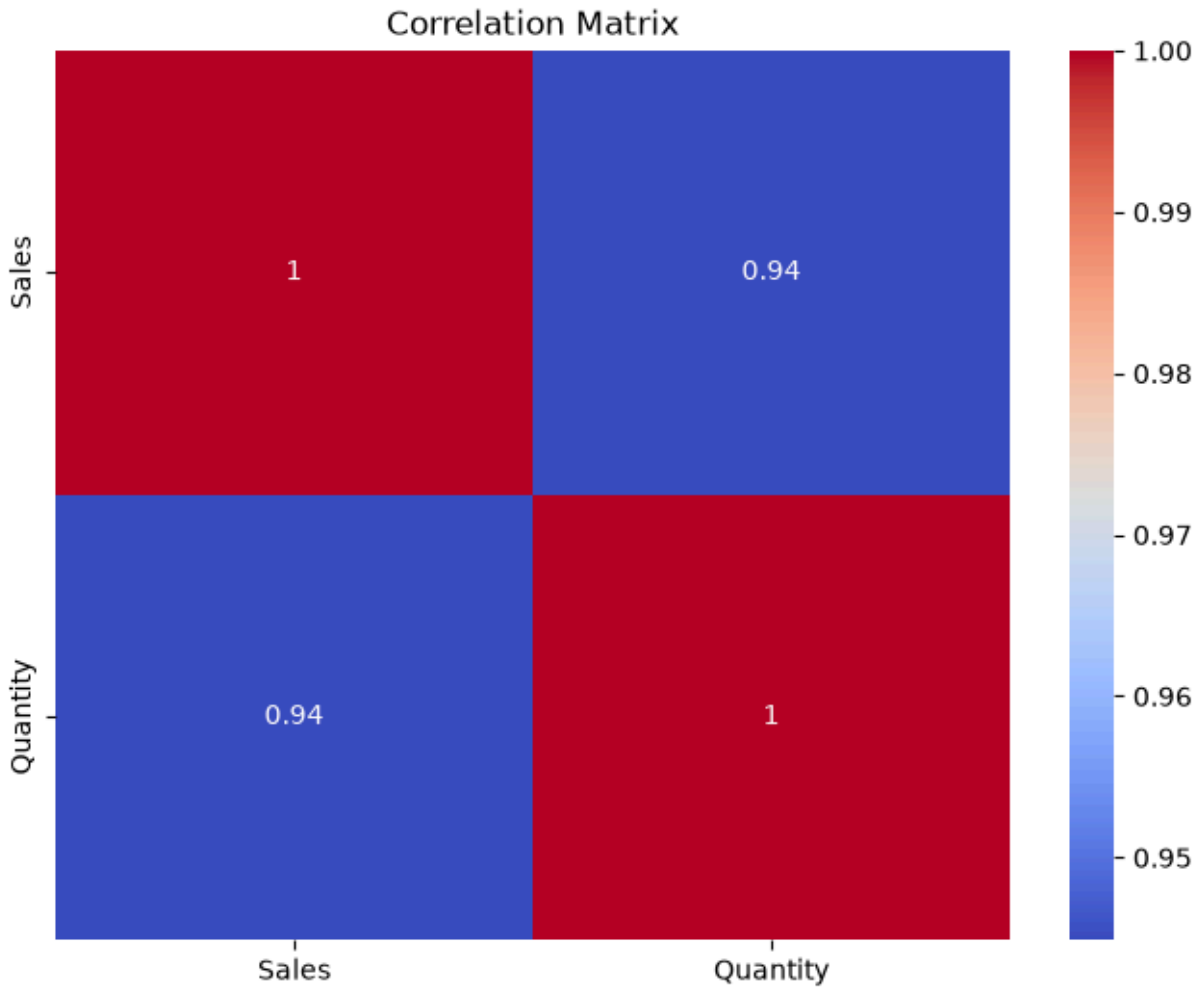
```
         Date    Product  Sales  Quantity Region
0  01-01-2023  Product A    200         4  North
1  02-01-2023  Product B    150         3  South
2  03-01-2023  Product A    220         5  North
3  04-01-2023  Product C    300         6   East
4  05-01-2023  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
std     64.031242   1.746425
min    150.000000   3.000000
25%    187.500000   4.000000
50%    225.000000   5.500000
75%    302.500000   7.000000
max    340.000000   8.000000
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```





Product	Product A	Product B	Product C
Region			
East	0	0	1600
North	1350	0	0
South	0	480	0
West	0	370	0

	Sales	Quantity
Sales	1.000000	0.944922
Quantity	0.944922	1.000000

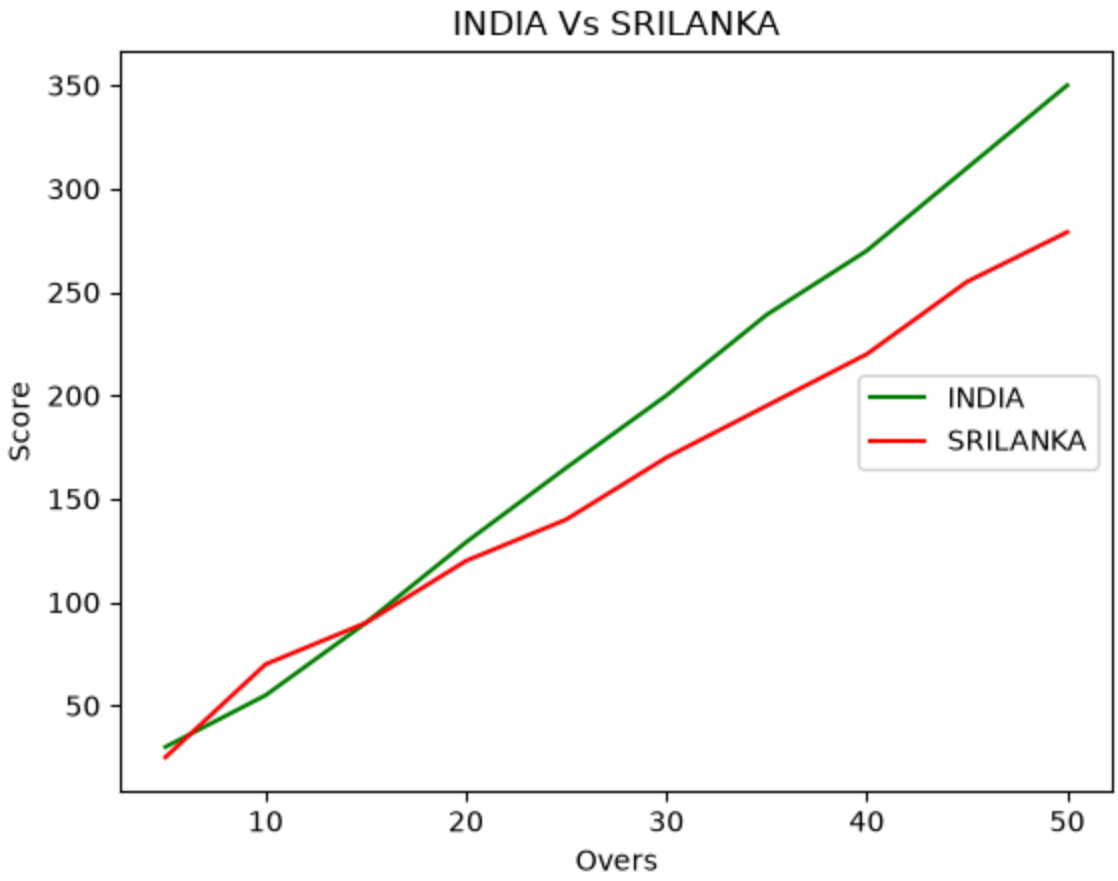

Correlation Matrix
Sales
Quantity
1
0.94
0.94
1
Sales
Quantity
1.00
0.99
0.98
0.97
0.96
0.95

In [ ]:

Exp No:3.a Conduct an experiment to show data visualization using line plot
Description: Take any sample data either through csv file or data fetched directly through code

In [5]:
```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
#cricket.plot(Overs,Indian_Score)
#cricket.plot(Overs,Srilankan_Score)

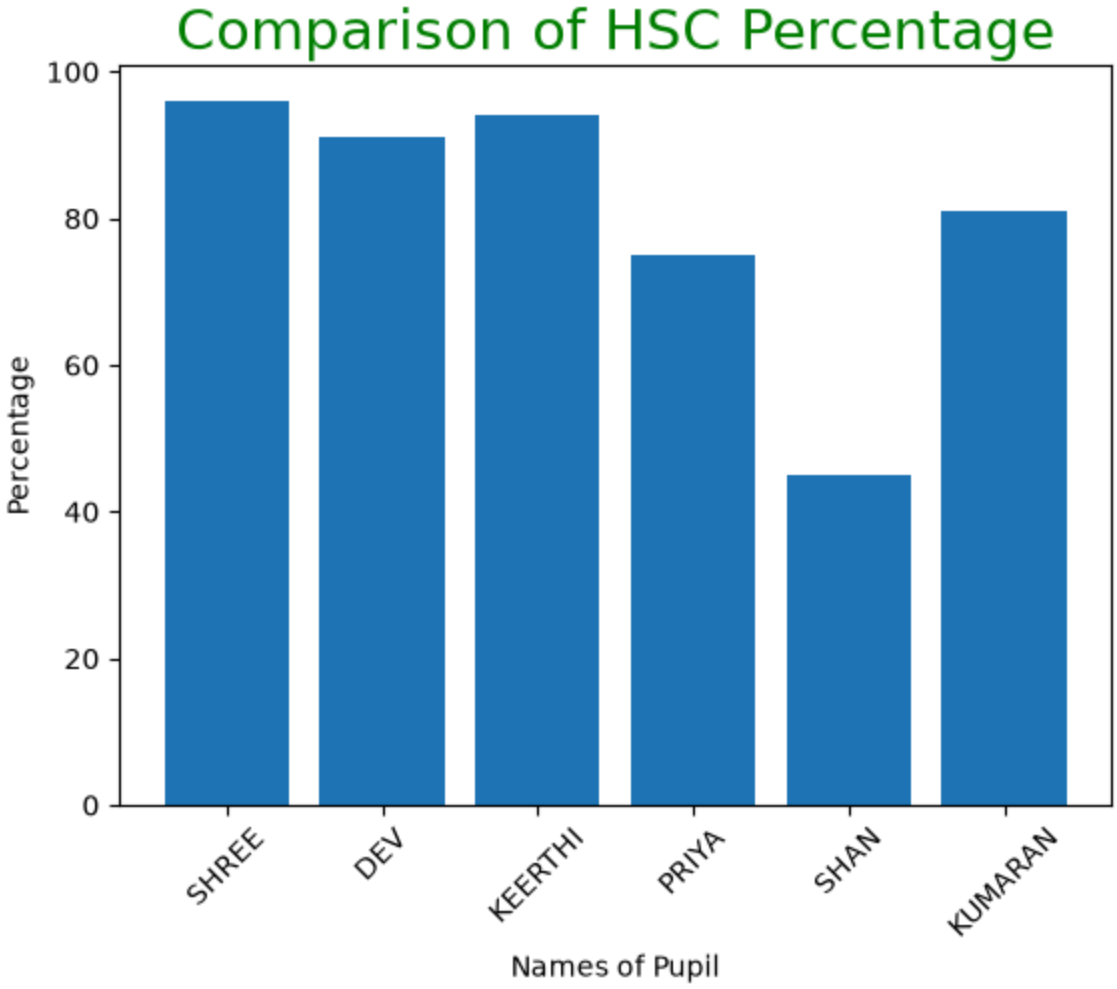
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
#cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
cricket.show()
```



Exp No:3.b Conduct an experiment to show data visualization using bar chart
Description: Take any sample data either through csv file or data fetched directly through code

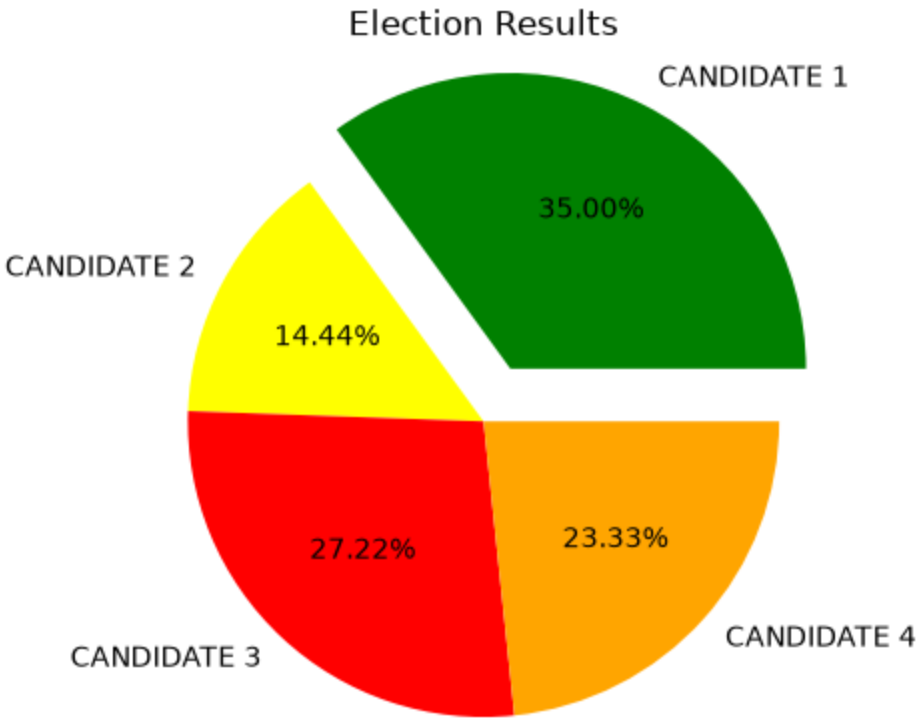
In [6]:
```
import matplotlib.pyplot as hscmark
import numpy as np
```

```
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```



Exp No:3.c Conduct an experiment to show data visualization using pie chart
Description: Take any sample data either through csv file or data fetched directly through code

In [7]:
```
import matplotlib.pyplot as election
# Election data
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)
# Plotting the pie chart
election.pie(Votes, labels=labels, colors=colors, explode=explode, autopct='
election.title('Election Results')
election.show()
```


Election Results
CANDIDATE 1
35.00%
CANDIDATE 2
14.44%
27.22%
23.33%
CANDIDATE 3
CANDIDATE 4

Exp No:4 To Count the frequency of occurrence of a word in a body of text is often needed during text processing Description: Import the word_tokenize function and gutenberg.

In [7]:
```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
# Download the gutenberg corpus if not already installed
nltk.download('gutenberg')
nltk.download('punkt') # Ensure the punkt tokenizer models are also downloaded
# Load the text from the gutenberg corpus
sample = gutenberg.raw("austen-emma.txt") # Change to an existing text from
# Tokenize the sample text
token = word_tokenize(sample)
# Create a list of the first 50 tokens
wlist = []
for i in range(50):
 wlist.append(token[i])
# Calculate the frequency of each word in the list
wordfreq = [wlist.count(w) for w in wlist]
# Print the word-frequency pairs
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

```
[nltk_data] Downloading package gutenberg to
[nltk_data]     C:\Users\jaiwa\AppData\Roaming\nltk_data...
[nltk_data]   Package gutenberg is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\jaiwa\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1),
(']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma', 2), ('W
oodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever', 1), (',', 5),
('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a', 1), ('comfortable', 1),
('home', 1), ('and', 3), ('happy', 1), ('disposition', 1), (',', 5), ('seeme
d', 1), ('to', 1), ('unite', 1), ('some', 1), ('of', 2), ('the', 2), ('best',
1), ('blessings', 1), ('of', 2), ('existence', 1), (';', 1), ('and', 3), ('ha
d', 1), ('lived', 1), ('nearly', 1), ('twenty-one', 1), ('years', 1), ('in',
1), ('the', 2), ('world', 1), ('with', 2)]
```

Exp No:5 Data Collection and Initial Exploration Objective: To collect, load, and perform initial exploration of the diabetes dataset.

In [4]:
```
import pandas as pd
db=pd.read_csv("C:\\Users\\jaiwa\\Downloads\\diabetes.csv")
print(db.head())
```

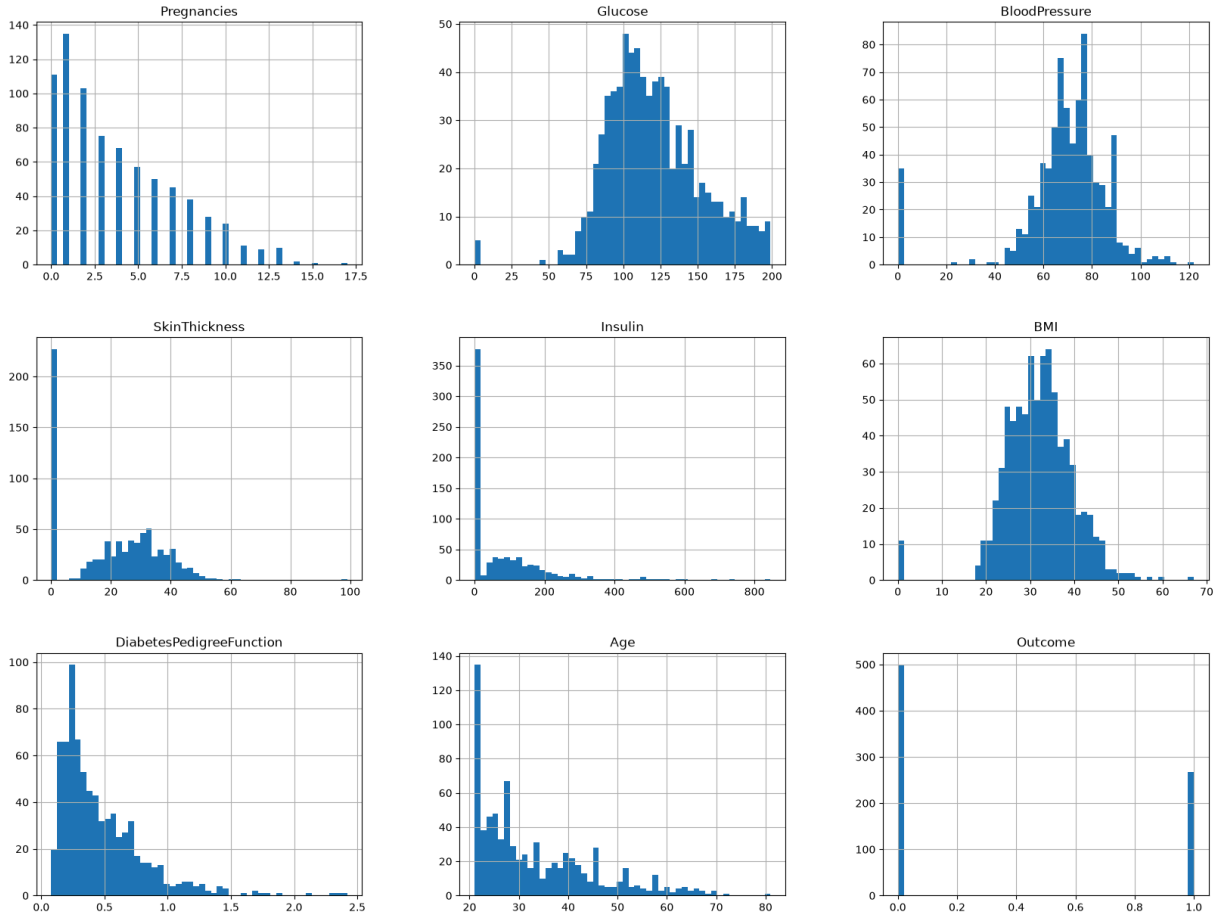
```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6   
1            1       85             66             29        0  26.6   
2            8      183             64              0        0  23.3   
3            1       89             66             23       94  28.1   
4            0      137             40             35      168  43.1   

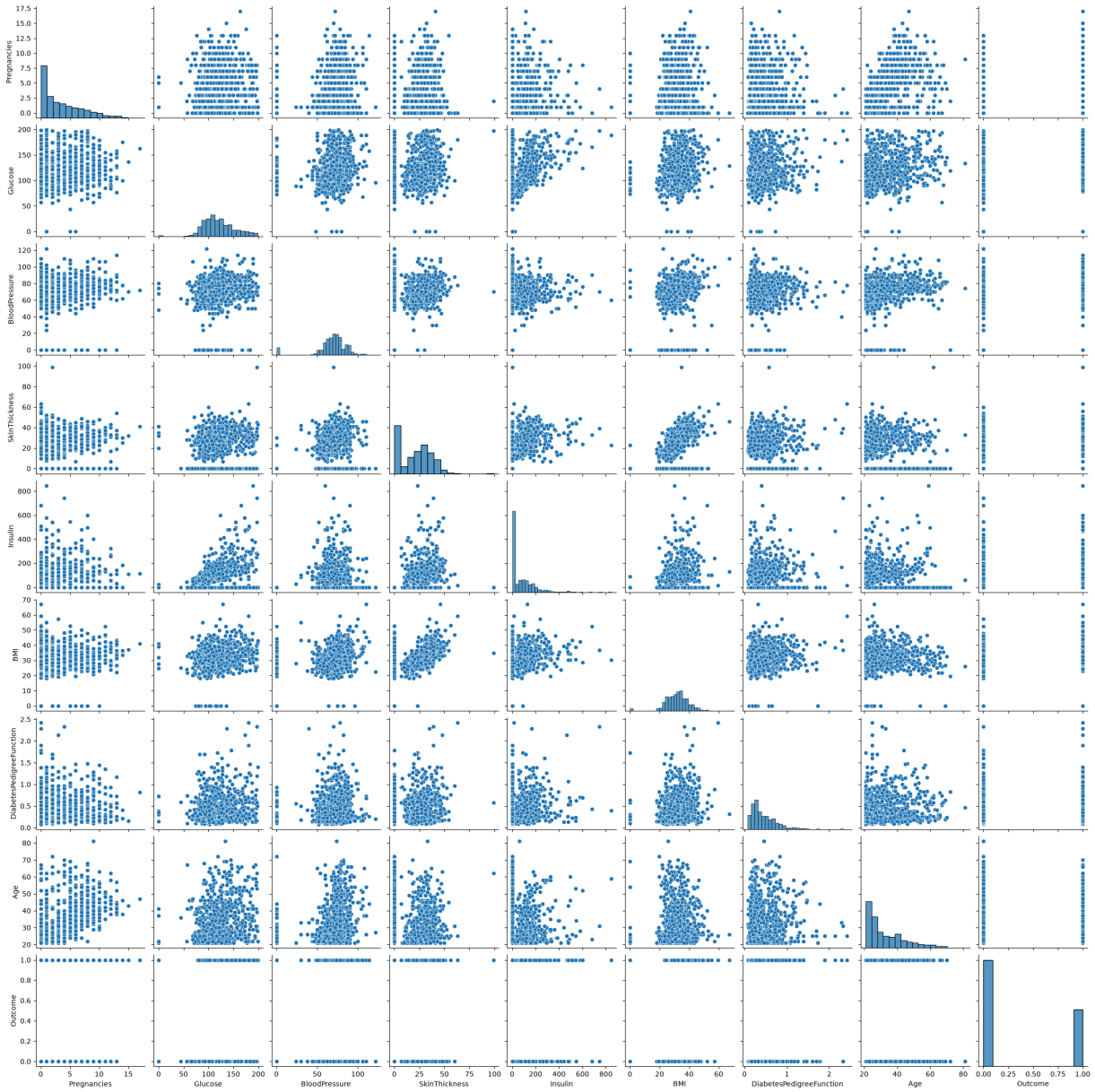
   DiabetesPedigreeFunction  Age  Outcome  
0                     0.627   50        1  
1                     0.351   31        0  
2                     0.672   32        1  
3                     0.167   21        0  
4                     2.288   33        1  
```

In [5]:
```
print(db.info())
print(db.describe())
import matplotlib.pyplot as plt
import seaborn as sns
db.hist(bins=50,figsize=(20,15))
plt.show()
sns.pairplot(db)
plt.show()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   Pregnancies               768 non-null    int64  
 1   Glucose                   768 non-null    int64  
 2   BloodPressure             768 non-null    int64  
 3   SkinThickness             768 non-null    int64  
 4   Insulin                   768 non-null    int64  
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64  
 8   Outcome                   768 non-null    int64  
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000   
mean      3.845052  120.894531      69.105469      20.536458   79.799479   
std       3.369578   31.972618      19.355807      15.952218  115.244002   
min       0.000000    0.000000       0.000000       0.000000    0.000000   
25%       1.000000   99.000000      62.000000       0.000000    0.000000   
50%       3.000000  117.000000      72.000000      23.000000   30.500000   
75%       6.000000  140.250000      80.000000      32.000000  127.250000   
max      17.000000  199.000000     122.000000      99.000000  846.000000   

              BMI  DiabetesPedigreeFunction         Age     Outcome  
count  768.000000                768.000000  768.000000  768.000000  
mean    31.992578                  0.471876   33.240885    0.348958  
std      7.884160                  0.331329   11.760232    0.476951  
min      0.000000                  0.078000   21.000000    0.000000  
25%     27.300000                  0.243750   24.000000    0.000000  
50%     32.000000                  0.372500   29.000000    0.000000  
75%     36.600000                  0.626250   41.000000    1.000000  
max     67.100000                  2.420000   81.000000    1.000000
```


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome


Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome

In [ ]:

Experiment 6: Time Series Analysis of Website Traffic Objective: To analyze the time series data of website traffic to identify trends, seasonality, and anomalies.

In [4]:
```
import scipy
import statsmodels
from statsmodels.tsa.seasonal import seasonal_decompose

print("SciPy:", scipy.__version__)
print("Statsmodels:", statsmodels.__version__)
print("Seasonal decomposition is working!")
```

```
SciPy: 1.18.1
Statsmodels: 0.15.0
Seasonal decomposition is working!
```

In [2]:
```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

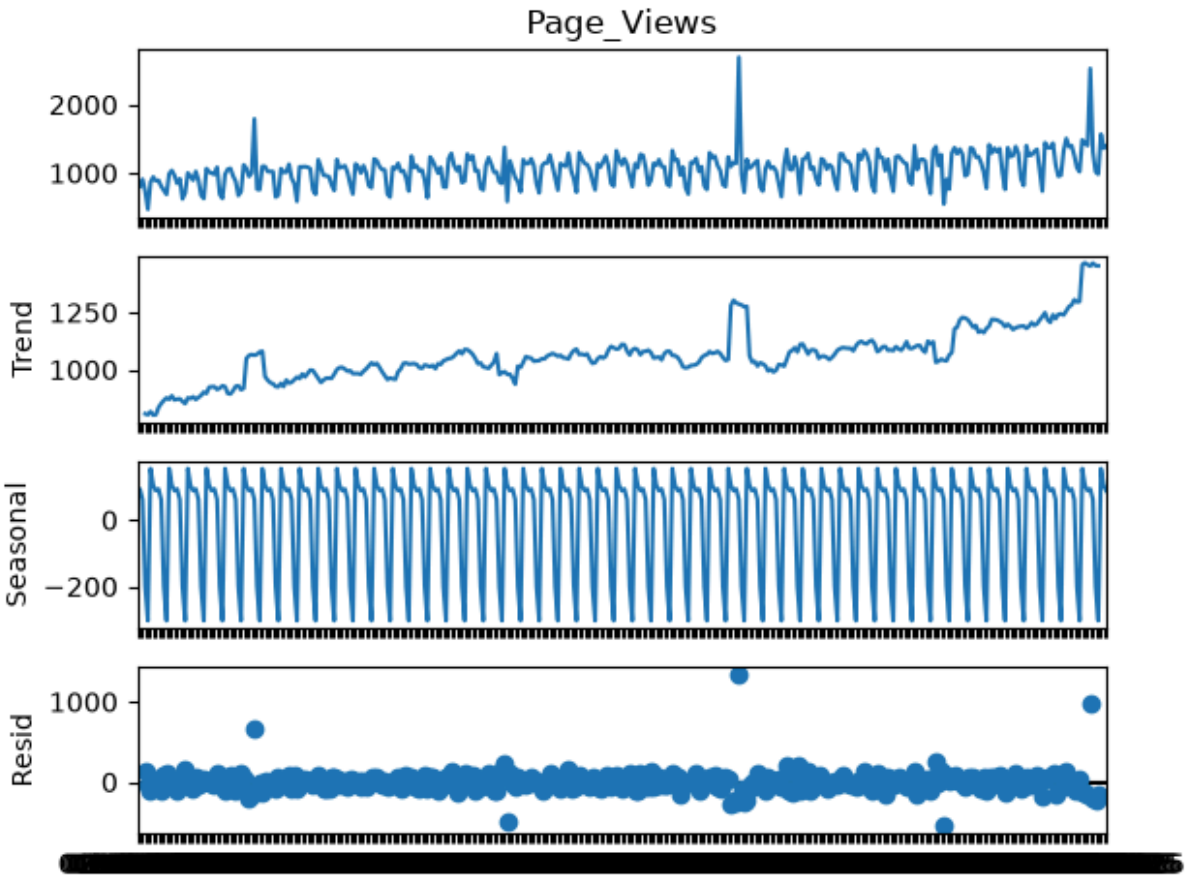
# Load dataset
data = pd.read_csv(
    r'C:\Users\jaiwa\Downloads\website_traffic.csv',
    parse_dates=['Date'],
    index_col='Date'
)

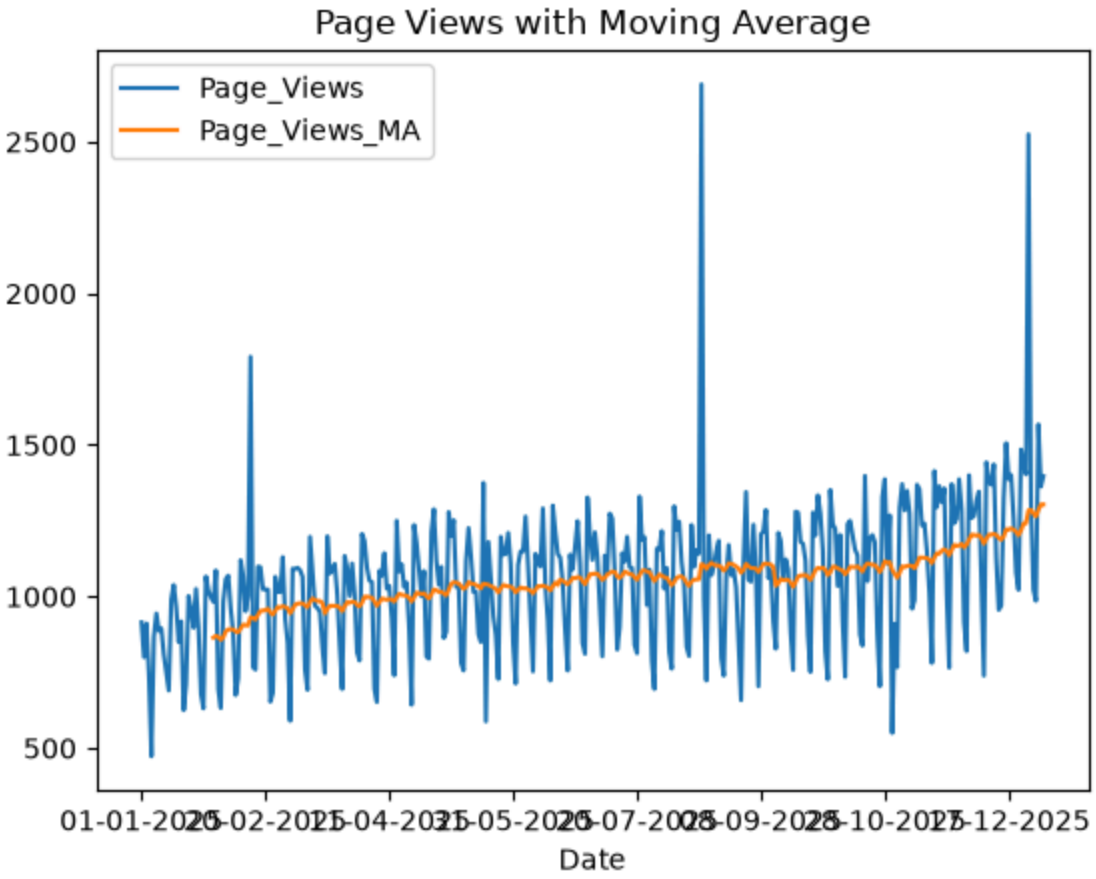
# Time Series Decomposition
decomposition = seasonal_decompose(
    data['Page_Views'],
    model='additive',
    period=7
)

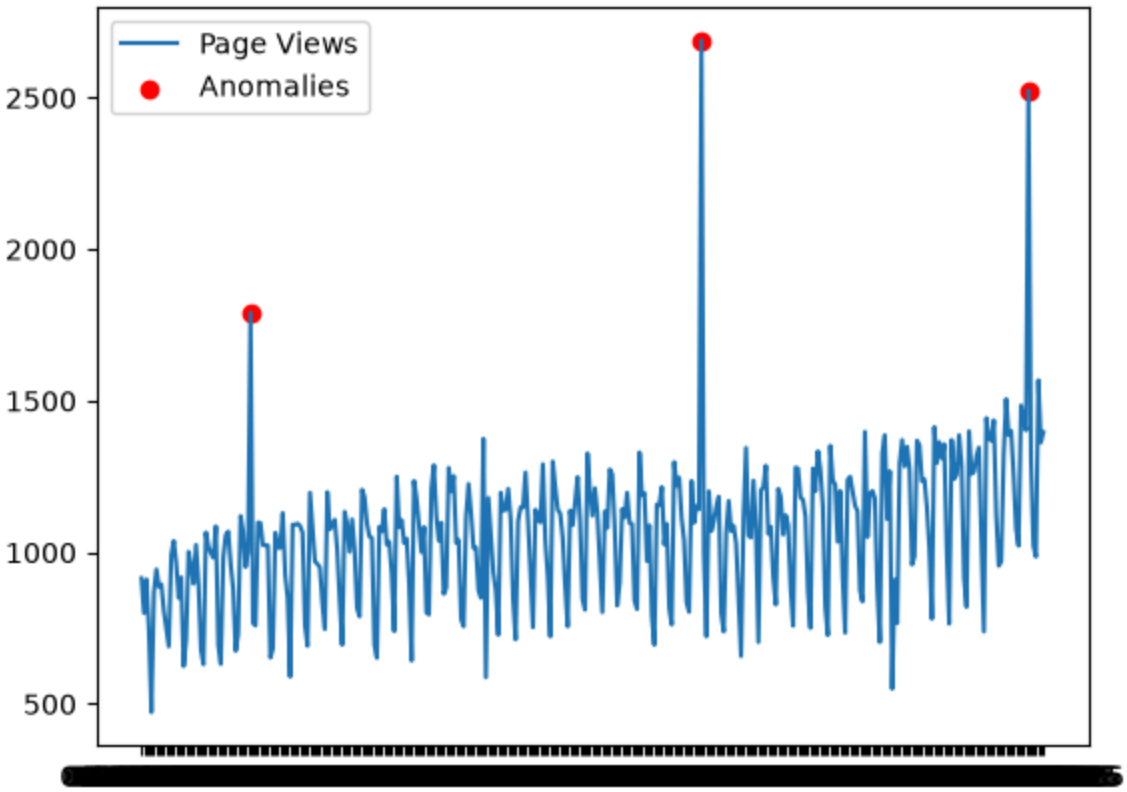
# Plot the components
decomposition.plot()
plt.show()


# Moving Average
data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()

# Anomaly Detection
data['Page_Views_Z'] = (data['Page_Views'] - data['Page_Views'].mean()) /dat
anomalies = data[data['Page_Views_Z'].abs() > 3]
plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red', label='An
plt.legend()
plt.show()
```


Page_Views
2000
1000
Trend
1250
1000
Seasonal
0
−200
Resid
1000
0


Page Views with Moving Average
Page_Views
Page_Views_MA
2500
2000
1500
1000
500
01-01-2025
01-02-2025
01-04-2025
01-05-2025
01-07-2025
01-09-2025
01-10-2025
01-12-2025
Date


Page Views
Anomalies
2500
2000
1500
1000
500

Experiment 7:. Random Sampling and Sampling Distribution Objective: To explore random sampling from a population and understand the concept of sampling distribution using Python in Jupyter Notebook.

In [8]:
```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate population
np.random.seed(42)

population_mean = 50
population_std = 10
population_size = 100000

population = np.random.normal(
    population_mean, population_std, population_size
)

# Step 2: Random sampling
sample_sizes = [30, 50, 100]
num_samples = 1000

sample_means = {}

for size in sample_sizes:
    sample_means[size] = []

    for _ in range(num_samples):
        sample = np.random.choice(
            population, size=size, replace=False
        )
        sample_means[size].append(np.mean(sample))

# Step 3: Plot sampling distributions
plt.figure(figsize=(12, 8))

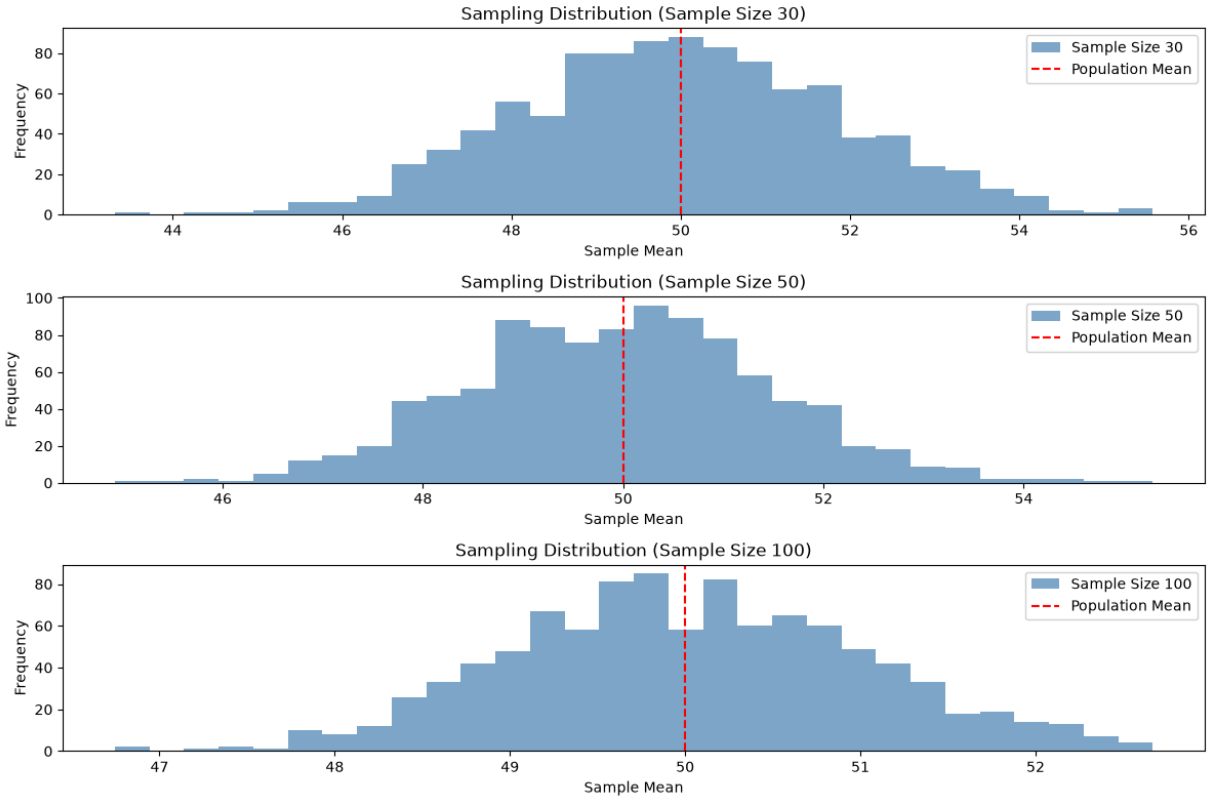
for i, size in enumerate(sample_sizes):
    plt.subplot(3, 1, i + 1)

    plt.hist(
        sample_means[size],
        bins=30,
        alpha=0.7,
        color='steelblue',
        label=f'Sample Size {size}'
    )

    plt.axvline(
        population_mean,
        color='red',
        linestyle='dashed',
        linewidth=1.5,
        label='Population Mean'
    )
```

```
    plt.title(f'Sampling Distribution (Sample Size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()

plt.tight_layout()
plt.show()
```



Experiment 8: Conduct Z test for the Data Given using Python Objective: To test whether the average weight of a species of birds differs from 150 grams.

In [3]:
```
import numpy as np
import scipy.stats as stats

# Define the sample data (hypothetical weights in grams)
sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152, 149,151
148, 150, 152, 149, 150,148, 153, 151, 150, 149, 152, 148, 151, 150, 153])
# Population mean under the null hypothesis
population_mean = 150
# Calculate sample statistics
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1) # Using sample standard deviation

# Number of observations
n = len(sample_data)
# Calculate the Z-statistic
z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))

# Calculate the p-value

p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic))) # Two-tailed test

# Print results
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")
# Decision based on the significance level
alpha = 0.05
if p_value < alpha:
 print("Reject the null hypothesis: The average weight is significantly diff
else:
 print("Fail to reject the null hypothesis: There is no significant differen
```

```
Sample Mean: 150.20
Z-Statistic: 0.6406
P-Value: 0.5218
Fail to reject the null hypothesis: There is no significant difference in ave
rage weight from 150 grams.
```

Exp No:9 Experiment to understand Matplotlib library use cases in Data Science through visualization Description: Use Iris data set to understand the Matplot lib library

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

%matplotlib inline

data = pd.read_csv(r'C:\Users\jaiwa\Downloads\iris.csv')

# Rename columns to match your experiment code
data.columns = [
    'SepalLengthCm',
    'SepalWidthCm',
    'PetalLengthCm',
    'PetalWidthCm',
    'variety'
]

data.info()
print(data.describe())
print(data.value_counts('variety'))

sns.countplot(x='variety', data=data)
plt.show()

dummies = pd.get_dummies(data['variety'])

FinalDataset = pd.concat(
    [dummies, data.iloc[:, [0, 1, 2, 3]]],
    axis=1
)

print(FinalDataset.head())

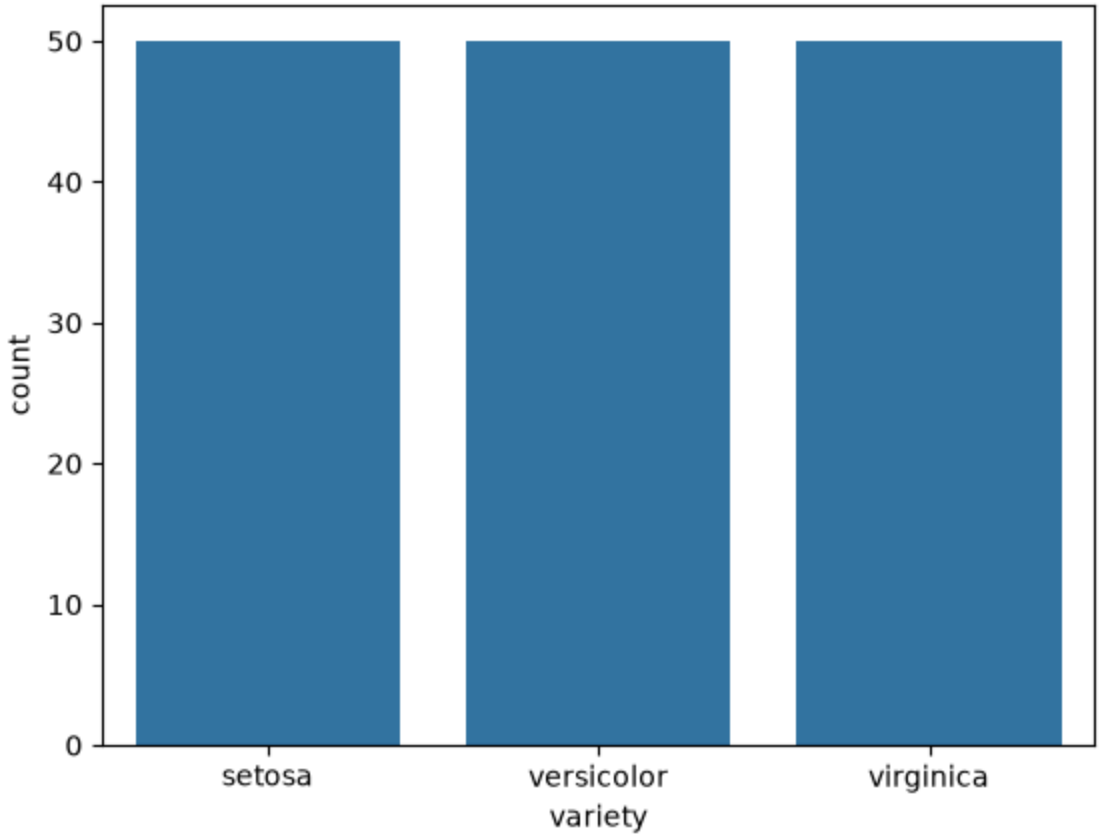
sns.scatterplot(
    x='PetalLengthCm',
    y='PetalWidthCm',
    hue='variety',
    data=data
)
plt.show()

sns.pairplot(data, hue='variety', height=3)
plt.show()

for column in ['PetalLengthCm', 'PetalWidthCm',
               'SepalLengthCm', 'SepalWidthCm']:
    sns.FacetGrid(data, hue='variety', height=5).map(
        sns.histplot, column
```

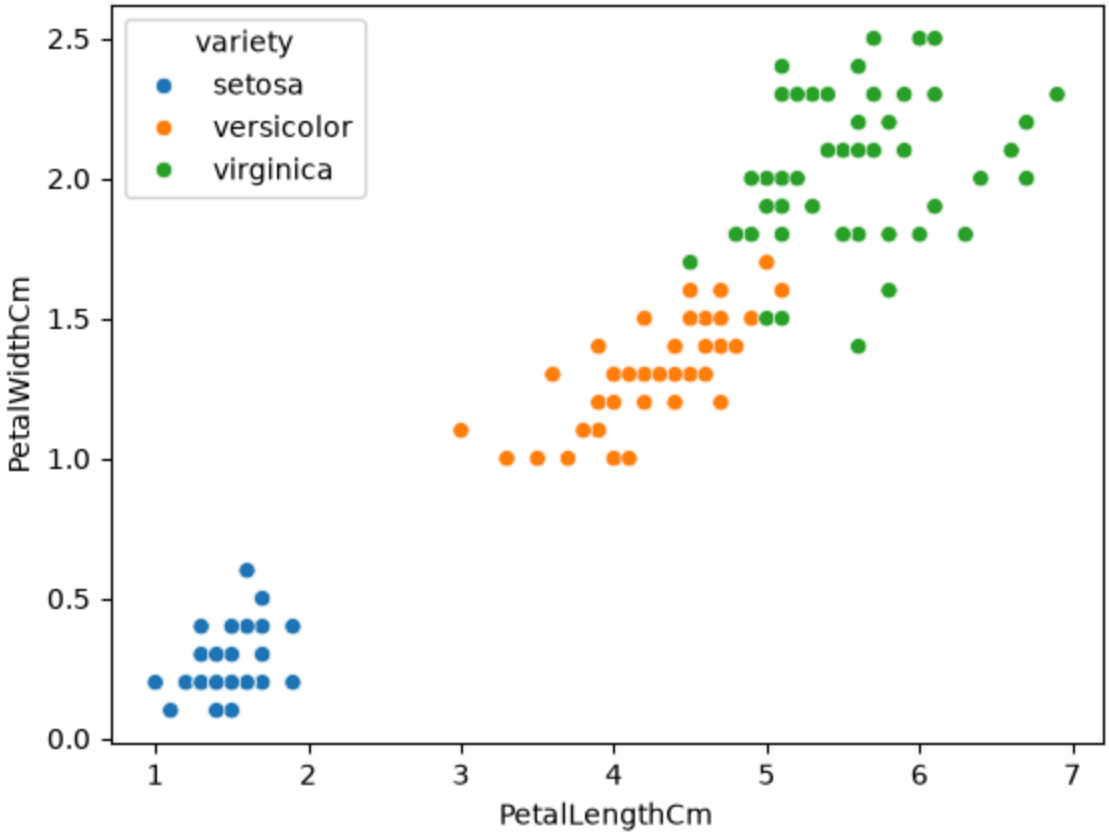
```
    ).add_legend()
    plt.show()
```

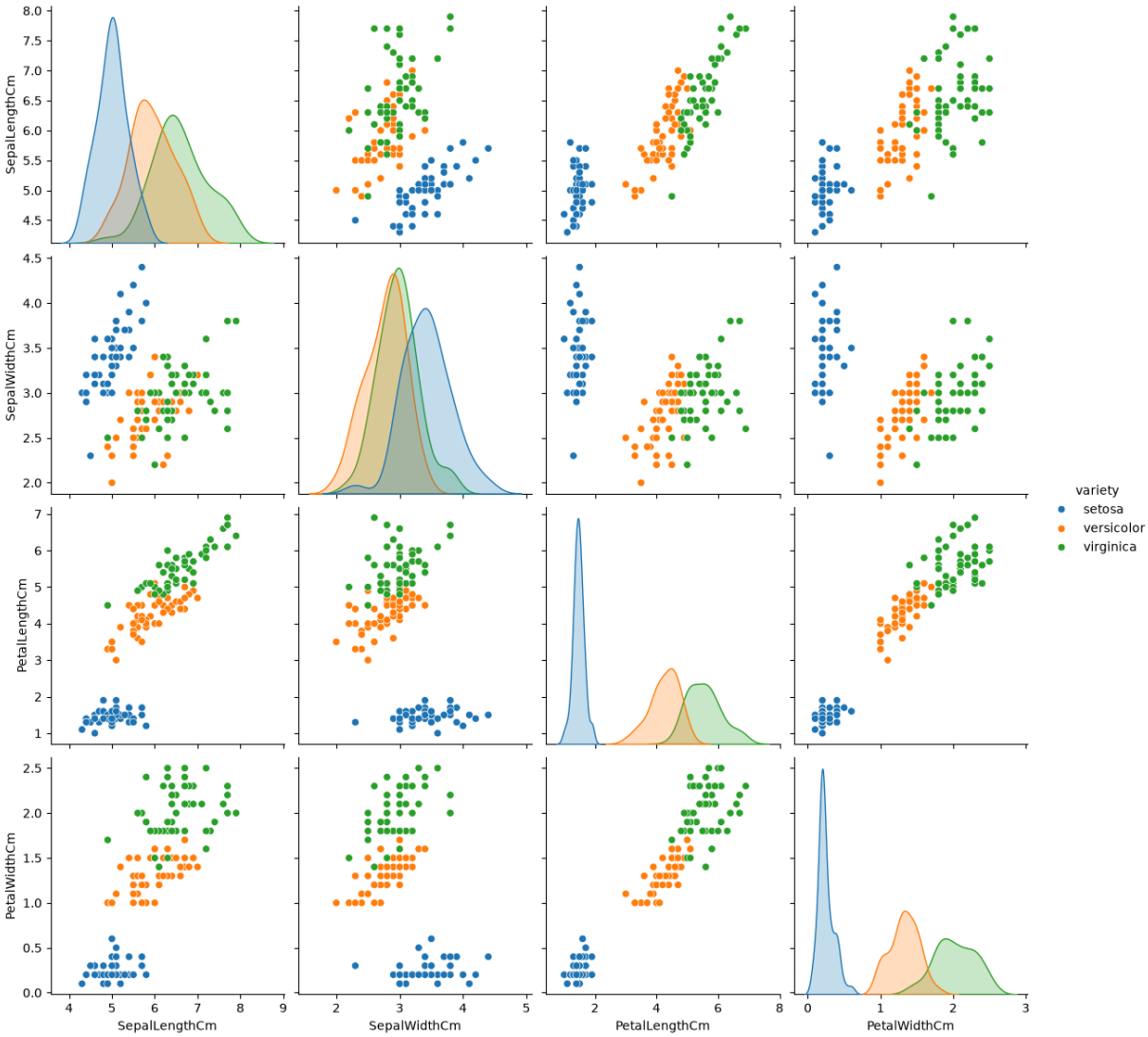
```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   SepalLengthCm  150 non-null    float64
 1   SepalWidthCm   150 non-null    float64
 2   PetalLengthCm  150 non-null    float64
 3   PetalWidthCm   150 non-null    float64
 4   variety        150 non-null    str
dtypes: float64(4), str(1)
memory usage: 6.0 KB
       SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
count     150.000000    150.000000     150.000000    150.000000
mean        5.843333      3.057333       3.758000      1.199333
std         0.828066      0.435866       1.765298      0.762238
min         4.300000      2.000000       1.000000      0.100000
25%         5.100000      2.800000       1.600000      0.300000
50%         5.800000      3.000000       4.350000      1.300000
75%         6.400000      3.300000       5.100000      1.800000
max         7.900000      4.400000       6.900000      2.500000
variety
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```

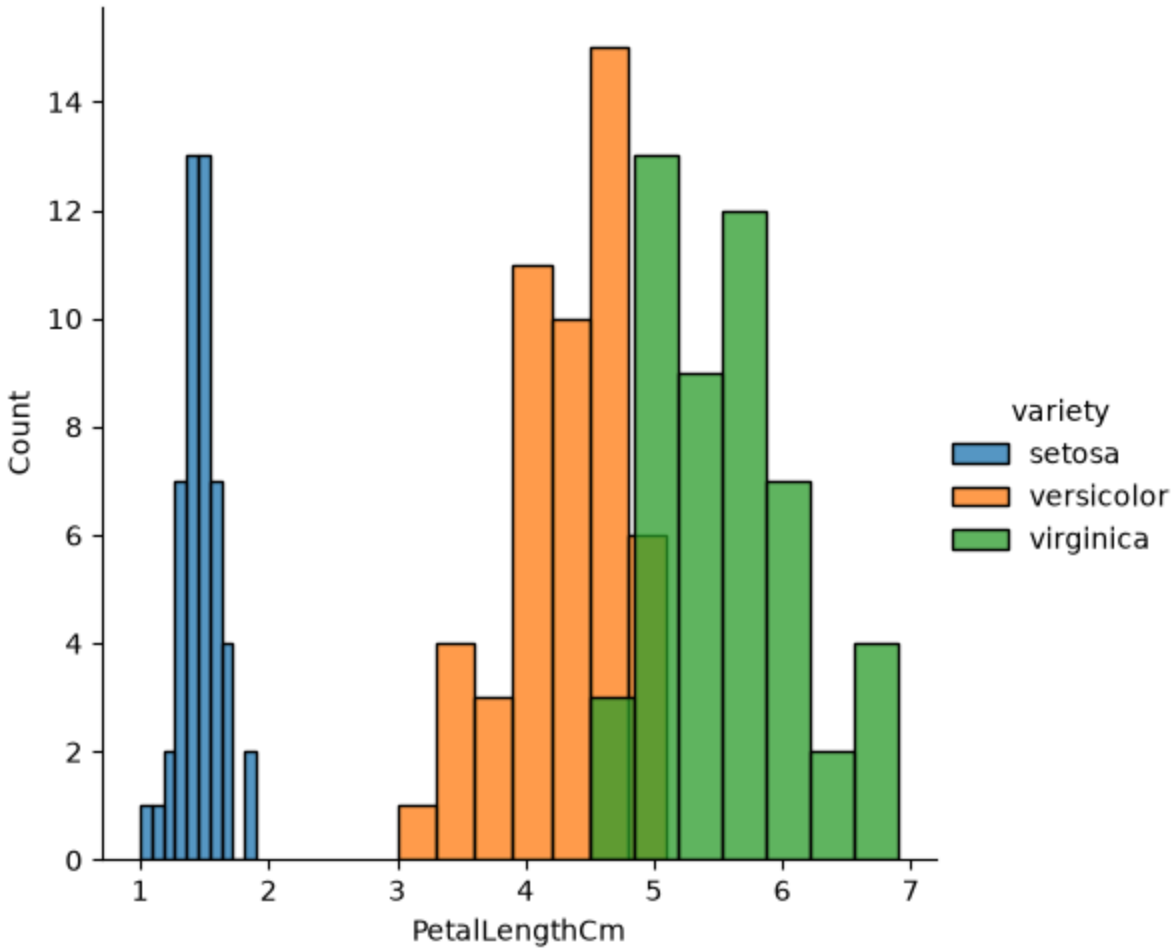


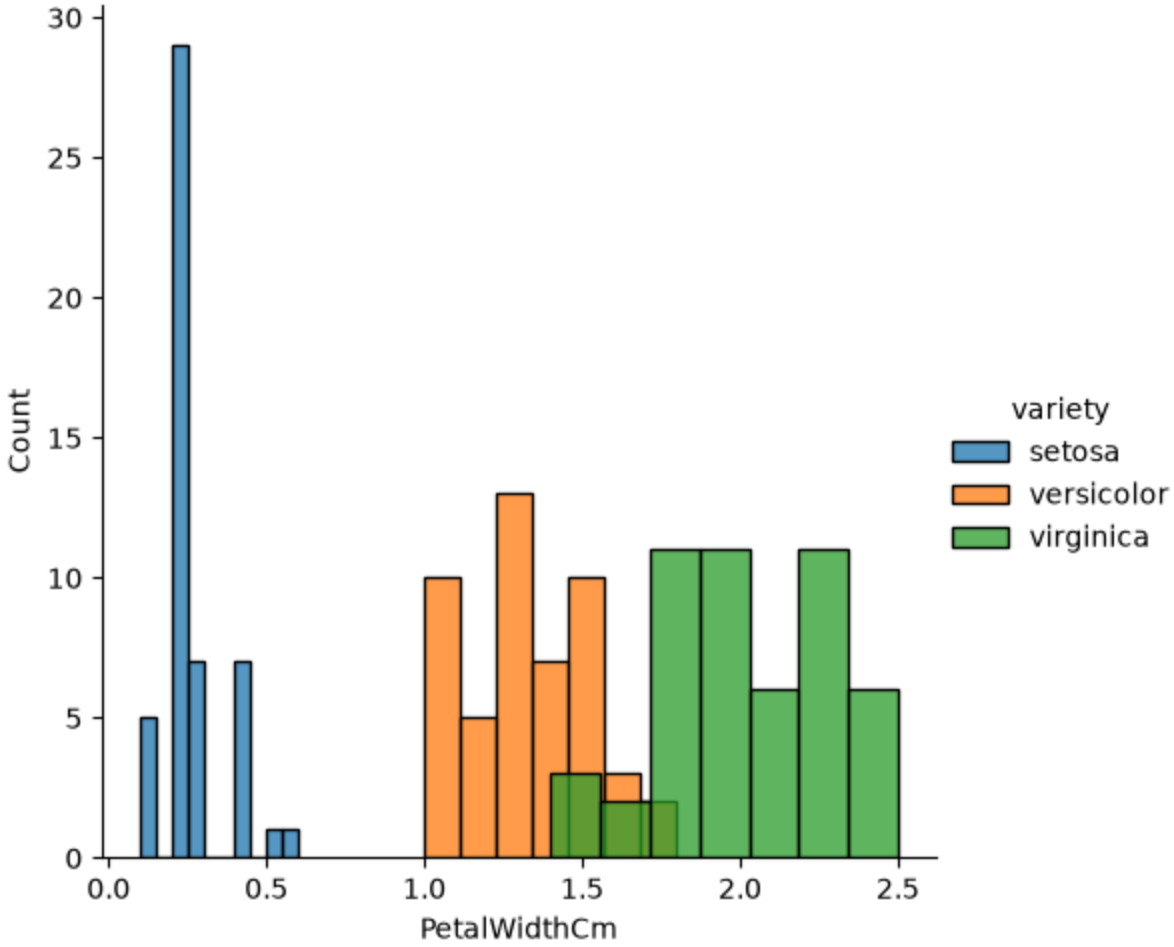
```
   setosa  versicolor  virginica  SepalLengthCm  SepalWidthCm  PetalLengthCm
\
0    True       False      False            5.1           3.5            1.4
1    True       False      False            4.9           3.0            1.4
2    True       False      False            4.7           3.2            1.3
3    True       False      False            4.6           3.1            1.5
4    True       False      False            5.0           3.6            1.4

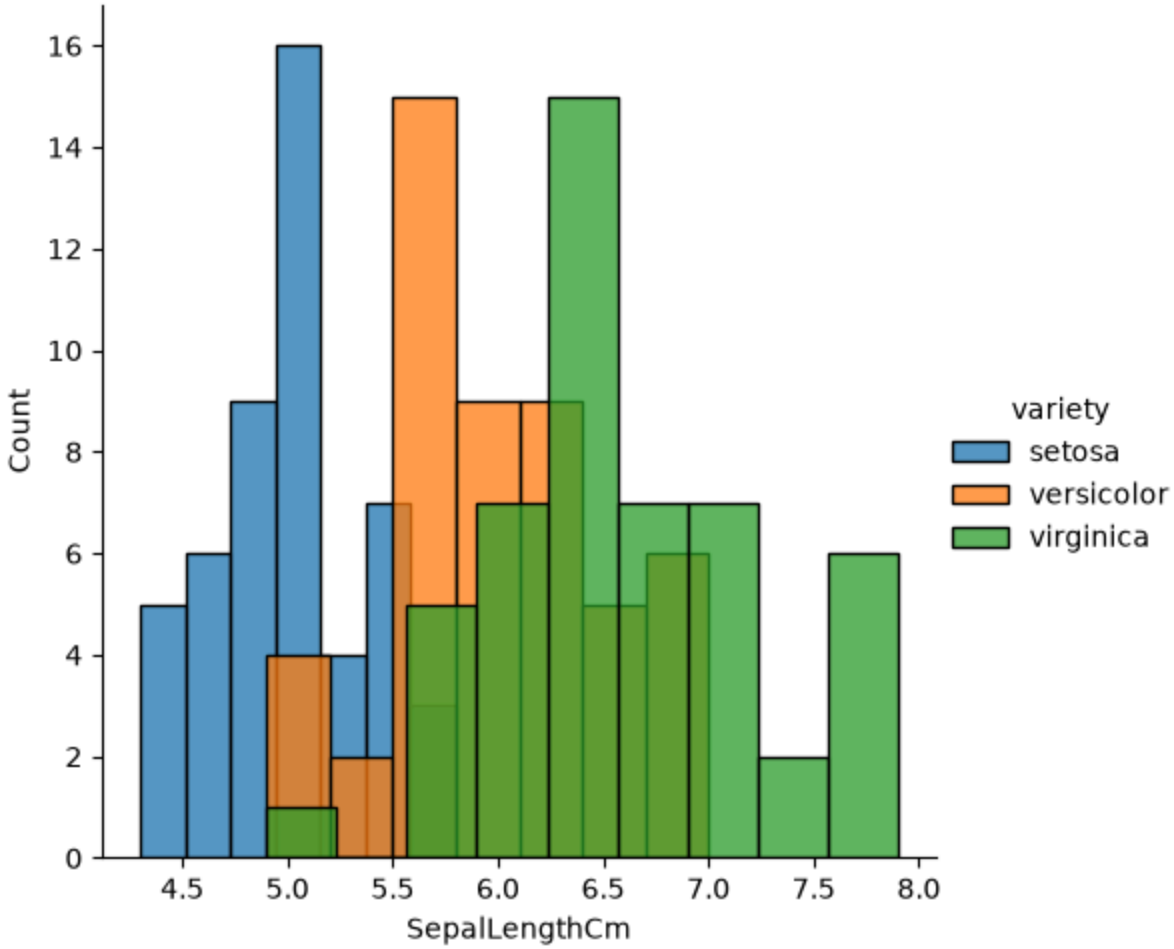
   PetalWidthCm
0           0.2
1           0.2
2           0.2
3           0.2
4           0.2
```

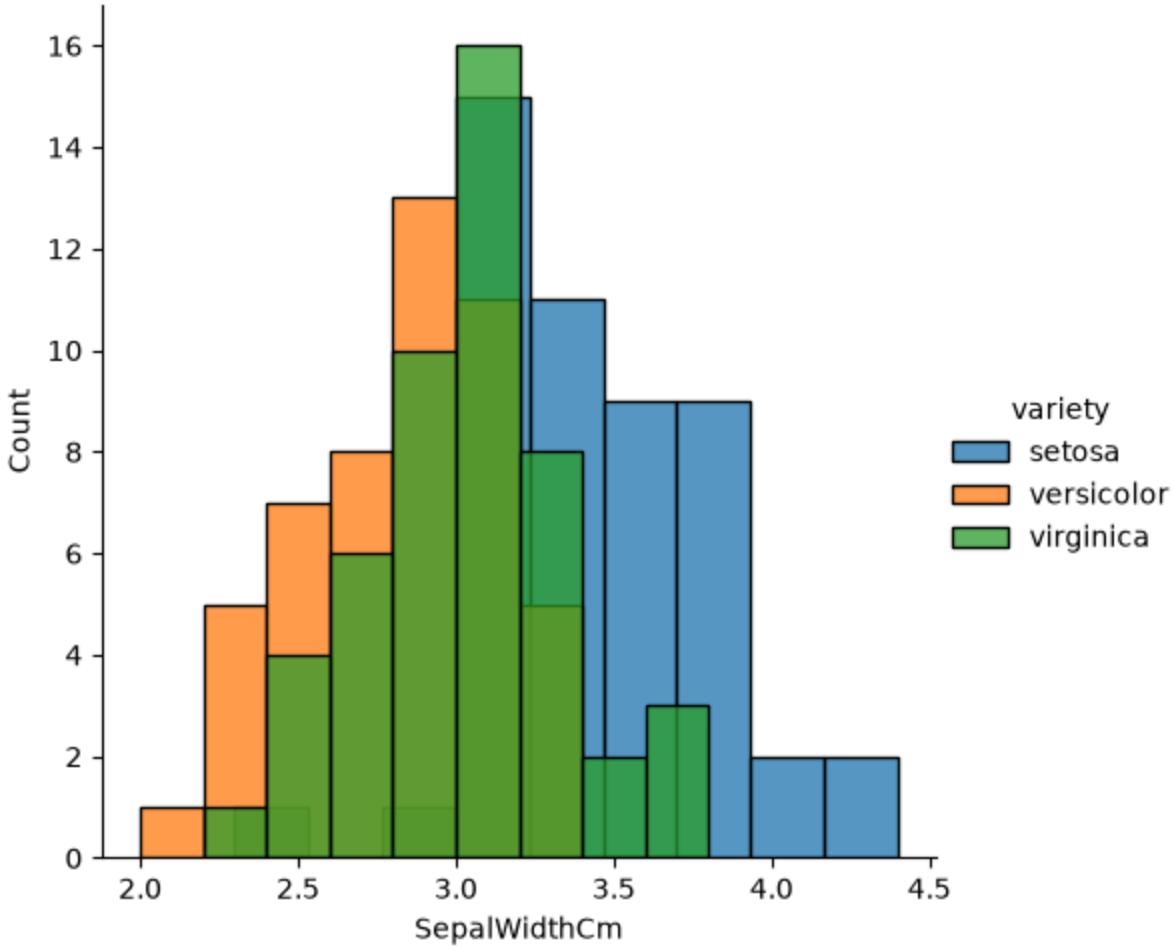



SepalLengthCm
SepalWidthCm
PetalLengthCm
PetalWidthCm
variety
setosa
versicolor
virginica


Count
PetalLengthCm
variety
setosa
versicolor
virginica


30
25
20
15
10
5
0
Count
0.0
0.5
1.0
1.5
2.0
2.5
PetalWidthCm
variety
setosa
versicolor
virginica


Count
16
14
12
10
8
6
4
2
0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0
SepalLengthCm
variety
setosa
versicolor
virginica


16
14
12
10
8
6
4
2
0
Count
2.0
2.5
3.0
3.5
4.0
4.5
SepalWidthCm
variety
setosa
versicolor
virginica

Exp No:10 Experiment to understand array function in Data science.

Description: Understand array function using Numpy library

In [1]:
```
import numpy as np
array=np.random.randint(1,100,9)
array
```

Out[1]:
```
array([80, 28, 84, 98, 83, 41, 49, 30, 77], dtype=int32)
```

In [2]:
```
np.sqrt(array)
```

Out[2]:
```
array([8.94427191, 5.29150262, 9.16515139, 9.89949494, 9.11043358,
       6.40312424, 7.        , 5.47722558, 8.77496439])
```

In [3]:
```
array.ndim
```

Out[3]:
```
1
```

In [4]:
```
new_array=array.reshape(3,3)
new_array
```

Out[4]:
```
array([[80, 28, 84],
       [98, 83, 41],
       [49, 30, 77]], dtype=int32)
```

In [5]:
```
new_array.ndim
```

Out[5]:
```
2
```

In [6]:
```
new_array.ravel()
```

Out[6]:
```
array([80, 28, 84, 98, 83, 41, 49, 30, 77], dtype=int32)
```

In [7]:
```
newm=new_array.reshape(3,3)
newm
```

Out[7]:
```
array([[80, 28, 84],
       [98, 83, 41],
       [49, 30, 77]], dtype=int32)
```

In [8]:
```
newm[2,1:3]
```

Out[8]:
```
array([30, 77], dtype=int32)
```

In [9]:
```
newm[1:2,1:3]
```

Out[9]:
```
array([[83, 41]], dtype=int32)
```

In [10]:
```
new_array[0:3,0:0]
```

Out[10]:
```
array([], shape=(3, 0), dtype=int32)
```

In [11]:
```
new_array[0:2,0:1]
```

Out[11]:
```
array([[80],
       [98]], dtype=int32)
```

In [12]:
```
new_array[0:3,0:1]
```

Out[12]:
```
array([[80],
       [98],
       [49]], dtype=int32)
```

In [13]:
```
new_array[1:3]
```

Out[13]:
```
array([[98, 83, 41],
       [49, 30, 77]], dtype=int32)
```

Exp No:11 Experiment to understand pandas library use cases in Data science.
Description: Understand data frame use cases using pandas library

In [1]:
```
import numpy as np
import pandas as pd
list=[[1,'Smith',50000],[2,'Jones',60000]]
df=pd.DataFrame(list)
df
```

Out[1]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>0</th> <th>1</th> <th>2</th> </thead> <tbody> <th>0</th> <th>1</th> </tbody> <tbody> </tbody>

1	Smith	50000
2	Jones	60000

In [2]:
```
df.columns=['Empd','Name','Salary']
df
```

Out[2]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Empd</th> <th>Name</th> <th>Salary</th> </thead> <tbody> <th>0</th> <th>1</th> </tbody> <tbody></tbody>

1	Smith	50000
2	Jones	60000

In [3]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      str
 2   Salary  2 non-null      int64
dtypes: int64(2), str(1)
memory usage: 180.0 bytes
```

In [4]:
```
df=pd.read_csv(r"C:\Users\jaiwa\Downloads\50_Startups.csv")
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     str    
 4   Profit           50 non-null     float64
dtypes: float64(4), str(1)
memory usage: 2.1 KB
```

In [5]:
```
df.head()
```

Out[5]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>R&D Spend</th> <th>Administration</th> <th>Marketing Spend</th> <th>State</th> <th>Profit</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

In [6]:
```
df.tail()
```

Out[6]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>R&D Spend</th> <th>Administration</th> <th>Marketing Spend</th> <th>State</th> <th>Profit</th> </thead> <tbody> <th>45</th> <th>46</th> <th>47</th> <th>48</th> <th>49</th> </tbody> <tbody></tbody>

45	1000.23	124153.04	1903.93	New York	64926.08
46	1315.46	115816.21	297114.46	Florida	49490.75
47	0.00	135426.92	0.00	California	42559.73
48	542.05	51743.15	0.00	New York	35673.41
49	0.00	116983.80	45173.06	California	14681.40

In [7]:
```
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\jaiwa\Downloads\employee.csv")
```

In [8]:
```
df.head()
```

Out[8]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>emp id</th> <th>name</th> <th>salary</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1		SREE VARSSINI K S	5000
2		SREEMATHI B	6000
3		SREYA G	7000
4		SREYASKARI MULLAPUDI	5000
5		SRI AKASH U G	8000

In [9]:
```
df.tail()
```

Out[9]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>emp id</th> <th>name</th> <th>salary</th> </thead> <tbody> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

3		SREYA G	7000
4		SREYASKARI MULLAPUDI	5000
5		SRI AKASH U G	8000
6		SRI HARSHAVARDHANAN R	3000
7		SRI HARSHAVARDHANAN R	6000

In [10]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  7 non-null      int64
 1   name    7 non-null      str
 2   salary  7 non-null      int64
dtypes: int64(2), str(1)
memory usage: 300.0 bytes
```

In [11]:
```
df.salary
```

Out[11]:
```
0    5000
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: salary, dtype: int64
```

In [12]:
```
type(df.salary)
```

Out[12]:
```
pandas.Series
```

In [14]:
```
df.salary.mean()
```

Out[14]:
```
np.float64(5714.285714285715)
```

In [15]:
```
df.salary.median()
```

Out[15]:
```
np.float64(6000.0)
```

In [16]:
```
df.salary.mode()
```

Out[16]:
```
0    5000
1    6000
Name: salary, dtype: int64
```

In [17]:
```
df.salary.var()
```

Out[17]:
```
np.float64(2571428.5714285714)
```

In [18]:
```
df.salary.std()
```

Out[18]:
```
np.float64(1603.5674514745463)
```

In [20]:
```
df.describe()
```

Out[20]:
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>emp id</th> <th>salary</th> </thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody></tbody>

7.000000	7.000000	
4.000000	5714.285714	
2.160247	1603.567451	
1.000000	3000.000000	
2.500000	5000.000000	
4.000000	6000.000000	
5.500000	6500.000000	
7.000000	8000.000000	

In [21]:
```
df.describe(include='all')
```

Out[21]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>emp id</th> <th>name</th> <th>salary</th> </thead> <tbody> <th>count</th> <th>unique</th> <th>top</th> <th>freq</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody></tbody>

7.000000	7	7.000000
NaN	6	NaN
NaN	SRI HARSHAVARDHANAN R	NaN
NaN	2	NaN
4.000000	NaN	5714.285714
2.160247	NaN	1603.567451
1.000000	NaN	3000.000000
2.500000	NaN	5000.000000
4.000000	NaN	6000.000000
5.500000	NaN	6500.000000
7.000000	NaN	8000.000000

In [22]:
```
empCol=df.columns
empCol
```

Out[22]:
```
Index(['emp id', 'name', 'salary'], dtype='str')
```

In [23]:
```
emparray=df.values
emparray
```

Out[23]:
```
array([[1, 'SREE VARSSINI K S', 5000],
       [2, 'SREEMATHI B', 6000],
       [3, 'SREYA G', 7000],
       [4, 'SREYASKARI MULLAPUDI', 5000],
       [5, 'SRI AKASH U G', 8000],
       [6, 'SRI HARSHAVARDHANAN R', 3000],
       [7, 'SRI HARSHAVARDHANAN R', 6000]], dtype=object)
```

In [24]:
```
employee_DF=pd.DataFrame(emparray, columns=empCol)

employee_DF
```

Out[24]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>emp id</th> <th>name</th> <th>salary</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody></tbody>

1	SREE VARSSINI K S	5000
2	SREEMATHI B	6000
3	SREYA G	7000
4	SREYASKARI MULLAPUDI	5000
5	SRI AKASH U G	8000
6	SRI HARSHAVARDHANAN R	3000
7	SRI HARSHAVARDHANAN R	6000

Exp No:12 Experiment to detect outliers in a given data set. Description: Understand the procedure to identify the outliers in a given dataset

In [2]:
```
#sample calculation for low range(lr) , upper range (ur),percentile
import numpy as np
array=np.random.randint(1,100,16) # randomly generate 16 numbers between 1 t
array
```

Out[2]:
```
array([74, 70,  6, 18, 16, 25, 64, 30, 52, 66, 55, 36, 98, 66, 81, 62],
      dtype=int32)
```

In [3]:
```
array.mean()
```

Out[3]:
```
np.float64(51.1875)
```

In [4]:
```
np.percentile(array,25)
```

Out[4]:
```
np.float64(28.75)
```

In [5]:
```
np.percentile(array,50)
```

Out[5]:
```
np.float64(58.5)
```

In [6]:
```
np.percentile(array,75)
```

Out[6]:
```
np.float64(67.0)
```

In [7]:
```
np.percentile(array,100)
```

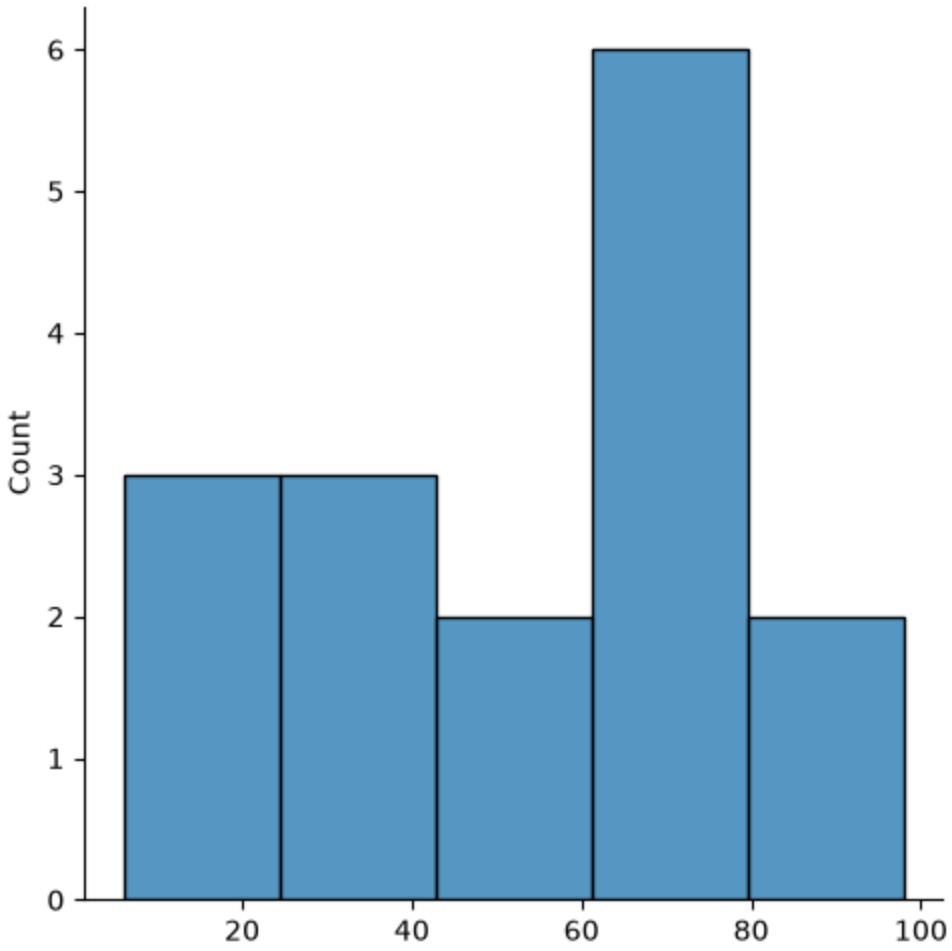
Out[7]:
```
np.float64(98.0)
```

In [8]:
```
#outliers detection
def outDetection(array):
 sorted(array)
 Q1,Q3=np.percentile(array,[25,75])
 IQR=Q3-Q1
 lr=Q1-(1.5*IQR)
 ur=Q3+(1.5*IQR)
 return lr,ur
lr,ur=outDetection(array)
lr,ur
```

Out[8]:
```
(np.float64(-28.625), np.float64(124.375))
```

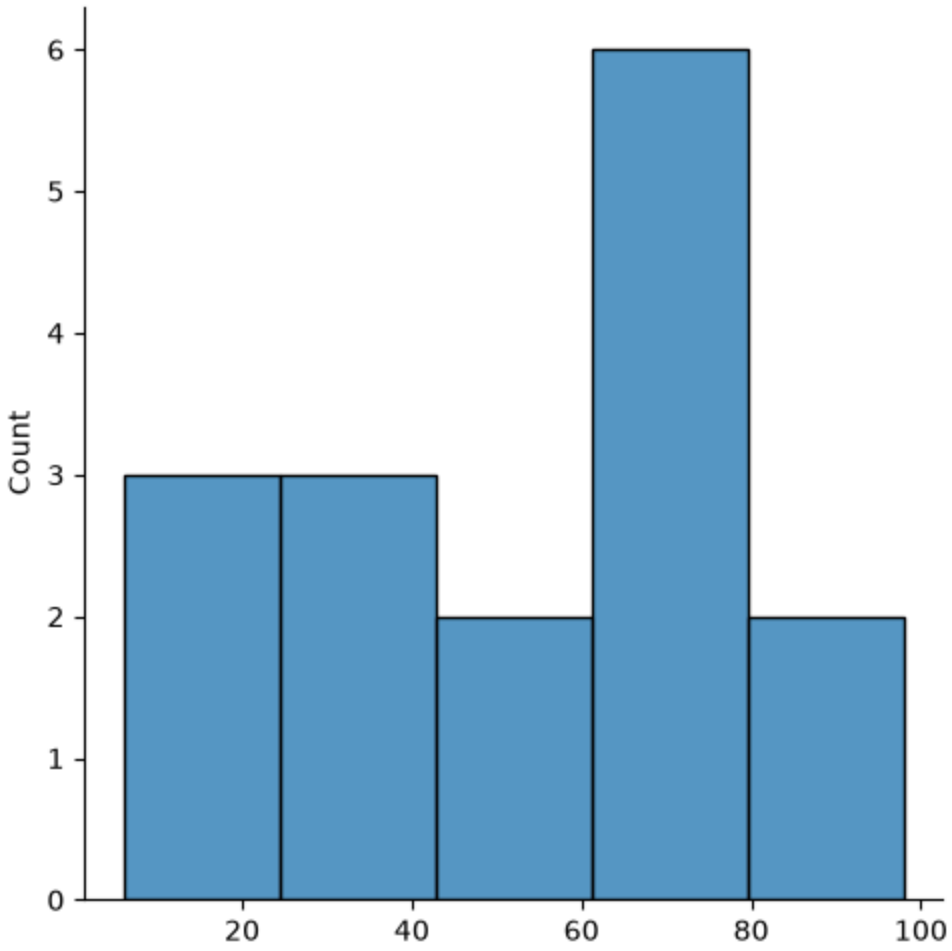
In [9]:
```
import seaborn as sns
%matplotlib inline
sns.displot(array)
```

Out[9]:
```
<seaborn.axisgrid.FacetGrid at 0x1c562e99be0>
```



In [10]:
```
sns.displot(array)
```

Out[10]: <seaborn.axisgrid.FacetGrid at 0x1c563fb3b10>

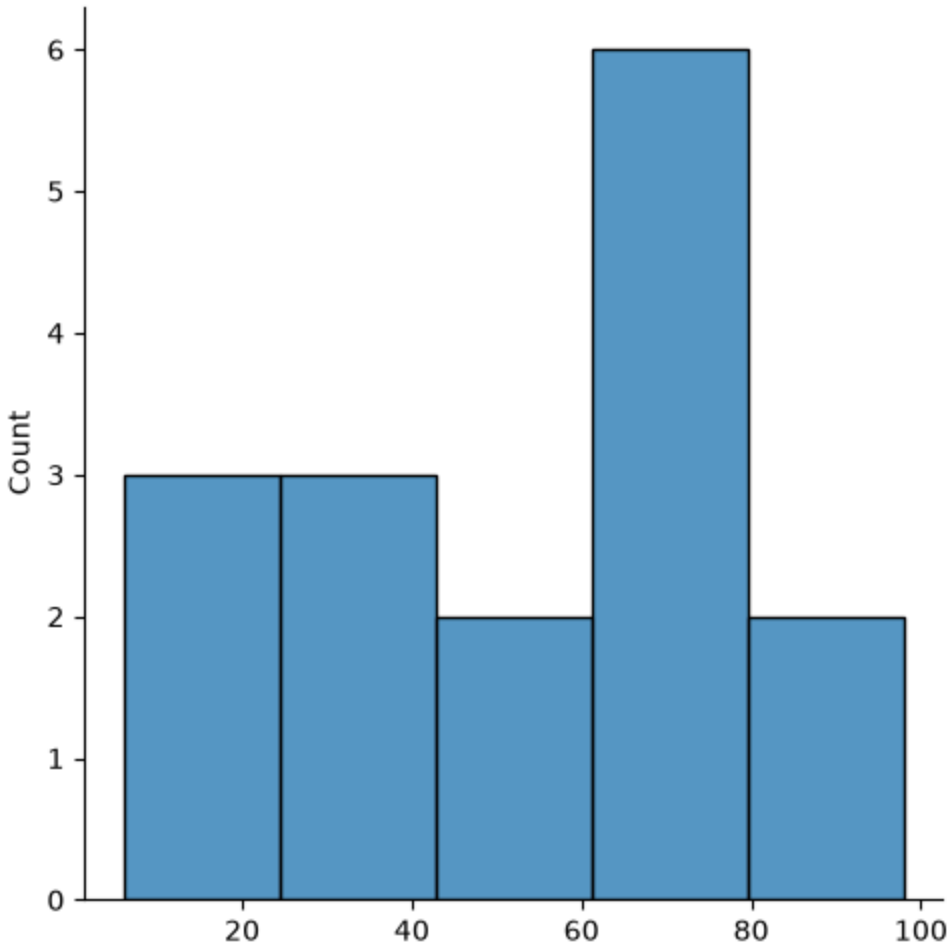


In [11]:
```
new_array=array[(array>lr) & (array<ur)]
new_array
```

Out[11]:
```
array([74, 70,  6, 18, 16, 25, 64, 30, 52, 66, 55, 36, 98, 66, 81, 62],
      dtype=int32)
```

In [12]:
```
sns.displot(new_array)
```

Out[12]:
```
<seaborn.axisgrid.FacetGrid at 0x1c56607ae90>
```



In [13]:
```
lr1,ur1=outDetection(new_array)
lr1,ur1
```

Out[13]:
```
(np.float64(-28.625), np.float64(124.375))
```

In [14]:
```
final_array=new_array[(new_array>lr1) & (new_array<ur1)]
final_array
```

Out[14]:
```
array([74, 70,  6, 18, 16, 25, 64, 30, 52, 66, 55, 36, 98, 66, 81, 62],
      dtype=int32)
```

In [17]:
```
sns.distplot(final_array)
```

```
C:\Users\jaiwa\AppData\Local\Temp\ipykernel_10468\2313597644.py:1: UserWarnin
g:

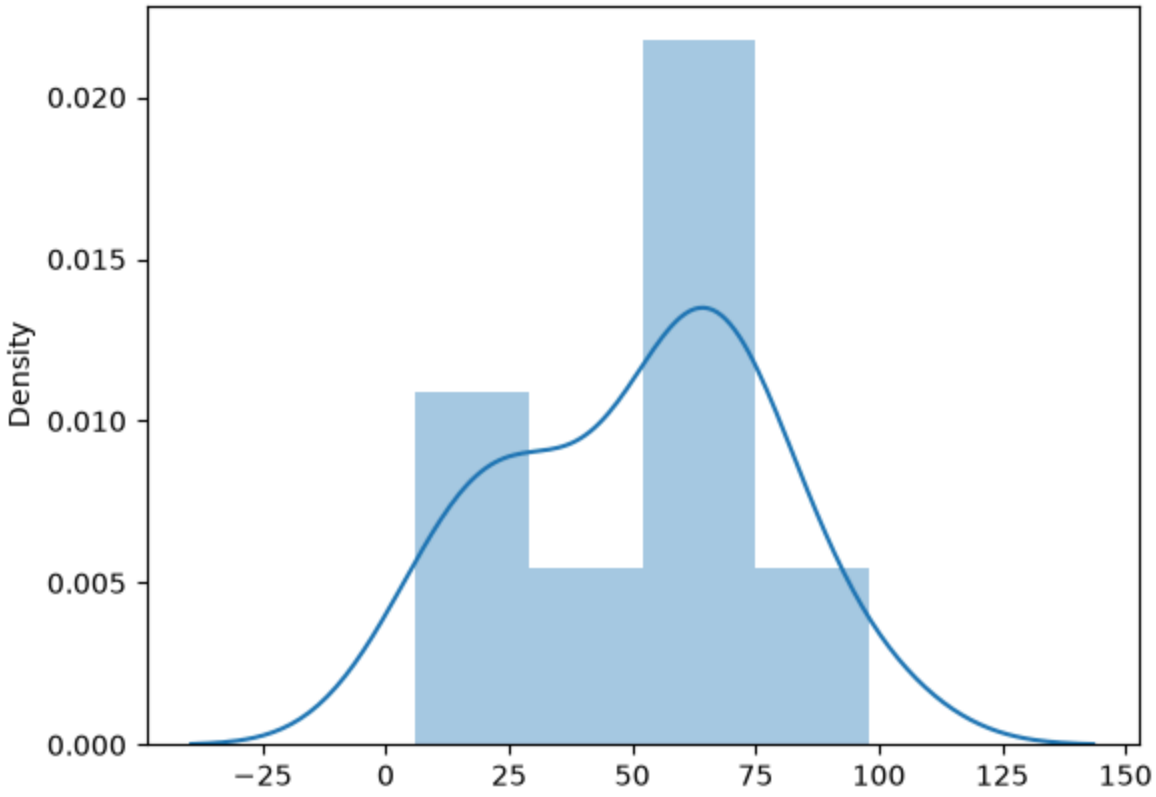
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(final_array)
```

Out[17]: <Axes: ylabel='Density'>



Exp No:13 Experiment to handle missing values and inappropriate data in a given data set Description: Understand the procedure to handle missing and inappropriate data

In [2]:
```
# Upload Hotel.csv and convert it into dataFrame
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\jaiwa\Downloads\Hotel.csv")
df
```

Out[2]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>CustomerID</th> <th>Age_Group</th> <th>Rating(1-5)</th> <th>Hotel</th> <th>FoodPreference</th> <th>Bill</th> <th>NoOfPax</th> <th>EstimatedSalary</th> <th>Age_Group.1</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> </tbody> <tbody></tbody>

1	20-25	4	Ibis	veg	1300	2	40000	20-25
2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
3	25-30	6	RedFox	Veg	1322	2	30000	25-30
4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
5	35+	3	Ibis	Vegetarian	989	2	45000	35+
6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
10	30-35	5	RedFox	non-Veg	6755	4	87777	30-35

In [3]:
```
#From the dataframe identify the duplicate row(i.e row 9)
# The duplicated() method returns a Series with True and False values that c
df.duplicated()
```

Out[3]:
```
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool
```

In [4]:
```
# The info() method prints information about the DataFrame. The information
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     str
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     str
 4   FoodPreference   11 non-null     str
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     str
dtypes: int64(5), str(4)
memory usage: 924.0 bytes
```

In [5]:
```
# The drop_duplicates() method removes duplicate rows.
df.drop_duplicates(inplace=True)
df
```

Out[5]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>CustomerID</th> <th>Age_Group</th> <th>Rating(1-5)</th> <th>Hotel</th> <th>FoodPreference</th> <th>Bill</th> <th>NoOfPax</th> <th>EstimatedSalary</th> <th>Age_Group.1</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>10</th> </tbody> <tbody> </tbody>

1	20-25	4	Ibis	veg	1300	2	40000	20-25
2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
3	25-30	6	RedFox	Veg	1322	2	30000	25-30
4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
5	35+	3	Ibis	Vegetarian	989	2	45000	35+
6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
10	30-35	5	RedFox	non-Veg	6755	4	87777	30-35

In [6]:
```
#While removing duplicate record row index also removed
# The len() function to return the length of an object. With a dataframe, th
len(df)
```

Out[6]: 10

In [7]:
```
#Reset the index
index=np.array(list(range(0,len(df))))
df.set_index(index,inplace=True)
index
```

Out[7]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

In [8]:
```
df
```

Out[8]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>CustomerID</th> <th>Age_Group</th> <th>Rating(1-5)</th> <th>Hotel</th> <th>FoodPreference</th> <th>Bill</th> <th>NoOfPax</th> <th>EstimatedSalary</th> <th>Age_Group.1</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody> </tbody>

1	20-25	4	Ibis	veg	1300	2	40000	20-25
2	30-35	5	LemonTree	Non-Veg	2000	3	59000	30-35
3	25-30	6	RedFox	Veg	1322	2	30000	25-30
4	20-25	-1	LemonTree	Veg	1234	2	120000	20-25
5	35+	3	Ibis	Vegetarian	989	2	45000	35+
6	35+	3	Ibys	Non-Veg	1909	2	122220	35+
7	35+	4	RedFox	Vegetarian	1000	-1	21122	35+
8	20-25	7	LemonTree	Veg	2999	-10	345673	20-25
9	25-30	2	Ibis	Non-Veg	3456	3	-99999	25-30
10	30-35	5	RedFox	non-Veg	6755	4	87777	30-35

In [9]:
```
# Axis refers to the dimensions of a DataFrame (index and columns) or Series
df.drop(['Age_Group.1'],axis=1,inplace=True)
df
```

Out[9]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>CustomerID</th> <th>Age_Group</th> <th>Rating(1-5)</th> <th>Hotel</th> <th>FoodPreference</th> <th>Bill</th> <th>NoOfPax</th> <th>EstimatedSalary</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

1	20-25	4	Ibis	veg	1300	2	40000
2	30-35	5	LemonTree	Non-Veg	2000	3	59000
3	25-30	6	RedFox	Veg	1322	2	30000
4	20-25	-1	LemonTree	Veg	1234	2	120000
5	35+	3	Ibis	Vegetarian	989	2	45000
6	35+	3	Ibys	Non-Veg	1909	2	122220
7	35+	4	RedFox	Vegetarian	1000	-1	21122
8	20-25	7	LemonTree	Veg	2999	-10	345673
9	25-30	2	Ibis	Non-Veg	3456	3	-99999
10	30-35	5	RedFox	non-Veg	6755	4	87777

In [10]:
```
# The function . loc is typically used for label indexing and can access mul
df.loc[df['CustomerID'] == 0, 'CustomerID'] = np.nan
df.loc[df['Bill'] < 0, 'Bill'] = np.nan
df.loc[df['EstimatedSalary'] < 0, 'EstimatedSalary'] = np.nan
df
```

Out[10]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>CustomerID</th> <th>Age_Group</th> <th>Rating(1-5)</th> <th>Hotel</th> <th>FoodPreference</th> <th>Bill</th> <th>NoOfPax</th> <th>EstimatedSalary</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

1.0	20-25	4	Ibis	veg	1300.0	2	40000.0
2.0	30-35	5	LemonTree	Non-Veg	2000.0	3	59000.0
3.0	25-30	6	RedFox	Veg	1322.0	2	30000.0
4.0	20-25	-1	LemonTree	Veg	1234.0	2	120000.0
5.0	35+	3	Ibis	Vegetarian	989.0	2	45000.0
6.0	35+	3	Ibys	Non-Veg	1909.0	2	122220.0
7.0	35+	4	RedFox	Vegetarian	1000.0	-1	21122.0
8.0	20-25	7	LemonTree	Veg	2999.0	-10	345673.0
9.0	25-30	2	Ibis	Non-Veg	3456.0	3	NaN
10.0	30-35	5	RedFox	non-Veg	6755.0	4	87777.0

In [11]:
```
df.loc[df['NoOfPax'] < 0, 'NoOfPax'] = np.nan
df
```

Out[11]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>CustomerID</th> <th>Age_Group</th> <th>Rating(1-5)</th> <th>Hotel</th> <th>FoodPreference</th> <th>Bill</th> <th>NoOfPax</th> <th>EstimatedSalary</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

1.0	20-25	4	Ibis	veg	1300.0	2.0	40000.0
2.0	30-35	5	LemonTree	Non-Veg	2000.0	3.0	59000.0
3.0	25-30	6	RedFox	Veg	1322.0	2.0	30000.0
4.0	20-25	-1	LemonTree	Veg	1234.0	2.0	120000.0
5.0	35+	3	Ibis	Vegetarian	989.0	2.0	45000.0
6.0	35+	3	Ibys	Non-Veg	1909.0	2.0	122220.0
7.0	35+	4	RedFox	Vegetarian	1000.0	NaN	21122.0
8.0	20-25	7	LemonTree	Veg	2999.0	NaN	345673.0
9.0	25-30	2	Ibis	Non-Veg	3456.0	3.0	NaN
10.0	30-35	5	RedFox	non-Veg	6755.0	4.0	87777.0

In [12]:
```
df.Age_Group.unique()
```

Out[12]:
```
<StringArray>
['20-25', '30-35', '25-30', '35+']
Length: 4, dtype: str
```

In [13]:
```
df.Hotel.unique()
```

Out[13]:
```
<StringArray>
['Ibis', 'LemonTree', 'RedFox', 'Ibys']
Length: 4, dtype: str
```

In [18]:
```
# Using the inplace=True keyword in a pandas method changes the default beha


df['Hotel'] = df['Hotel'].replace(['Ibys'], 'Ibis')

df['FoodPreference'] = df['FoodPreference'].replace(
    ['Vegetarian', 'veg', 'non-veg'],
    ['Veg', 'Veg', 'Non-Veg']
)

df['FoodPreference'].unique()
```

Out[18]:
```
<StringArray>
['Veg', 'Non-Veg']
Length: 2, dtype: str
```

In [14]:
```
df['FoodPreference'] = df['FoodPreference'].replace({
    'Vegetarian': 'Veg',
    'vegetarian': 'Veg',
    'veg': 'Veg',
    'Non-Veg.': 'Non-Veg',
    'non-veg': 'Non-Veg',
    'non veg': 'Non-Veg'
})

print(df['FoodPreference'].unique())
```

```
<StringArray>
['Veg', 'Non-Veg', 'non-Veg']
Length: 3, dtype: str
```

In [15]:
```
# Fillna is a Pandas function to fill the NA/NaN values with the specified m
# If column or feature is numerical continuous data then replace the missing
# If column or feature is numerical discrete data then replace the missing(N

# If column or feature is non-numerical i.e Categorical data then replace th
# Fill missing numerical values
df['EstimatedSalary'] = df['EstimatedSalary'].fillna(
    round(df['EstimatedSalary'].mean())
)

df['NoOfPax'] = df['NoOfPax'].fillna(
    round(df['NoOfPax'].median())
)

df['Rating(1-5)'] = df['Rating(1-5)'].fillna(
```

```
    round(df['Rating(1-5)'].median())
)

df['Bill'] = df['Bill'].fillna(
    round(df['Bill'].mean())
)

df
```

Out[15]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>CustomerID</th> <th>Age_Group</th> <th>Rating(1-5)</th> <th>Hotel</th> <th>FoodPreference</th> <th>Bill</th> <th>NoOfPax</th> <th>EstimatedSalary</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

1.0	20-25	4	Ibis	Veg	1300.0	2.0	40000.0
2.0	30-35	5	LemonTree	Non-Veg	2000.0	3.0	59000.0
3.0	25-30	6	RedFox	Veg	1322.0	2.0	30000.0
4.0	20-25	-1	LemonTree	Veg	1234.0	2.0	120000.0
5.0	35+	3	Ibis	Veg	989.0	2.0	45000.0
6.0	35+	3	Ibys	Non-Veg	1909.0	2.0	122220.0
7.0	35+	4	RedFox	Veg	1000.0	2.0	21122.0
8.0	20-25	7	LemonTree	Veg	2999.0	2.0	345673.0
9.0	25-30	2	Ibis	Non-Veg	3456.0	3.0	96755.0
10.0	30-35	5	RedFox	non-Veg	6755.0	4.0	87777.0

Exp No:14 Experiment to understand feature scaling. Description: Understand the importance of feature scaling

In [2]:
```
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\jaiwa\Downloads\Data.csv")
df
```

Out[2]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Country</th> <th>Age</th> <th>Salary</th> <th>Purchased</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

	France	44.0	72000.0	No
	Spain	27.0	48000.0	Yes
	Germany	30.0	54000.0	No
	Spain	38.0	61000.0	No
	Germany	40.0	NaN	Yes
	France	35.0	58000.0	Yes
	Spain	NaN	52000.0	No
	France	48.0	79000.0	Yes
	Germany	50.0	83000.0	No
	France	37.0	67000.0	Yes

In [3]:
```
df.head()
```

Out[3]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Country</th> <th>Age</th> <th>Salary</th> <th>Purchased</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

	France	44.0	72000.0	No
	Spain	27.0	48000.0	Yes
	Germany	30.0	54000.0	No
	Spain	38.0	61000.0	No
	Germany	40.0	NaN	Yes

In [4]:
```
df.Country.fillna(df.Country.mode()[0],inplace=True)

features=df.iloc[:,:-1].values
```

```
C:\Users\jaiwa\AppData\Local\Temp\ipykernel_11992\2471854110.py:1: ChainedAss
ignmentError: A value is being set on a copy of a DataFrame or Series through
chained assignment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, b
ecause the intermediate object on which we are setting values always behaves
as a copy (due to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.
method({col: value}, inplace=True)' instead, to perform the operation inplace
on the original object, or try to avoid an inplace operation using 'df[col] =
df[col].method(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.
org/pandas-docs/stable/user_guide/copy_on_write.html
  df.Country.fillna(df.Country.mode()[0],inplace=True)
```

In [5]:
```
label=df.iloc[:,-1].values
from sklearn.impute import SimpleImputer
age=SimpleImputer(strategy="mean",missing_values=np.nan)
Salary=SimpleImputer(strategy="mean",missing_values=np.nan)
age.fit(features[:,[1]])
```

@returns {boolean} Always returns `false` so callers can prevent the default click behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem = element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open; detailsElem.open = true; var content = element.closest('.features').querySelector('tbody') .innerText.trim(); if (!wasOpen) detailsElem.open = false; const rows = content.split('\n').map(row => ` "$row$ " `); $const formattedText = `[\n$ {rows.join(',\n')},\n]`; const originalHTML = element.innerHTML.replace('âœ"', "); const originalStyle = element.style; const copyMark = document.createElement('span'); copyMark.innerHTML = 'âœ"'; copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em'; navigator.clipboard.writeText(formattedText) .then(() => { element.style.display = 'none'; element.parentElement.appendChild(copyMark); setTimeout(() => { copyMark.remove(); element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'orange'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub * https://github.com/skrub-data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_data/temp * @returns "light" or "dark" */ function detectTheme(element) { const body = document.querySelector('body'); // Check VSCode theme const themeKindAttr = body.getAttribute('data-vscode-theme-kind'); const themeNameAttr = body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) { const themeKind = themeKindAttr.toLowerCase(); const themeName = themeNameAttr.toLowerCase(); if (themeKind.includes("dark") || themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") || themeName.includes("light")) { return "light"; } } // Check Jupyter theme if (body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if (body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a parent element's color const color = window.getComputedStyle(element.parentNode, null).getPropertyValue('color'); const match = color.match(/^rgb\s* $\s*(\d+)\s*,\s*(\d+)\s*,\s*(\d+)\s*$ \s*$/i); if (match) { const [r, g, b] = [parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3])]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to system preference return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement === null) { console.error(`Element with id ${elementId} not found.`); } else { const theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } } forceTheme('sk-container-id-1');</script></body>

In [6]:
```
Salary.fit(features[:,[2]])
```

Jupyter theme if (body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if (body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a parent element's color const color = window.getComputedStyle(element.parentNode, null).getPropertyValue('color'); const match = color.match(/^rgb\s* \s * (\d+)\s*, \s * (\d+)\s*, \s * (\d+)\s*\s*$/i); if (match) { const [r, g, b] = [parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3])]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to system preference return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement === null) { console.error(`Element with id ${elementId} not found.`); } else { const theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } } forceTheme('sk-container-id-3');</script></body>

In [8]:
```
features[:,[1]]=age.transform(features[:,[1]])
features[:,[2]]=Salary.transform(features[:,[2]])
features
```

Out[8]:
```
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, 63777.77777777778],
       ['France', 35.0, 58000.0],
       ['Spain', 38.77777777777778, 52000.0],
       ['France', 48.0, 79000.0],
       ['Germany', 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)
```

In [9]:
```
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
Country
```

Out[9]:
```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [1., 0., 0.]])
```

In [10]:
```
final_set=np.concatenate((Country,features[:,[1,2]]),axis=1)
final_set
```

Out[10]:
```
array([[1.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 27.0, 48000.0],
       [0.0, 1.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 48.0, 79000.0],
       [0.0, 1.0, 0.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

In [11]:
```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
sc.fit(final_set)
feat_standard_scaler=sc.transform(final_set)
feat_standard_scaler
```

Out[11]:
```
array([[ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         7.58874362e-01,  7.49473254e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -1.71150388e+00, -1.43817841e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
        -1.27555478e+00, -8.91265492e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -1.13023841e-01, -2.53200424e-01],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
         1.77608893e-01,  6.63219199e-16],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -5.48972942e-01, -5.26656882e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
         0.00000000e+00, -1.07356980e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         1.34013983e+00,  1.38753832e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
         1.63077256e+00,  1.75214693e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -2.58340208e-01,  2.93712492e-01]])
```

In [12]:
```
from sklearn.preprocessing import MinMaxScaler
mms=MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)
feat_minmax_scaler=mms.transform(final_set)
feat_minmax_scaler
```

Out[12]:
```
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
       [0.        , 1.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

Exp No:15 Experiment to understand the data preprocessing in Data science
Description: Understand the importance of Data preprocessing in data science

In [1]:
```
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\jaiwa\Downloads\Data.csv")
df
```

Out[1]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Country</th> <th>Age</th> <th>Salary</th> <th>Purchased</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

	France	44.0	72000.0	No
	Spain	27.0	48000.0	Yes
	Germany	30.0	54000.0	No
	Spain	38.0	61000.0	No
	Germany	40.0	NaN	Yes
	France	35.0	58000.0	Yes
	Spain	NaN	52000.0	No
	France	48.0	79000.0	Yes
	Germany	50.0	83000.0	No
	France	37.0	67000.0	Yes

In [2]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Country    10 non-null     str    
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     str    
dtypes: float64(2), str(2)
memory usage: 452.0 bytes
```

In [3]:
```
df.Country.mode()
```

Out[3]:
```
0    France
Name: Country, dtype: str
```

In [4]:
```
df.Country.mode()[0]
```

Out[4]: 'France'

In [5]:
```
type(df.Country.mode())
```

Out[5]: pandas.Series

In [10]:
```
df['Country'] = df['Country'].fillna(df['Country'].mode()[0])
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(round(df['Salary'].mean()))

df
```

Out[10]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Country</th> <th>Age</th> <th>Salary</th> <th>Purchased</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

	France	44.0	72000.0	No
	Spain	27.0	48000.0	Yes
	Germany	30.0	54000.0	No
	Spain	38.0	61000.0	No
	Germany	40.0	63778.0	Yes
	France	35.0	58000.0	Yes
	Spain	38.0	52000.0	No
	France	48.0	79000.0	Yes
	Germany	50.0	83000.0	No
	France	37.0	67000.0	Yes

In [7]:
```
pd.get_dummies(df.Country)
```

Out[7]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>France</th> <th>Germany</th> <th>Spain</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

True	False	False	
False	False	True	
False	True	False	
False	False	True	
False	True	False	
True	False	False	
False	False	True	
True	False	False	
False	True	False	
True	False	False	

In [7]:
```
updated_dataset=pd.concat([pd.get_dummies(df.Country),df.iloc[:,[1,2,3]]],ax
updated_dataset
```

Out[7]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>France</th> <th>Germany</th> <th>Spain</th> <th>Age</th> <th>Salary</th> <th>Purchased</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody> </tbody>

True	False	False	44.0	72000.0	No
False	False	True	27.0	48000.0	Yes
False	True	False	30.0	54000.0	No
False	False	True	38.0	61000.0	No
False	True	False	40.0	NaN	Yes
True	False	False	35.0	58000.0	Yes
False	False	True	NaN	52000.0	No
True	False	False	48.0	79000.0	Yes
False	True	False	50.0	83000.0	No
True	False	False	37.0	67000.0	Yes

In [8]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   Country    10 non-null     str    
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     str    
dtypes: float64(2), str(2)
memory usage: 452.0 bytes
```

In [11]:
```
updated_dataset = df.copy()

updated_dataset['Purchased'] = updated_dataset['Purchased'].replace({
    'No': 0,
    'Yes': 1
})

updated_dataset
```

Out[11]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Country</th> <th>Age</th> <th>Salary</th> <th>Purchased</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

	France	44.0	72000.0	0
	Spain	27.0	48000.0	1
	Germany	30.0	54000.0	0
	Spain	38.0	61000.0	0
	Germany	40.0	63778.0	1
	France	35.0	58000.0	1
	Spain	38.0	52000.0	0
	France	48.0	79000.0	1
	Germany	50.0	83000.0	0
	France	37.0	67000.0	1

Exp No:15 Experiment to understand EDA-Quantitative and Qualitative analysis.
Description: Understand the importance of EDA-Quantitative and Qualitative analysis.

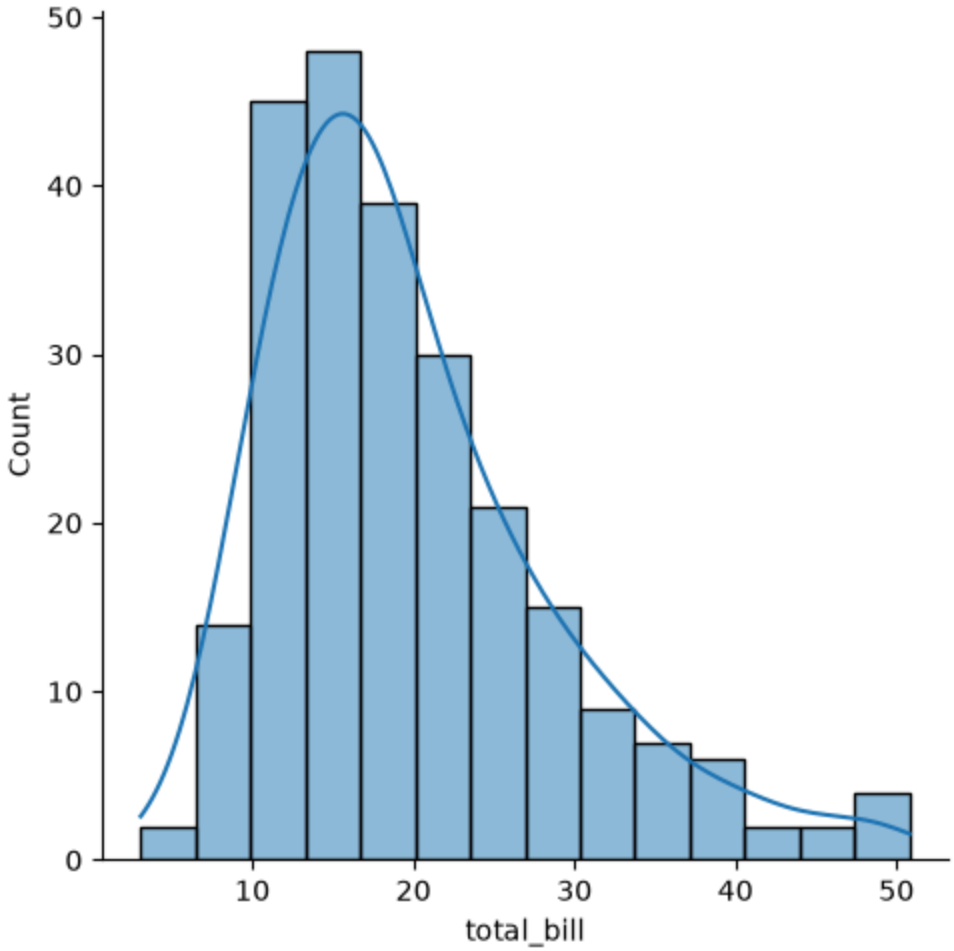
In [1]:
```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
tips=sns.load_dataset('tips')
tips.head()
```

Out[1]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>total_bill</th> <th>tip</th> <th>sex</th> <th>smoker</th> <th>day</th> <th>time</th> <th>size</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

16.99	1.01	Female	No	Sun	Dinner	2
10.34	1.66	Male	No	Sun	Dinner	3
21.01	3.50	Male	No	Sun	Dinner	3
23.68	3.31	Male	No	Sun	Dinner	2
24.59	3.61	Female	No	Sun	Dinner	4

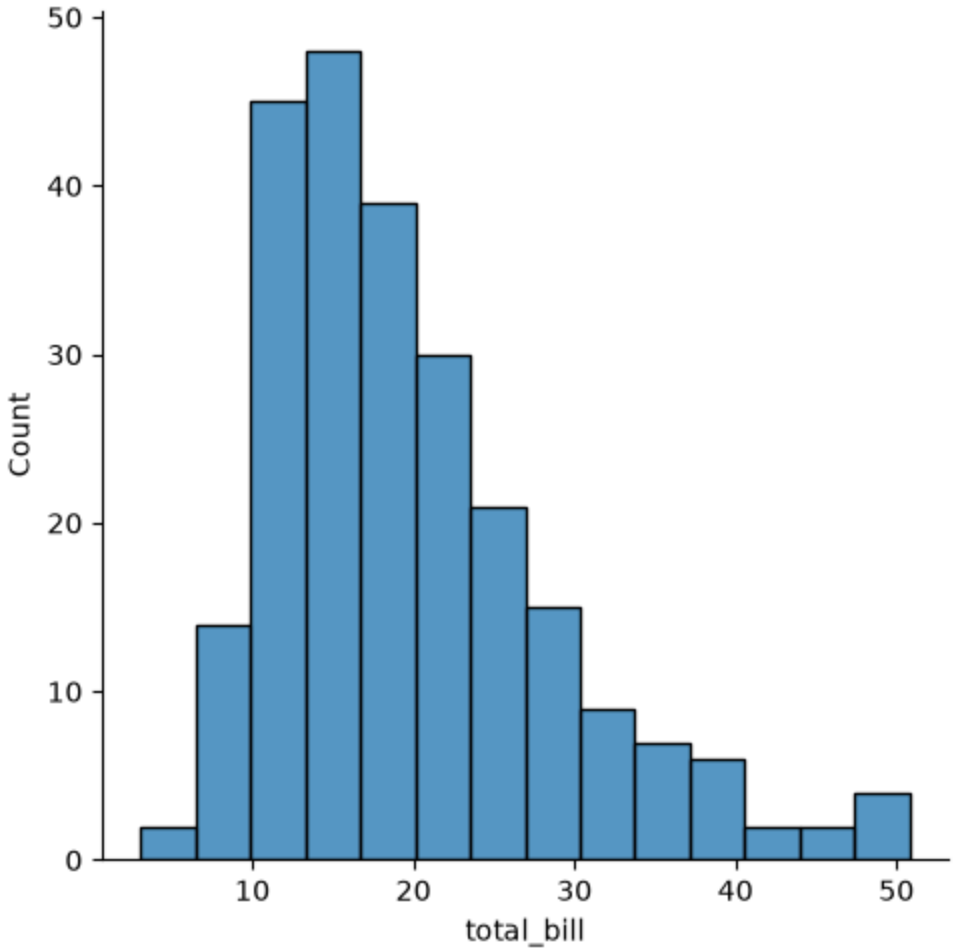
In [2]:
```
sns.displot(tips.total_bill,kde=True)
```

Out[2]: <seaborn.axisgrid.FacetGrid at 0x211e78f06e0>



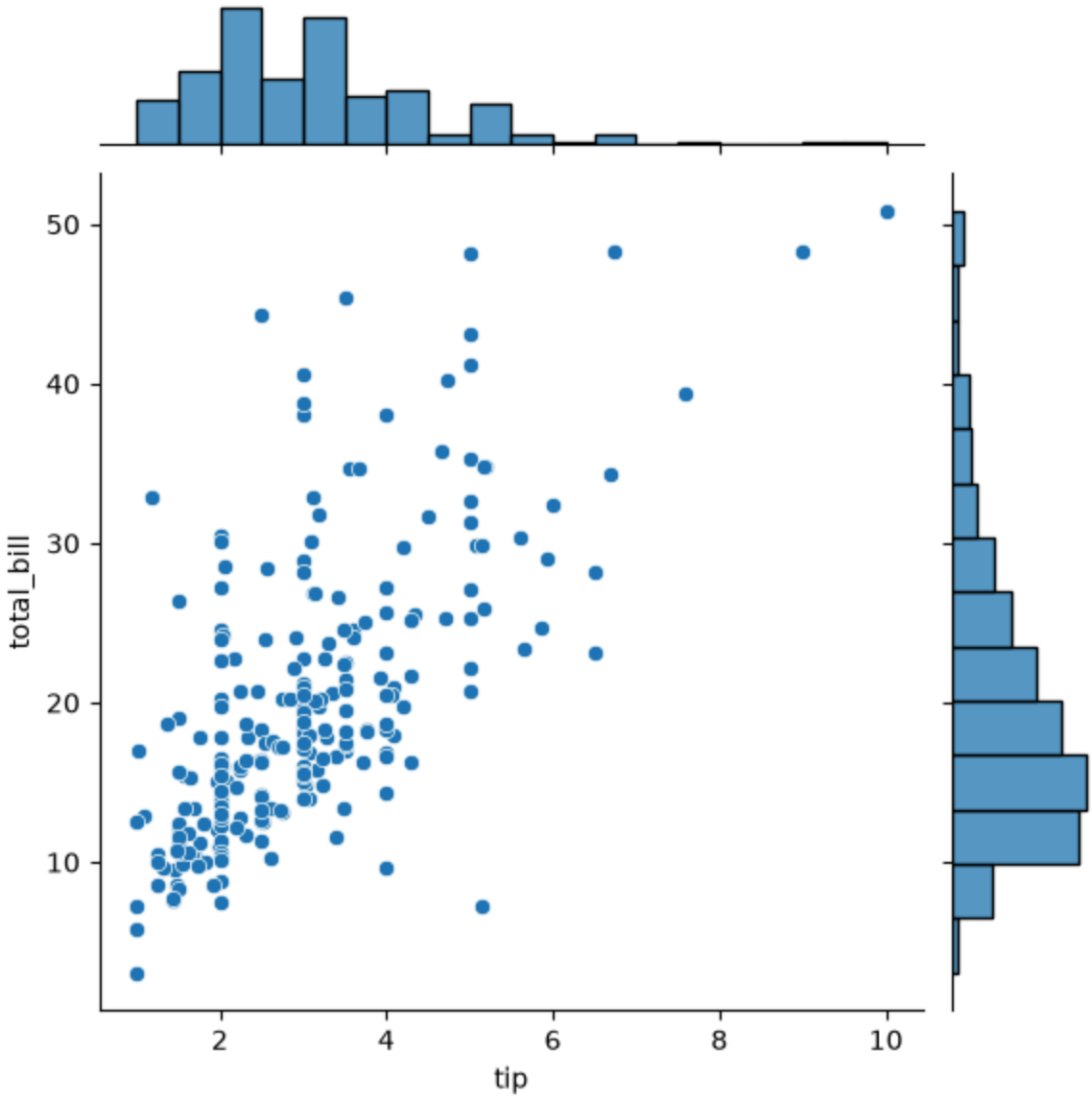
In [3]:
```
sns.displot(tips.total_bill,kde=False)
```

Out[3]: <seaborn.axisgrid.FacetGrid at 0x211e8989590>



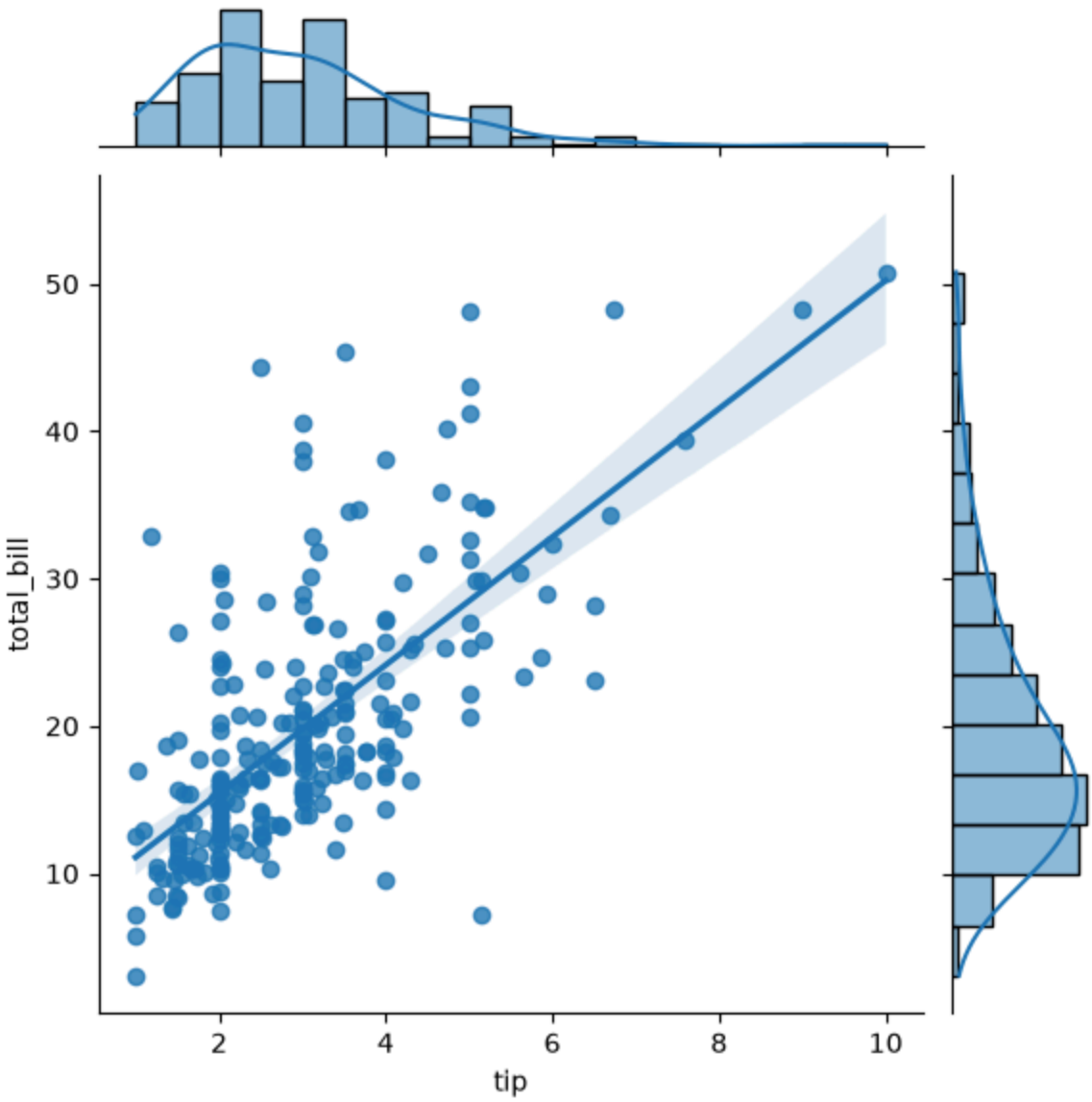
In [4]:
```
sns.jointplot(x=tips.tip,y=tips.total_bill)
```

Out[4]:
```
<seaborn.axisgrid.JointGrid at 0x211e88fb230>
```



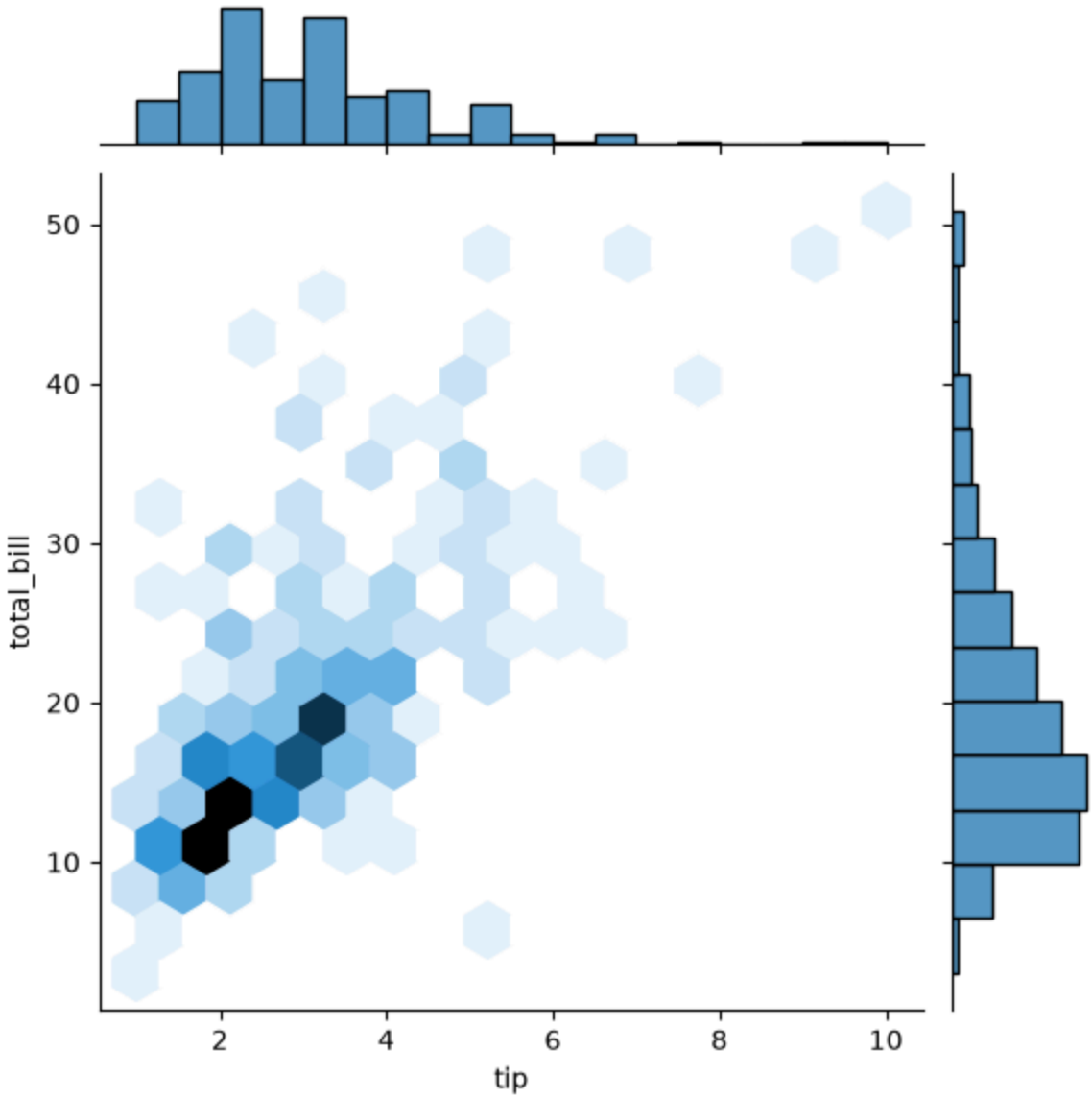
In [5]:
```
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="reg")
```

Out[5]: <seaborn.axisgrid.JointGrid at 0x211ecb99310>



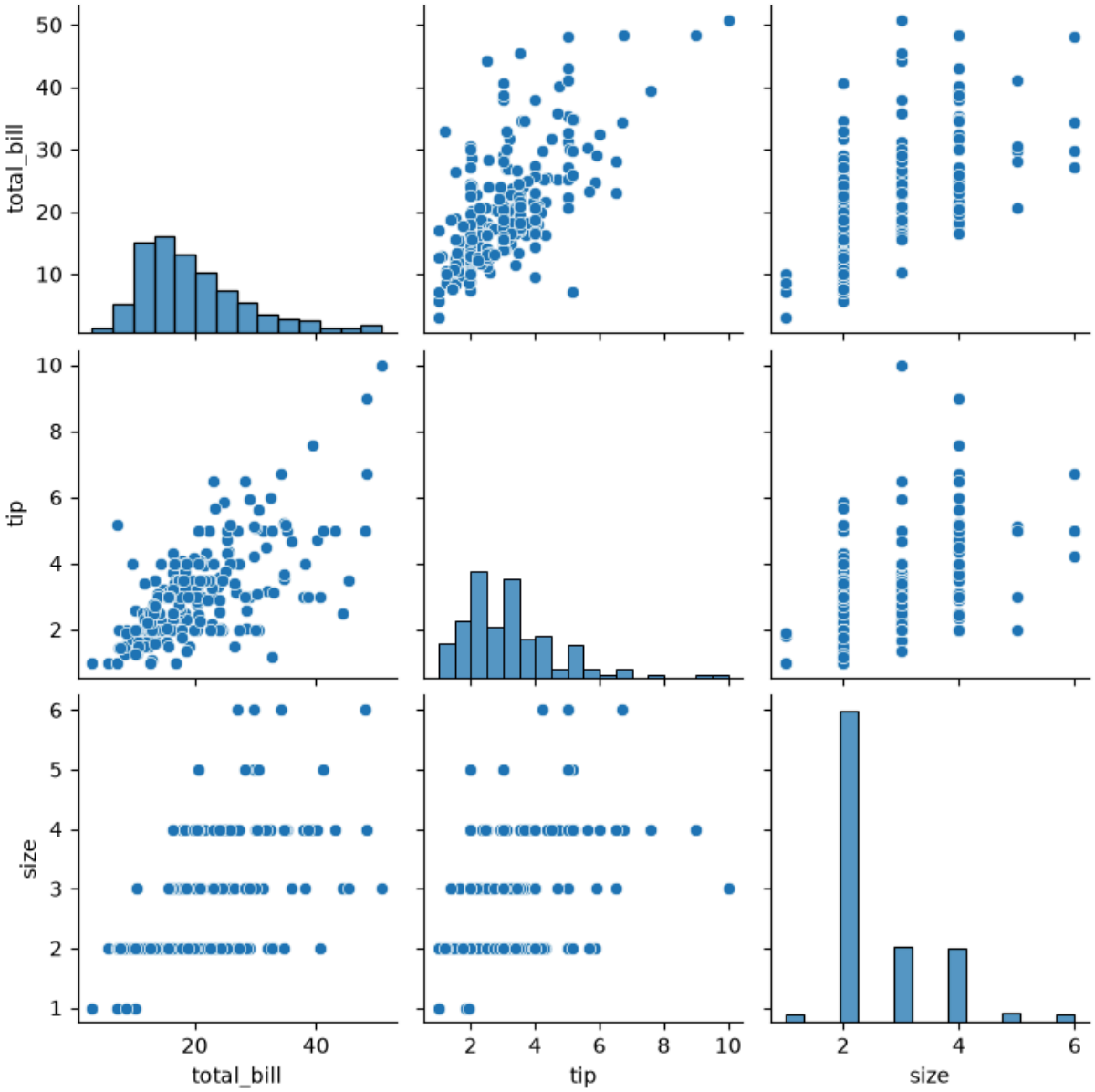
In [6]:
```
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="hex")
```

Out[6]: <seaborn.axisgrid.JointGrid at 0x211ed134b90>



In [7]:
```
sns.pairplot(tips)
```

Out[7]: <seaborn.axisgrid.PairGrid at 0x211ed21a900>

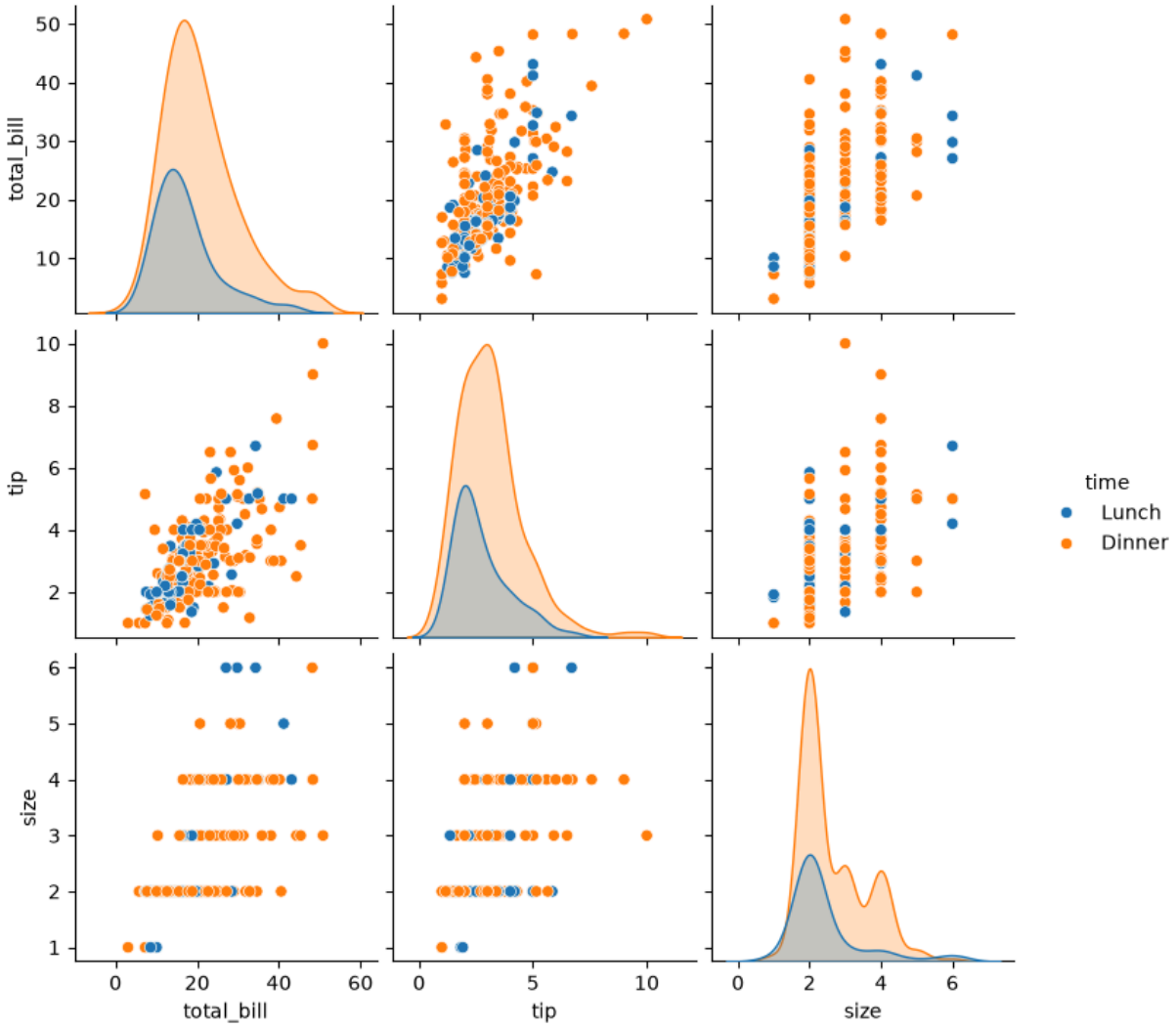


In [8]:
```
tips.time.value_counts()
```

Out[8]:
```
time
Dinner    176
Lunch      68
Name: count, dtype: int64
```

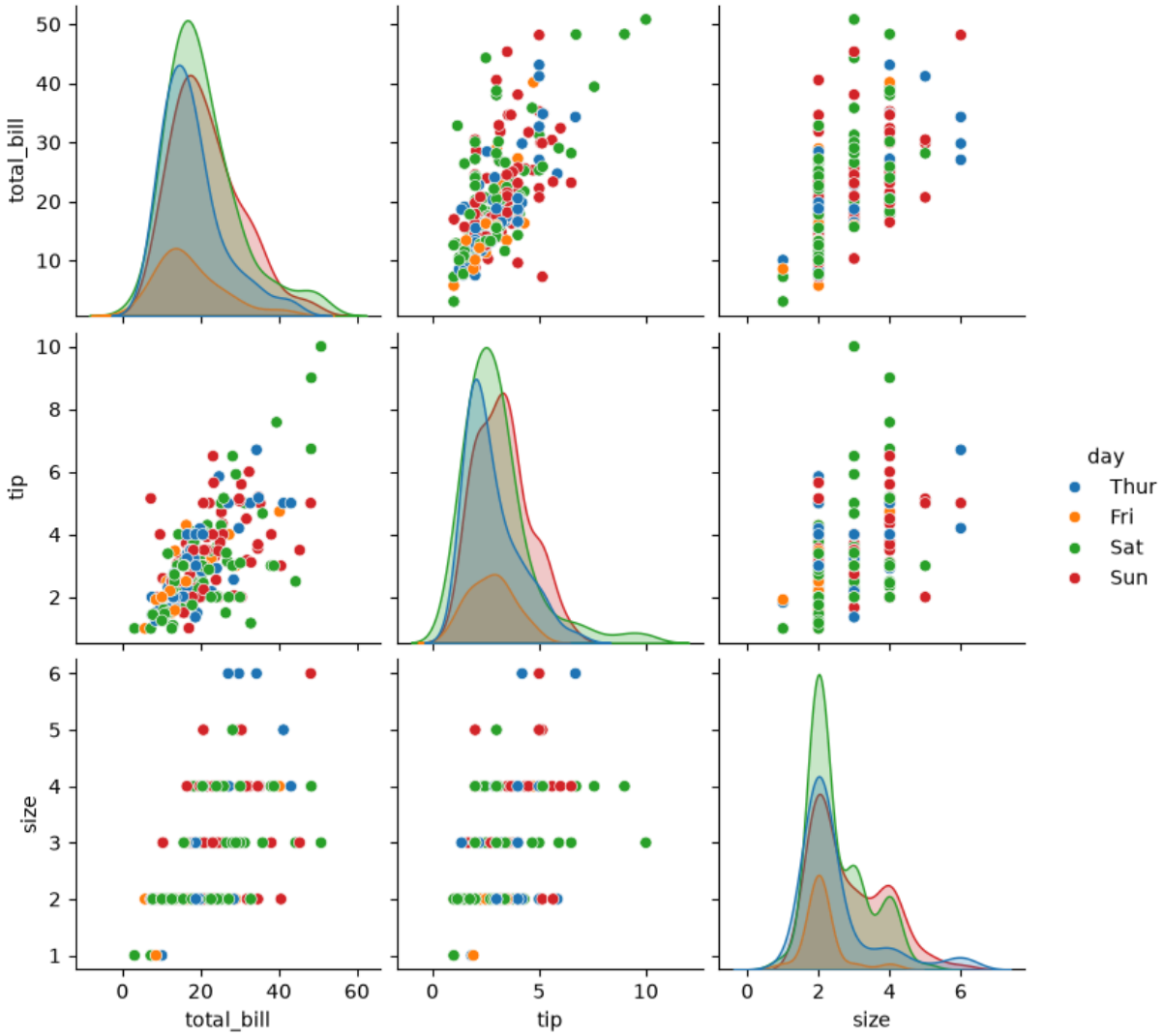
In [9]:
```
sns.pairplot(tips,hue='time')
```

Out[9]:
```
<seaborn.axisgrid.PairGrid at 0x211edd50f50>
```



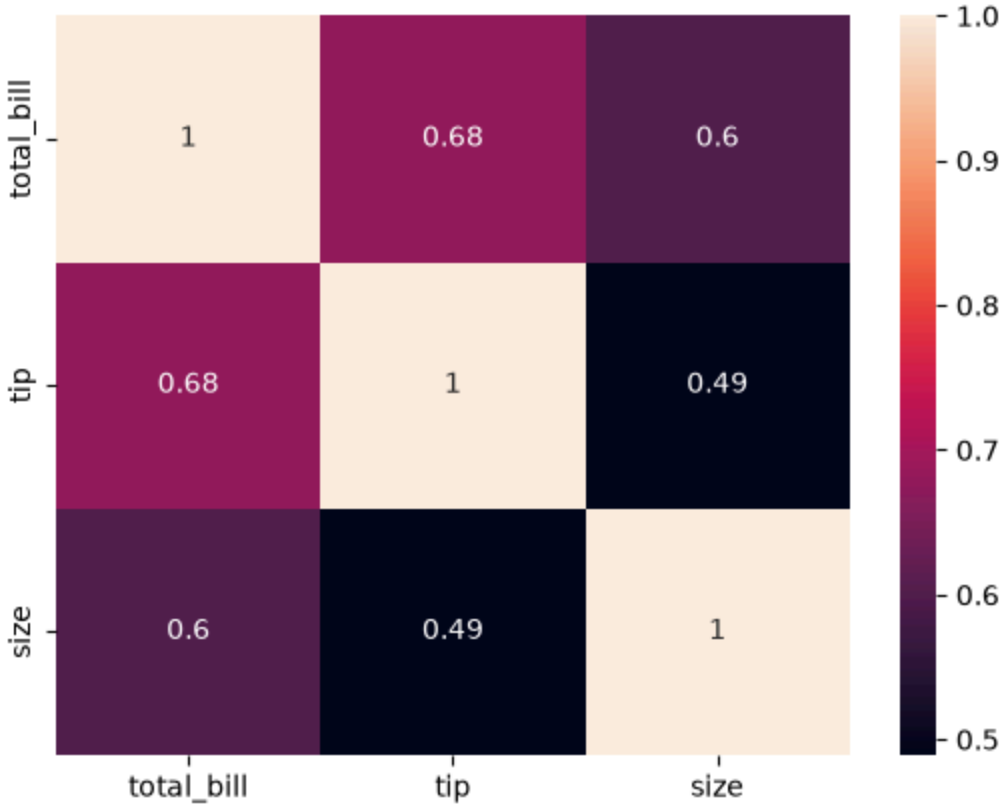
In [10]:
```
sns.pairplot(tips,hue='day')
```

Out[10]: <seaborn.axisgrid.PairGrid at 0x211ee5b9310>



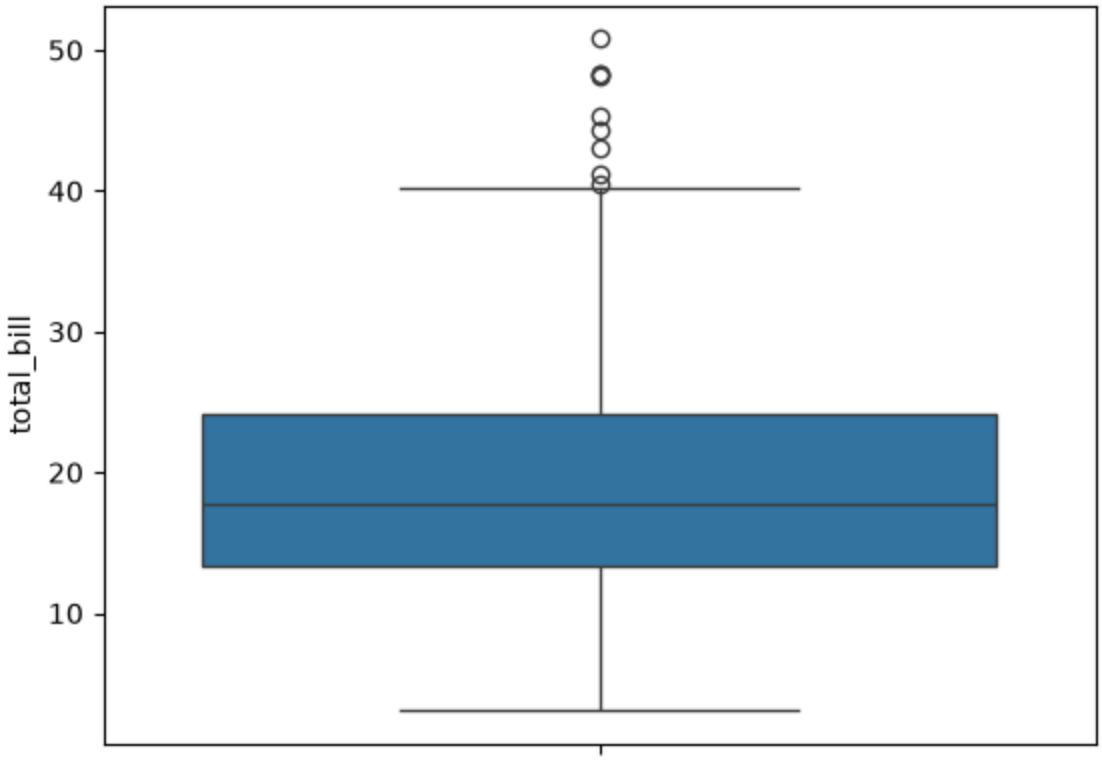
In [11]:
```
sns.heatmap(tips.corr(numeric_only=True),annot=True)
```

Out[11]: <Axes: >



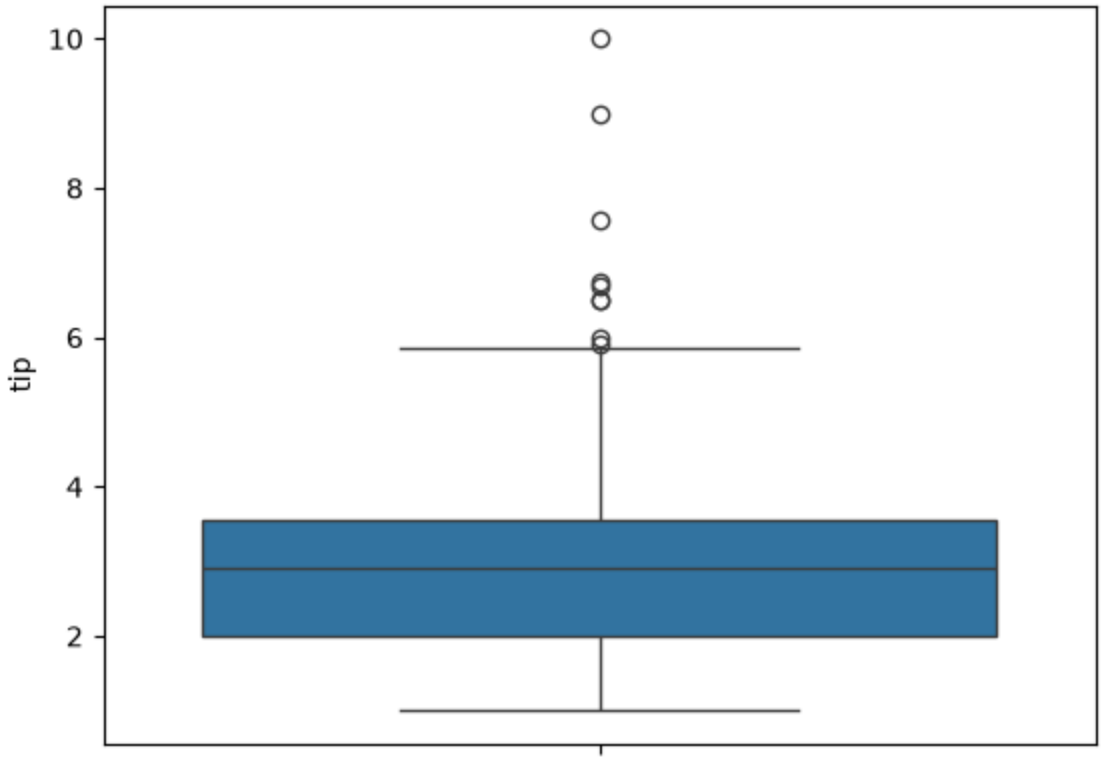
In [12]:
```
sns.boxplot(tips.total_bill)
```

Out[12]: <Axes: ylabel='total_bill'>



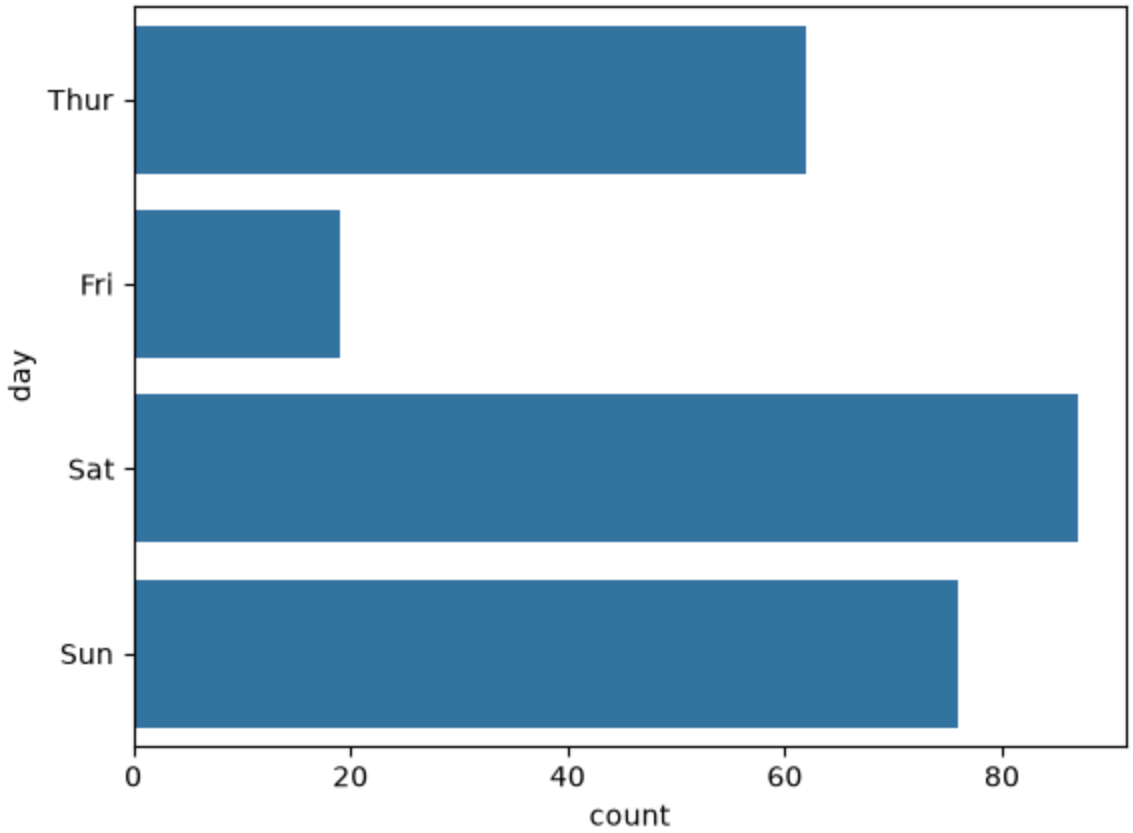
In [13]:
```
sns.boxplot(tips.tip)
```

Out[13]: <Axes: ylabel='tip'>



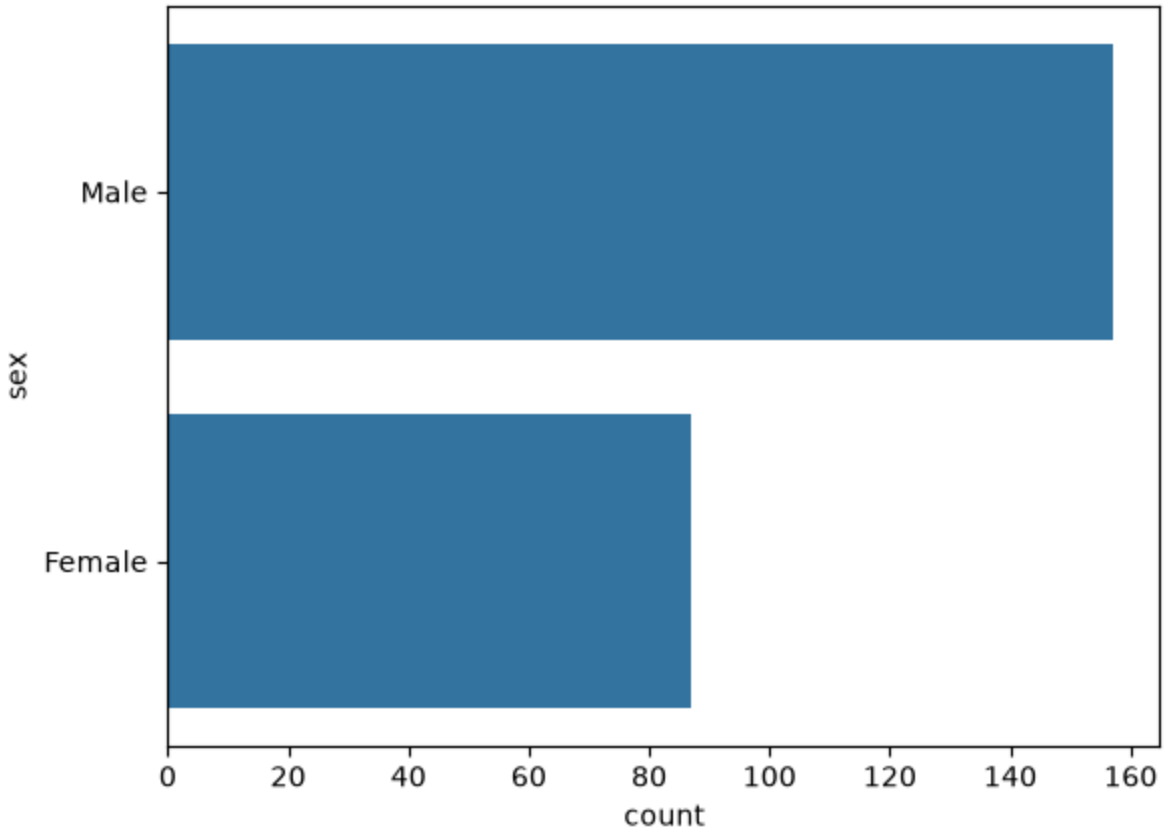
In [14]:
```
sns.countplot(tips.day)
```

Out[14]: <Axes: xlabel='count', ylabel='day'>



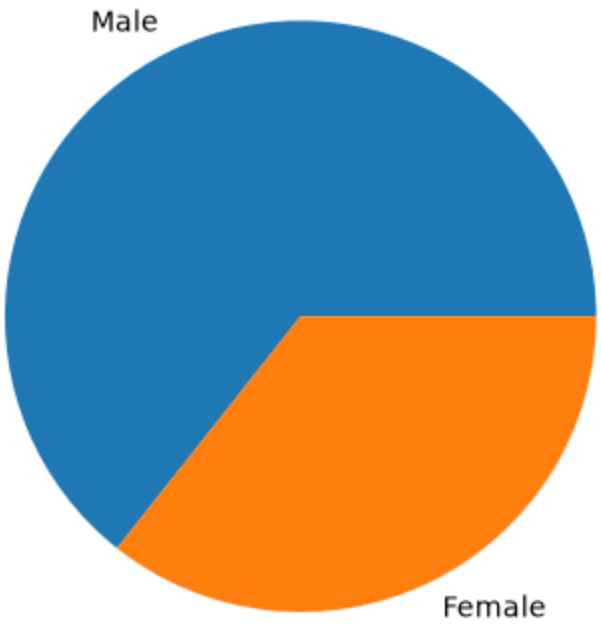
In [15]:
```
sns.countplot(tips.sex)
```

Out[15]: <Axes: xlabel='count', ylabel='sex'>



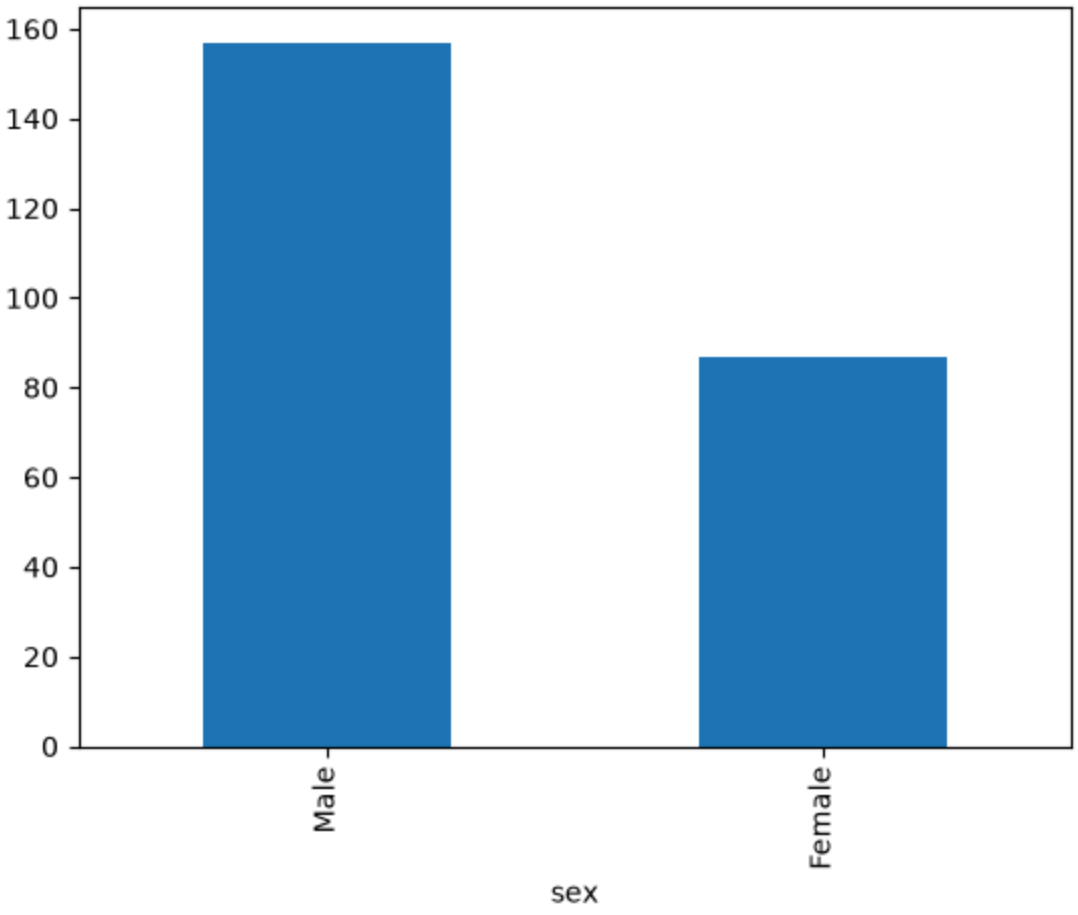
In [16]:
```
tips.sex.value_counts().plot(kind='pie')
```

Out[16]: <Axes: >



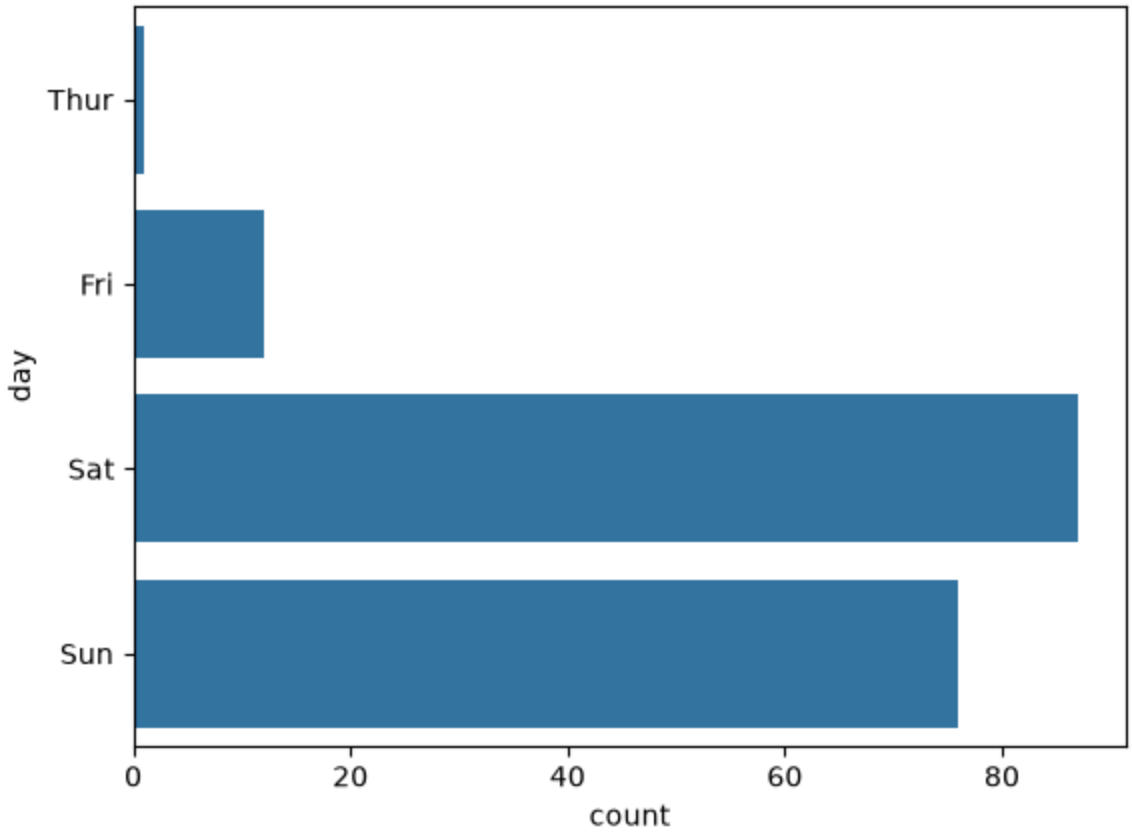
In [17]:
```
tips.sex.value_counts().plot(kind='bar')
```

Out[17]:
```
<Axes: xlabel='sex'>
```



In [18]:
```
sns.countplot(tips[tips.time=='Dinner']['day'])
```

Out[18]: <Axes: xlabel='count', ylabel='day'>



Exp No:17 Experiment to understand Linear Regression for a given data set.

Description: Understand the Linear regression for the dataset given.

In [2]:
```
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\jaiwa\Downloads\salary_data.csv")
df
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     float64
dtypes: float64(2)
memory usage: 612.0 bytes
```

In [3]:
```
df.dropna(inplace=True)
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     float64
dtypes: float64(2)
memory usage: 612.0 bytes
```

In [3]:
```
df.describe()
```

Out[3]:
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>YearsExperience</th> <th>Salary</th> </thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody></tbody>

30.000000	30.000000
5.313333	76003.000000
2.837888	27414.429785
1.100000	37731.000000
3.200000	56720.750000
4.700000	65237.000000
7.700000	100544.750000
10.500000	122391.000000

In [4]:
```
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
```

In [6]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,
```

In [7]:
```
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
```

detectTheme(estimatorElement); estimatorElement.classList.add(theme); } }
forceTheme('sk-container-id-1');</script></body>

In [8]:
```
model.score(x_train,y_train)
```

Out[8]:
```
0.9645401573418146
```

In [9]:
```
model.score(x_test,y_test)
```

Out[9]:
```
0.9024461774180497
```

In [10]:
```
model.coef_
```

Out[10]:
```
array([[9423.81532303]])
```

In [11]:
```
model.intercept_
```

Out[11]:
```
array([25321.58301178])
```

In [12]:
```
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
```

In [13]:
```
model=pickle.load(open('SalaryPred.model','rb'))
```

In [14]:
```
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
```

In [16]:
```
print("Estimated Salary for {} years of experience is {}: " .format(yr_of_ex
```

```
Estimated Salary for 44.0 years of experience is [[439969.45722514]]:
```

Exp No:18 Experiment to understand KNN algorithm for a given data set

Description: Understand the KNN algorithm for the dataset given.

In [1]:
```
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\jaiwa\Downloads\iris.csv")
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    str    
dtypes: float64(4), str(1)
memory usage: 6.0 KB
```

In [3]:
```
df["species"].value_counts()
```

Out[3]:
```
species
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```

In [4]:
```
df.head()
```

Out[4]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>sepal_length</th> <th>sepal_width</th> <th>petal_length</th> <th>petal_width</th> <th>species</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa

In [5]:
```
features=df.iloc[:,:-1].values
label=df.iloc[:,4].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

In [7]:
```
xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,rando
model_KNN=KNeighborsClassifier(n_neighbors=5)
```

```
model_KNN.fit(xtrain,ytrain)
```

(themeKind.includes("light") || themeName.includes("light")) { return "light"; } } // Check Jupyter theme if (body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if (body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a parent element's color const color = window.getComputedStyle(element.parentNode, null).getPropertyValue('color'); const match = color.match(/^rgb\s* \s * (\d+)\s*, \s * (\d+)\s*, \s * (\d+)\s*\s*$/i); if (match) { const [r, g, b] = [parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3])]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to system preference return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement === null) { console.error(`Element with id ${elementId} not found.`); } else { const theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } } forceTheme('sk-container-id-1');</script></body>

In [8]:
```
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
```

```
0.9666666666666667
1.0
```

In [9]:
```
from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))
```

Out[9]:
```
array([[50,  0,  0],
       [ 0, 47,  3],
       [ 0,  1, 49]])
```

In [10]:
```
from sklearn.metrics import classification_report
print(classification_report(label,model_KNN.predict(features)))
```

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        50
  versicolor       0.98      0.94      0.96        50
   virginica       0.94      0.98      0.96        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

Exp No:19 Experiment to understand Logistic Regression for a given data set.
Description: Understand the Logistic Regression algorithm for the dataset given.

In [2]:
```
import numpy as np
import pandas as pd
df=pd.read_csv(r"C:\Users\jaiwa\Downloads\Social_Network_Ads.csv")
df
```

Out[2]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>User ID</th> <th>Gender</th> <th>Age</th> <th>EstimatedSalary</th> <th>Purchased</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>...</th> <th>395</th> <th>396</th> <th>397</th> <th>398</th> <th>399</th> </tbody> <tbody></tbody>

0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
...	...	...	...	...	...
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

400 rows × 5 columns

In [3]:
```
df.head()
```

Out[3]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>User ID</th> <th>Gender</th> <th>Age</th> <th>EstimatedSalary</th> <th>Purchased</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

15624510	Male	19	19000	0
15810944	Male	35	20000	0
15668575	Female	26	43000	0
15603246	Female	27	57000	0
15804002	Male	19	76000	0

In [4]:
```
features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
features
```

Out[4]:
```
array([[    19,  19000],
       [    35,  20000],
       [    26,  43000],
       [    27,  57000],
       [    19,  76000],
       [    27,  58000],
       [    27,  84000],
       [    32, 150000],
       [    25,  33000],
       [    35,  65000],
       [    26,  80000],
       [    26,  52000],
       [    20,  86000],
       [    32,  18000],
       [    18,  82000],
       [    29,  80000],
       [    47,  25000],
       [    45,  26000],
       [    46,  28000],
       [    48,  29000],
       [    45,  22000],
       [    47,  49000],
       [    48,  41000],
       [    45,  22000],
       [    46,  23000],
       [    47,  20000],
       [    49,  28000],
       [    47,  30000],
       [    29,  43000],
       [    31,  18000],
       [    31,  74000],
       [    27, 137000],
       [    21,  16000],
       [    28,  44000],
       [    27,  90000],
       [    35,  27000],
       [    33,  28000],
       [    30,  49000],
       [    26,  72000],
       [    27,  31000],
       [    27,  17000],
       [    33,  51000],
       [    35, 108000],
       [    30,  15000],
       [    28,  84000],
       [    23,  20000],
       [    25,  79000],
       [    27,  54000],
       [    30, 135000],
       [    31,  89000],
       [    24,  32000],
       [    18,  44000],
       [    29,  83000],
       [    35,  23000],
       [    27,  58000],
       [    24,  55000],
```

```
[    23,  48000],
[    28,  79000],
[    22,  18000],
[    32, 117000],
[    27,  20000],
[    25,  87000],
[    23,  66000],
[    32, 120000],
[    59,  83000],
[    24,  58000],
[    24,  19000],
[    23,  82000],
[    22,  63000],
[    31,  68000],
[    25,  80000],
[    24,  27000],
[    20,  23000],
[    33, 113000],
[    32,  18000],
[    34, 112000],
[    18,  52000],
[    22,  27000],
[    28,  87000],
[    26,  17000],
[    30,  80000],
[    39,  42000],
[    20,  49000],
[    35,  88000],
[    30,  62000],
[    31, 118000],
[    24,  55000],
[    28,  85000],
[    26,  81000],
[    35,  50000],
[    22,  81000],
[    30, 116000],
[    26,  15000],
[    29,  28000],
[    29,  83000],
[    35,  44000],
[    35,  25000],
[    28, 123000],
[    35,  73000],
[    28,  37000],
[    27,  88000],
[    28,  59000],
[    32,  86000],
[    33, 149000],
[    19,  21000],
[    21,  72000],
[    26,  35000],
[    27,  89000],
[    26,  86000],
[    38,  80000],
[    39,  71000],
[    37,  71000],
```

```
[    38,  61000],
[    37,  55000],
[    42,  80000],
[    40,  57000],
[    35,  75000],
[    36,  52000],
[    40,  59000],
[    41,  59000],
[    36,  75000],
[    37,  72000],
[    40,  75000],
[    35,  53000],
[    41,  51000],
[    39,  61000],
[    42,  65000],
[    26,  32000],
[    30,  17000],
[    26,  84000],
[    31,  58000],
[    33,  31000],
[    30,  87000],
[    21,  68000],
[    28,  55000],
[    23,  63000],
[    20,  82000],
[    30, 107000],
[    28,  59000],
[    19,  25000],
[    19,  85000],
[    18,  68000],
[    35,  59000],
[    30,  89000],
[    34,  25000],
[    24,  89000],
[    27,  96000],
[    41,  30000],
[    29,  61000],
[    20,  74000],
[    26,  15000],
[    41,  45000],
[    31,  76000],
[    36,  50000],
[    40,  47000],
[    31,  15000],
[    46,  59000],
[    29,  75000],
[    26,  30000],
[    32, 135000],
[    32, 100000],
[    25,  90000],
[    37,  33000],
[    35,  38000],
[    33,  69000],
[    18,  86000],
[    22,  55000],
[    35,  71000],
```

```
[    29, 148000],
[    29,  47000],
[    21,  88000],
[    34, 115000],
[    26, 118000],
[    34,  43000],
[    34,  72000],
[    23,  28000],
[    35,  47000],
[    25,  22000],
[    24,  23000],
[    31,  34000],
[    26,  16000],
[    31,  71000],
[    32, 117000],
[    33,  43000],
[    33,  60000],
[    31,  66000],
[    20,  82000],
[    33,  41000],
[    35,  72000],
[    28,  32000],
[    24,  84000],
[    19,  26000],
[    29,  43000],
[    19,  70000],
[    28,  89000],
[    34,  43000],
[    30,  79000],
[    20,  36000],
[    26,  80000],
[    35,  22000],
[    35,  39000],
[    49,  74000],
[    39, 134000],
[    41,  71000],
[    58, 101000],
[    47,  47000],
[    55, 130000],
[    52, 114000],
[    40, 142000],
[    46,  22000],
[    48,  96000],
[    52, 150000],
[    59,  42000],
[    35,  58000],
[    47,  43000],
[    60, 108000],
[    49,  65000],
[    40,  78000],
[    46,  96000],
[    59, 143000],
[    41,  80000],
[    35,  91000],
[    37, 144000],
[    60, 102000],
```

```
[    35,  60000],
[    37,  53000],
[    36, 126000],
[    56, 133000],
[    40,  72000],
[    42,  80000],
[    35, 147000],
[    39,  42000],
[    40, 107000],
[    49,  86000],
[    38, 112000],
[    46,  79000],
[    40,  57000],
[    37,  80000],
[    46,  82000],
[    53, 143000],
[    42, 149000],
[    38,  59000],
[    50,  88000],
[    56, 104000],
[    41,  72000],
[    51, 146000],
[    35,  50000],
[    57, 122000],
[    41,  52000],
[    35,  97000],
[    44,  39000],
[    37,  52000],
[    48, 134000],
[    37, 146000],
[    50,  44000],
[    52,  90000],
[    41,  72000],
[    40,  57000],
[    58,  95000],
[    45, 131000],
[    35,  77000],
[    36, 144000],
[    55, 125000],
[    35,  72000],
[    48,  90000],
[    42, 108000],
[    40,  75000],
[    37,  74000],
[    47, 144000],
[    40,  61000],
[    43, 133000],
[    59,  76000],
[    60,  42000],
[    39, 106000],
[    57,  26000],
[    57,  74000],
[    38,  71000],
[    49,  88000],
[    52,  38000],
[    50,  36000],
```

```
[    59,  88000],
[    35,  61000],
[    37,  70000],
[    52,  21000],
[    48, 141000],
[    37,  93000],
[    37,  62000],
[    48, 138000],
[    41,  79000],
[    37,  78000],
[    39, 134000],
[    49,  89000],
[    55,  39000],
[    37,  77000],
[    35,  57000],
[    36,  63000],
[    42,  73000],
[    43, 112000],
[    45,  79000],
[    46, 117000],
[    58,  38000],
[    48,  74000],
[    37, 137000],
[    37,  79000],
[    40,  60000],
[    42,  54000],
[    51, 134000],
[    47, 113000],
[    36, 125000],
[    38,  50000],
[    42,  70000],
[    39,  96000],
[    38,  50000],
[    49, 141000],
[    39,  79000],
[    39,  75000],
[    54, 104000],
[    35,  55000],
[    45,  32000],
[    36,  60000],
[    52, 138000],
[    53,  82000],
[    41,  52000],
[    48,  30000],
[    48, 131000],
[    41,  60000],
[    41,  72000],
[    42,  75000],
[    36, 118000],
[    47, 107000],
[    38,  51000],
[    48, 119000],
[    42,  65000],
[    40,  65000],
[    57,  60000],
[    36,  54000],
```

```
[    58, 144000],
[    35,  79000],
[    38,  55000],
[    39, 122000],
[    53, 104000],
[    35,  75000],
[    38,  65000],
[    47,  51000],
[    47, 105000],
[    41,  63000],
[    53,  72000],
[    54, 108000],
[    39,  77000],
[    38,  61000],
[    38, 113000],
[    37,  75000],
[    42,  90000],
[    37,  57000],
[    36,  99000],
[    60,  34000],
[    54,  70000],
[    41,  72000],
[    40,  71000],
[    42,  54000],
[    43, 129000],
[    53,  34000],
[    47,  50000],
[    42,  79000],
[    42, 104000],
[    59,  29000],
[    58,  47000],
[    46,  88000],
[    38,  71000],
[    54,  26000],
[    60,  46000],
[    60,  83000],
[    39,  73000],
[    59, 130000],
[    37,  80000],
[    46,  32000],
[    46,  74000],
[    42,  53000],
[    41,  87000],
[    58,  23000],
[    42,  64000],
[    48,  33000],
[    44, 139000],
[    49,  28000],
[    57,  33000],
[    56,  60000],
[    49,  39000],
[    39,  71000],
[    47,  34000],
[    48,  35000],
[    48,  33000],
[    47,  23000],
```

```
       [    45,  45000],
       [    60,  42000],
       [    39,  59000],
       [    46,  41000],
       [    51,  23000],
       [    50,  20000],
       [    36,  33000],
       [    49,  36000]])
```

In [5]:
```
label
```

Out[5]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0,
       1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0,
       1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1,
       1, 1, 0, 1])
```

In [6]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

In [10]:
```
for i in range(1,401):
 x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2
 model=LogisticRegression()
 model.fit(x_train,y_train)
 train_score=model.score(x_train,y_train)
 test_score=model.score(x_test,y_test)
 if test_score>train_score:
  print("Test {} Train{} Random State {}".format(test_score,train_score,i))
```

Test 0.8875 Train0.8375 Random State 1
Test 0.8875 Train0.8375 Random State 2
Test 0.8875 Train0.8375 Random State 3
Test 0.8875 Train0.8375 Random State 4
Test 0.8875 Train0.8375 Random State 5
Test 0.8875 Train0.8375 Random State 6
Test 0.8875 Train0.8375 Random State 7
Test 0.8875 Train0.8375 Random State 8
Test 0.8875 Train0.8375 Random State 9
Test 0.8875 Train0.8375 Random State 10
Test 0.8875 Train0.8375 Random State 11
Test 0.8875 Train0.8375 Random State 12
Test 0.8875 Train0.8375 Random State 13
Test 0.8875 Train0.8375 Random State 14
Test 0.8875 Train0.8375 Random State 15
Test 0.8875 Train0.8375 Random State 16
Test 0.8875 Train0.8375 Random State 17
Test 0.8875 Train0.8375 Random State 18
Test 0.8875 Train0.8375 Random State 19
Test 0.8875 Train0.8375 Random State 20
Test 0.8875 Train0.8375 Random State 21
Test 0.8875 Train0.8375 Random State 22
Test 0.8875 Train0.8375 Random State 23
Test 0.8875 Train0.8375 Random State 24
Test 0.8875 Train0.8375 Random State 25
Test 0.8875 Train0.8375 Random State 26
Test 0.8875 Train0.8375 Random State 27
Test 0.8875 Train0.8375 Random State 28
Test 0.8875 Train0.8375 Random State 29
Test 0.8875 Train0.8375 Random State 30
Test 0.8875 Train0.8375 Random State 31
Test 0.8875 Train0.8375 Random State 32
Test 0.8875 Train0.8375 Random State 33
Test 0.8875 Train0.8375 Random State 34
Test 0.8875 Train0.8375 Random State 35
Test 0.8875 Train0.8375 Random State 36
Test 0.8875 Train0.8375 Random State 37
Test 0.8875 Train0.8375 Random State 38
Test 0.8875 Train0.8375 Random State 39
Test 0.8875 Train0.8375 Random State 40
Test 0.8875 Train0.8375 Random State 41
Test 0.8875 Train0.8375 Random State 42
Test 0.8875 Train0.8375 Random State 43
Test 0.8875 Train0.8375 Random State 44
Test 0.8875 Train0.8375 Random State 45
Test 0.8875 Train0.8375 Random State 46
Test 0.8875 Train0.8375 Random State 47
Test 0.8875 Train0.8375 Random State 48
Test 0.8875 Train0.8375 Random State 49
Test 0.8875 Train0.8375 Random State 50
Test 0.8875 Train0.8375 Random State 51
Test 0.8875 Train0.8375 Random State 52
Test 0.8875 Train0.8375 Random State 53
Test 0.8875 Train0.8375 Random State 54
Test 0.8875 Train0.8375 Random State 55
Test 0.8875 Train0.8375 Random State 56

Test 0.8875 Train0.8375 Random State 57
Test 0.8875 Train0.8375 Random State 58
Test 0.8875 Train0.8375 Random State 59
Test 0.8875 Train0.8375 Random State 60
Test 0.8875 Train0.8375 Random State 61
Test 0.8875 Train0.8375 Random State 62
Test 0.8875 Train0.8375 Random State 63
Test 0.8875 Train0.8375 Random State 64
Test 0.8875 Train0.8375 Random State 65
Test 0.8875 Train0.8375 Random State 66
Test 0.8875 Train0.8375 Random State 67
Test 0.8875 Train0.8375 Random State 68
Test 0.8875 Train0.8375 Random State 69
Test 0.8875 Train0.8375 Random State 70
Test 0.8875 Train0.8375 Random State 71
Test 0.8875 Train0.8375 Random State 72
Test 0.8875 Train0.8375 Random State 73
Test 0.8875 Train0.8375 Random State 74
Test 0.8875 Train0.8375 Random State 75
Test 0.8875 Train0.8375 Random State 76
Test 0.8875 Train0.8375 Random State 77
Test 0.8875 Train0.8375 Random State 78
Test 0.8875 Train0.8375 Random State 79
Test 0.8875 Train0.8375 Random State 80
Test 0.8875 Train0.8375 Random State 81
Test 0.8875 Train0.8375 Random State 82
Test 0.8875 Train0.8375 Random State 83
Test 0.8875 Train0.8375 Random State 84
Test 0.8875 Train0.8375 Random State 85
Test 0.8875 Train0.8375 Random State 86
Test 0.8875 Train0.8375 Random State 87
Test 0.8875 Train0.8375 Random State 88
Test 0.8875 Train0.8375 Random State 89
Test 0.8875 Train0.8375 Random State 90
Test 0.8875 Train0.8375 Random State 91
Test 0.8875 Train0.8375 Random State 92
Test 0.8875 Train0.8375 Random State 93
Test 0.8875 Train0.8375 Random State 94
Test 0.8875 Train0.8375 Random State 95
Test 0.8875 Train0.8375 Random State 96
Test 0.8875 Train0.8375 Random State 97
Test 0.8875 Train0.8375 Random State 98
Test 0.8875 Train0.8375 Random State 99
Test 0.8875 Train0.8375 Random State 100
Test 0.8875 Train0.8375 Random State 101
Test 0.8875 Train0.8375 Random State 102
Test 0.8875 Train0.8375 Random State 103
Test 0.8875 Train0.8375 Random State 104
Test 0.8875 Train0.8375 Random State 105
Test 0.8875 Train0.8375 Random State 106
Test 0.8875 Train0.8375 Random State 107
Test 0.8875 Train0.8375 Random State 108
Test 0.8875 Train0.8375 Random State 109
Test 0.8875 Train0.8375 Random State 110
Test 0.8875 Train0.8375 Random State 111
Test 0.8875 Train0.8375 Random State 112

Test 0.8875 Train0.8375 Random State 113
Test 0.8875 Train0.8375 Random State 114
Test 0.8875 Train0.8375 Random State 115
Test 0.8875 Train0.8375 Random State 116
Test 0.8875 Train0.8375 Random State 117
Test 0.8875 Train0.8375 Random State 118
Test 0.8875 Train0.8375 Random State 119
Test 0.8875 Train0.8375 Random State 120
Test 0.8875 Train0.8375 Random State 121
Test 0.8875 Train0.8375 Random State 122
Test 0.8875 Train0.8375 Random State 123
Test 0.8875 Train0.8375 Random State 124
Test 0.8875 Train0.8375 Random State 125
Test 0.8875 Train0.8375 Random State 126
Test 0.8875 Train0.8375 Random State 127
Test 0.8875 Train0.8375 Random State 128
Test 0.8875 Train0.8375 Random State 129
Test 0.8875 Train0.8375 Random State 130
Test 0.8875 Train0.8375 Random State 131
Test 0.8875 Train0.8375 Random State 132
Test 0.8875 Train0.8375 Random State 133
Test 0.8875 Train0.8375 Random State 134
Test 0.8875 Train0.8375 Random State 135
Test 0.8875 Train0.8375 Random State 136
Test 0.8875 Train0.8375 Random State 137
Test 0.8875 Train0.8375 Random State 138
Test 0.8875 Train0.8375 Random State 139
Test 0.8875 Train0.8375 Random State 140
Test 0.8875 Train0.8375 Random State 141
Test 0.8875 Train0.8375 Random State 142
Test 0.8875 Train0.8375 Random State 143
Test 0.8875 Train0.8375 Random State 144
Test 0.8875 Train0.8375 Random State 145
Test 0.8875 Train0.8375 Random State 146
Test 0.8875 Train0.8375 Random State 147
Test 0.8875 Train0.8375 Random State 148
Test 0.8875 Train0.8375 Random State 149
Test 0.8875 Train0.8375 Random State 150
Test 0.8875 Train0.8375 Random State 151
Test 0.8875 Train0.8375 Random State 152
Test 0.8875 Train0.8375 Random State 153
Test 0.8875 Train0.8375 Random State 154
Test 0.8875 Train0.8375 Random State 155
Test 0.8875 Train0.8375 Random State 156
Test 0.8875 Train0.8375 Random State 157
Test 0.8875 Train0.8375 Random State 158
Test 0.8875 Train0.8375 Random State 159
Test 0.8875 Train0.8375 Random State 160
Test 0.8875 Train0.8375 Random State 161
Test 0.8875 Train0.8375 Random State 162
Test 0.8875 Train0.8375 Random State 163
Test 0.8875 Train0.8375 Random State 164
Test 0.8875 Train0.8375 Random State 165
Test 0.8875 Train0.8375 Random State 166
Test 0.8875 Train0.8375 Random State 167
Test 0.8875 Train0.8375 Random State 168

Test 0.8875 Train0.8375 Random State 169
Test 0.8875 Train0.8375 Random State 170
Test 0.8875 Train0.8375 Random State 171
Test 0.8875 Train0.8375 Random State 172
Test 0.8875 Train0.8375 Random State 173
Test 0.8875 Train0.8375 Random State 174
Test 0.8875 Train0.8375 Random State 175
Test 0.8875 Train0.8375 Random State 176
Test 0.8875 Train0.8375 Random State 177
Test 0.8875 Train0.8375 Random State 178
Test 0.8875 Train0.8375 Random State 179
Test 0.8875 Train0.8375 Random State 180
Test 0.8875 Train0.8375 Random State 181
Test 0.8875 Train0.8375 Random State 182
Test 0.8875 Train0.8375 Random State 183
Test 0.8875 Train0.8375 Random State 184
Test 0.8875 Train0.8375 Random State 185
Test 0.8875 Train0.8375 Random State 186
Test 0.8875 Train0.8375 Random State 187
Test 0.8875 Train0.8375 Random State 188
Test 0.8875 Train0.8375 Random State 189
Test 0.8875 Train0.8375 Random State 190
Test 0.8875 Train0.8375 Random State 191
Test 0.8875 Train0.8375 Random State 192
Test 0.8875 Train0.8375 Random State 193
Test 0.8875 Train0.8375 Random State 194
Test 0.8875 Train0.8375 Random State 195
Test 0.8875 Train0.8375 Random State 196
Test 0.8875 Train0.8375 Random State 197
Test 0.8875 Train0.8375 Random State 198
Test 0.8875 Train0.8375 Random State 199
Test 0.8875 Train0.8375 Random State 200
Test 0.8875 Train0.8375 Random State 201
Test 0.8875 Train0.8375 Random State 202
Test 0.8875 Train0.8375 Random State 203
Test 0.8875 Train0.8375 Random State 204
Test 0.8875 Train0.8375 Random State 205
Test 0.8875 Train0.8375 Random State 206
Test 0.8875 Train0.8375 Random State 207
Test 0.8875 Train0.8375 Random State 208
Test 0.8875 Train0.8375 Random State 209
Test 0.8875 Train0.8375 Random State 210
Test 0.8875 Train0.8375 Random State 211
Test 0.8875 Train0.8375 Random State 212
Test 0.8875 Train0.8375 Random State 213
Test 0.8875 Train0.8375 Random State 214
Test 0.8875 Train0.8375 Random State 215
Test 0.8875 Train0.8375 Random State 216
Test 0.8875 Train0.8375 Random State 217
Test 0.8875 Train0.8375 Random State 218
Test 0.8875 Train0.8375 Random State 219
Test 0.8875 Train0.8375 Random State 220
Test 0.8875 Train0.8375 Random State 221
Test 0.8875 Train0.8375 Random State 222
Test 0.8875 Train0.8375 Random State 223
Test 0.8875 Train0.8375 Random State 224

Test 0.8875 Train0.8375 Random State 225
Test 0.8875 Train0.8375 Random State 226
Test 0.8875 Train0.8375 Random State 227
Test 0.8875 Train0.8375 Random State 228
Test 0.8875 Train0.8375 Random State 229
Test 0.8875 Train0.8375 Random State 230
Test 0.8875 Train0.8375 Random State 231
Test 0.8875 Train0.8375 Random State 232
Test 0.8875 Train0.8375 Random State 233
Test 0.8875 Train0.8375 Random State 234
Test 0.8875 Train0.8375 Random State 235
Test 0.8875 Train0.8375 Random State 236
Test 0.8875 Train0.8375 Random State 237
Test 0.8875 Train0.8375 Random State 238
Test 0.8875 Train0.8375 Random State 239
Test 0.8875 Train0.8375 Random State 240
Test 0.8875 Train0.8375 Random State 241
Test 0.8875 Train0.8375 Random State 242
Test 0.8875 Train0.8375 Random State 243
Test 0.8875 Train0.8375 Random State 244
Test 0.8875 Train0.8375 Random State 245
Test 0.8875 Train0.8375 Random State 246
Test 0.8875 Train0.8375 Random State 247
Test 0.8875 Train0.8375 Random State 248
Test 0.8875 Train0.8375 Random State 249
Test 0.8875 Train0.8375 Random State 250
Test 0.8875 Train0.8375 Random State 251
Test 0.8875 Train0.8375 Random State 252
Test 0.8875 Train0.8375 Random State 253
Test 0.8875 Train0.8375 Random State 254
Test 0.8875 Train0.8375 Random State 255
Test 0.8875 Train0.8375 Random State 256
Test 0.8875 Train0.8375 Random State 257
Test 0.8875 Train0.8375 Random State 258
Test 0.8875 Train0.8375 Random State 259
Test 0.8875 Train0.8375 Random State 260
Test 0.8875 Train0.8375 Random State 261
Test 0.8875 Train0.8375 Random State 262
Test 0.8875 Train0.8375 Random State 263
Test 0.8875 Train0.8375 Random State 264
Test 0.8875 Train0.8375 Random State 265
Test 0.8875 Train0.8375 Random State 266
Test 0.8875 Train0.8375 Random State 267
Test 0.8875 Train0.8375 Random State 268
Test 0.8875 Train0.8375 Random State 269
Test 0.8875 Train0.8375 Random State 270
Test 0.8875 Train0.8375 Random State 271
Test 0.8875 Train0.8375 Random State 272
Test 0.8875 Train0.8375 Random State 273
Test 0.8875 Train0.8375 Random State 274
Test 0.8875 Train0.8375 Random State 275
Test 0.8875 Train0.8375 Random State 276
Test 0.8875 Train0.8375 Random State 277
Test 0.8875 Train0.8375 Random State 278
Test 0.8875 Train0.8375 Random State 279
Test 0.8875 Train0.8375 Random State 280

Test 0.8875 Train0.8375 Random State 281
Test 0.8875 Train0.8375 Random State 282
Test 0.8875 Train0.8375 Random State 283
Test 0.8875 Train0.8375 Random State 284
Test 0.8875 Train0.8375 Random State 285
Test 0.8875 Train0.8375 Random State 286
Test 0.8875 Train0.8375 Random State 287
Test 0.8875 Train0.8375 Random State 288
Test 0.8875 Train0.8375 Random State 289
Test 0.8875 Train0.8375 Random State 290
Test 0.8875 Train0.8375 Random State 291
Test 0.8875 Train0.8375 Random State 292
Test 0.8875 Train0.8375 Random State 293
Test 0.8875 Train0.8375 Random State 294
Test 0.8875 Train0.8375 Random State 295
Test 0.8875 Train0.8375 Random State 296
Test 0.8875 Train0.8375 Random State 297
Test 0.8875 Train0.8375 Random State 298
Test 0.8875 Train0.8375 Random State 299
Test 0.8875 Train0.8375 Random State 300
Test 0.8875 Train0.8375 Random State 301
Test 0.8875 Train0.8375 Random State 302
Test 0.8875 Train0.8375 Random State 303
Test 0.8875 Train0.8375 Random State 304
Test 0.8875 Train0.8375 Random State 305
Test 0.8875 Train0.8375 Random State 306
Test 0.8875 Train0.8375 Random State 307
Test 0.8875 Train0.8375 Random State 308
Test 0.8875 Train0.8375 Random State 309
Test 0.8875 Train0.8375 Random State 310
Test 0.8875 Train0.8375 Random State 311
Test 0.8875 Train0.8375 Random State 312
Test 0.8875 Train0.8375 Random State 313
Test 0.8875 Train0.8375 Random State 314
Test 0.8875 Train0.8375 Random State 315
Test 0.8875 Train0.8375 Random State 316
Test 0.8875 Train0.8375 Random State 317
Test 0.8875 Train0.8375 Random State 318
Test 0.8875 Train0.8375 Random State 319
Test 0.8875 Train0.8375 Random State 320
Test 0.8875 Train0.8375 Random State 321
Test 0.8875 Train0.8375 Random State 322
Test 0.8875 Train0.8375 Random State 323
Test 0.8875 Train0.8375 Random State 324
Test 0.8875 Train0.8375 Random State 325
Test 0.8875 Train0.8375 Random State 326
Test 0.8875 Train0.8375 Random State 327
Test 0.8875 Train0.8375 Random State 328
Test 0.8875 Train0.8375 Random State 329
Test 0.8875 Train0.8375 Random State 330
Test 0.8875 Train0.8375 Random State 331
Test 0.8875 Train0.8375 Random State 332
Test 0.8875 Train0.8375 Random State 333
Test 0.8875 Train0.8375 Random State 334
Test 0.8875 Train0.8375 Random State 335
Test 0.8875 Train0.8375 Random State 336

Test 0.8875 Train0.8375 Random State 337
Test 0.8875 Train0.8375 Random State 338
Test 0.8875 Train0.8375 Random State 339
Test 0.8875 Train0.8375 Random State 340
Test 0.8875 Train0.8375 Random State 341
Test 0.8875 Train0.8375 Random State 342
Test 0.8875 Train0.8375 Random State 343
Test 0.8875 Train0.8375 Random State 344
Test 0.8875 Train0.8375 Random State 345
Test 0.8875 Train0.8375 Random State 346
Test 0.8875 Train0.8375 Random State 347
Test 0.8875 Train0.8375 Random State 348
Test 0.8875 Train0.8375 Random State 349
Test 0.8875 Train0.8375 Random State 350
Test 0.8875 Train0.8375 Random State 351
Test 0.8875 Train0.8375 Random State 352
Test 0.8875 Train0.8375 Random State 353
Test 0.8875 Train0.8375 Random State 354
Test 0.8875 Train0.8375 Random State 355
Test 0.8875 Train0.8375 Random State 356
Test 0.8875 Train0.8375 Random State 357
Test 0.8875 Train0.8375 Random State 358
Test 0.8875 Train0.8375 Random State 359
Test 0.8875 Train0.8375 Random State 360
Test 0.8875 Train0.8375 Random State 361
Test 0.8875 Train0.8375 Random State 362
Test 0.8875 Train0.8375 Random State 363
Test 0.8875 Train0.8375 Random State 364
Test 0.8875 Train0.8375 Random State 365
Test 0.8875 Train0.8375 Random State 366
Test 0.8875 Train0.8375 Random State 367
Test 0.8875 Train0.8375 Random State 368
Test 0.8875 Train0.8375 Random State 369
Test 0.8875 Train0.8375 Random State 370
Test 0.8875 Train0.8375 Random State 371
Test 0.8875 Train0.8375 Random State 372
Test 0.8875 Train0.8375 Random State 373
Test 0.8875 Train0.8375 Random State 374
Test 0.8875 Train0.8375 Random State 375
Test 0.8875 Train0.8375 Random State 376
Test 0.8875 Train0.8375 Random State 377
Test 0.8875 Train0.8375 Random State 378
Test 0.8875 Train0.8375 Random State 379
Test 0.8875 Train0.8375 Random State 380
Test 0.8875 Train0.8375 Random State 381
Test 0.8875 Train0.8375 Random State 382
Test 0.8875 Train0.8375 Random State 383
Test 0.8875 Train0.8375 Random State 384
Test 0.8875 Train0.8375 Random State 385
Test 0.8875 Train0.8375 Random State 386
Test 0.8875 Train0.8375 Random State 387
Test 0.8875 Train0.8375 Random State 388
Test 0.8875 Train0.8375 Random State 389
Test 0.8875 Train0.8375 Random State 390
Test 0.8875 Train0.8375 Random State 391
Test 0.8875 Train0.8375 Random State 392

```
Test 0.8875 Train0.8375 Random State 393
Test 0.8875 Train0.8375 Random State 394
Test 0.8875 Train0.8375 Random State 395
Test 0.8875 Train0.8375 Random State 396
Test 0.8875 Train0.8375 Random State 397
Test 0.8875 Train0.8375 Random State 398
Test 0.8875 Train0.8375 Random State 399
Test 0.8875 Train0.8375 Random State 400
```

In [11]:
```
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,
finalModel=LogisticRegression()
finalModel.fit(x_train,y_train)
```

navigator.clipboard.writeText(formattedText) .then(() => { element.style.display = 'none'; element.parentElement.appendChild(copyMark); setTimeout(() => { copyMark.remove(); element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'orange'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub * https://github.com/skrub-data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_data/temp * @returns "light" or "dark" */ function detectTheme(element) { const body = document.querySelector('body'); // Check VSCode theme const themeKindAttr = body.getAttribute('data-vscode-theme-kind'); const themeNameAttr = body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) { const themeKind = themeKindAttr.toLowerCase(); const themeName = themeNameAttr.toLowerCase(); if (themeKind.includes("dark") || themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") || themeName.includes("light")) { return "light"; } } // Check Jupyter theme if (body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if (body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a parent element's color const color = window.getComputedStyle(element.parentNode, null).getPropertyValue('color'); const match = color.match(/^rgb\s* \s * (\d+)\s*, \s * (\d+)\s*, \s * (\d+)\s*\s*$/i); if (match) { const [r, g, b] = [parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3])]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to system preference return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement === null) { console.error(`Element with id ${elementId} not found.`); } else { const theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } } forceTheme('sk-container-id-1');</script></body>

In [12]:
```
print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))
```

```
0.8375
0.8875
```

In [13]:
```
from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))
```

	precision	recall	f1-score	support
0	0.85	0.93	0.89	257
1	0.85	0.70	0.77	143
accuracy			0.85	400
macro avg	0.85	0.81	0.83	400
weighted avg	0.85	0.85	0.84	400

Exp No:20Experiment to understand K-means clustering algorithm for a given data set. Description: Understand the K-means clustering algorithm for the dataset given.

In [6]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

In [7]:
```
df=pd.read_csv(r"C:\Users\jaiwa\Downloads\Mall_Customers.csv")
```

In [8]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype 
---  ------                  --------------  ----- 
 0   CustomerID              200 non-null    int64 
 1   Genre                   200 non-null    str   
 2   Age                     200 non-null    int64 
 3   Annual Income (k$)      200 non-null    int64 
 4   Spending Score (1-100)  200 non-null    int64 
dtypes: int64(4), str(1)
memory usage: 7.9 KB
```

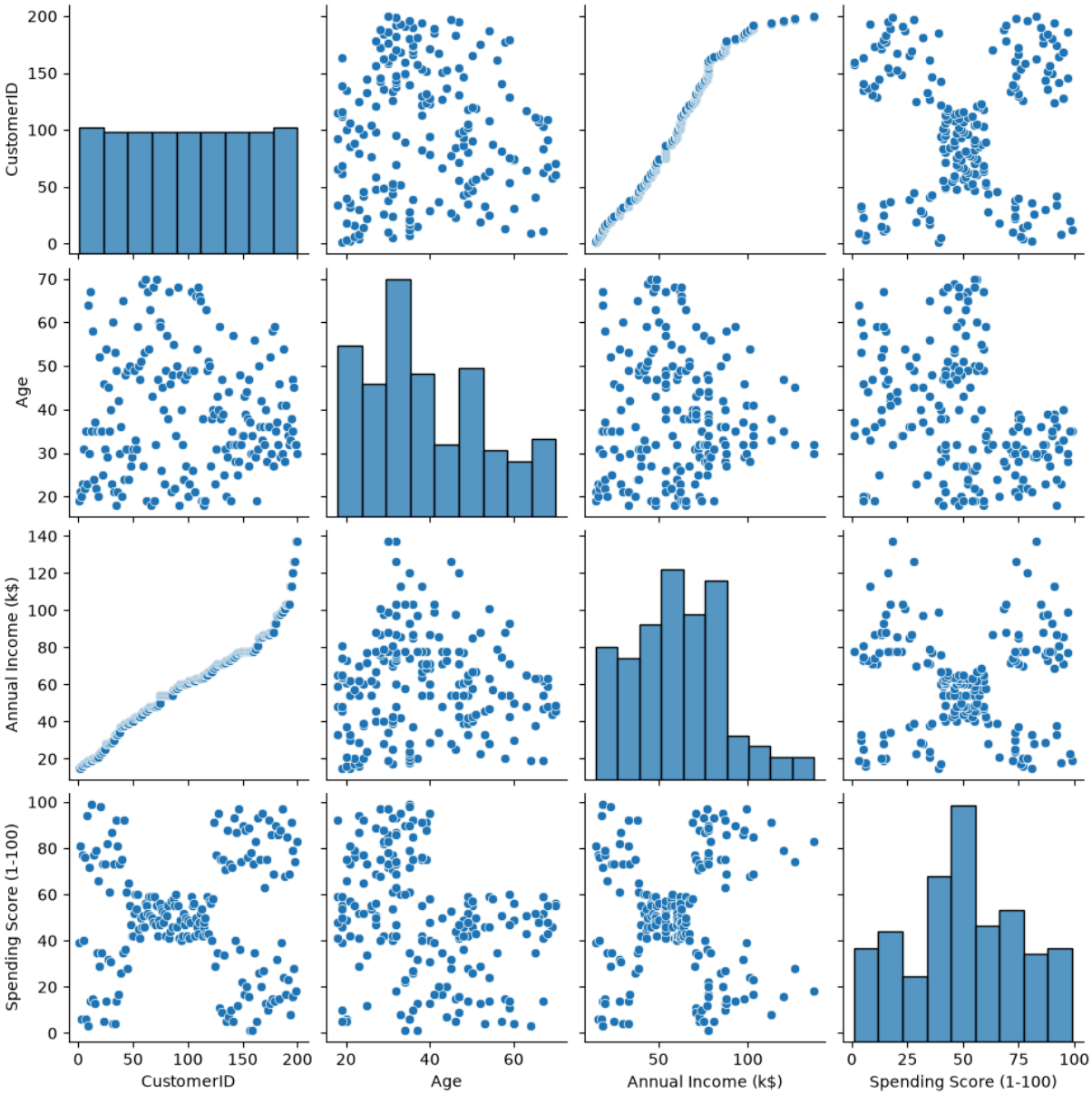
In [9]:
```
df.head()
```

Out[9]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>CustomerID</th> <th>Genre</th> <th>Age</th> <th>Annual Income (k$)</th> <th>Spending Score (1-100)</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1		Male	19	15	39
2		Male	21	15	81
3	Female		20	16	6
4	Female		23	16	77
5	Female		31	17	40

In [10]:
```
sns.pairplot(df)
```

Out[10]: <seaborn.axisgrid.PairGrid at 0x2cb2bb856a0>



In [11]:
```
features=df.iloc[:,[3,4]].values
```

In [12]:
```
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
```

detectTheme(estimatorElement); estimatorElement.classList.add(theme); } }
forceTheme('sk-container-id-1');</script></body>

In [8]:
```
%pip install --upgrade --force-reinstall scikit-learn scipy
```

```
   ---------------------------------- ----- 6/7 [scikit-learn]
   ---------------------------------- ----- 6/7 [scikit-learn]
   ---------------------------------- ----- 6/7 [scikit-learn]
   ---------------------------------- ----- 6/7 [scikit-learn]
   ---------------------------------------- 7/7 [scikit-learn]

Successfully installed cloudpickle-3.1.2 joblib-1.6.0 narwhals-2.27.1 numpy-
2.5.3 scikit-learn-1.9.1 scipy-1.18.1 threadpoolctl-3.7.0
Note: you may need to restart the kernel to use updated packages.
  WARNING: The scripts f2py.exe and numpy-config.exe are installed in 'C:\Use
rs\jaiwa\AppData\Roaming\Python\Python314\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this w
arning, use --no-warn-script-location.
  WARNING: Failed to remove contents in a temporary directory 'C:\Users\jaiwa
\AppData\Roaming\Python\Python314\site-packages\~-mpy.libs'.
  You can safely remove it manually.
  WARNING: Failed to remove contents in a temporary directory 'C:\Users\jaiwa
\AppData\Roaming\Python\Python314\site-packages\~-mpy'.
  You can safely remove it manually.
  WARNING: Failed to remove contents in a temporary directory 'C:\Users\jaiwa
\AppData\Roaming\Python\Python314\site-packages\~-ipy.libs'.
  You can safely remove it manually.
  WARNING: Failed to remove contents in a temporary directory 'C:\Users\jaiwa
\AppData\Roaming\Python\Python314\site-packages\~-ipy'.
  You can safely remove it manually.
  WARNING: Failed to remove contents in a temporary directory 'C:\Users\jaiwa
\AppData\Roaming\Python\Python314\site-packages\~-learn'.
  You can safely remove it manually.
```

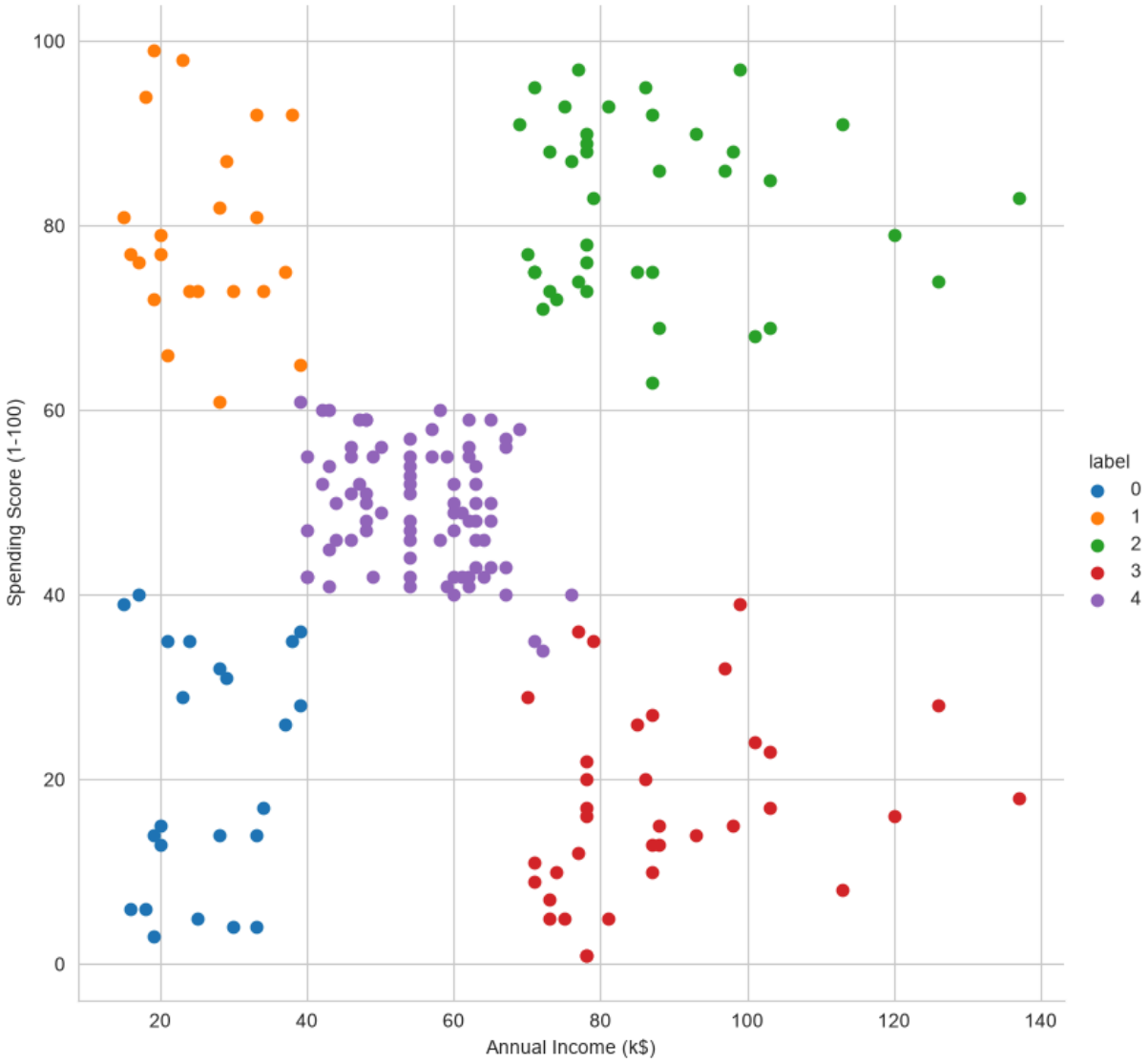
In [13]:
```
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
```

Out[13]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style>

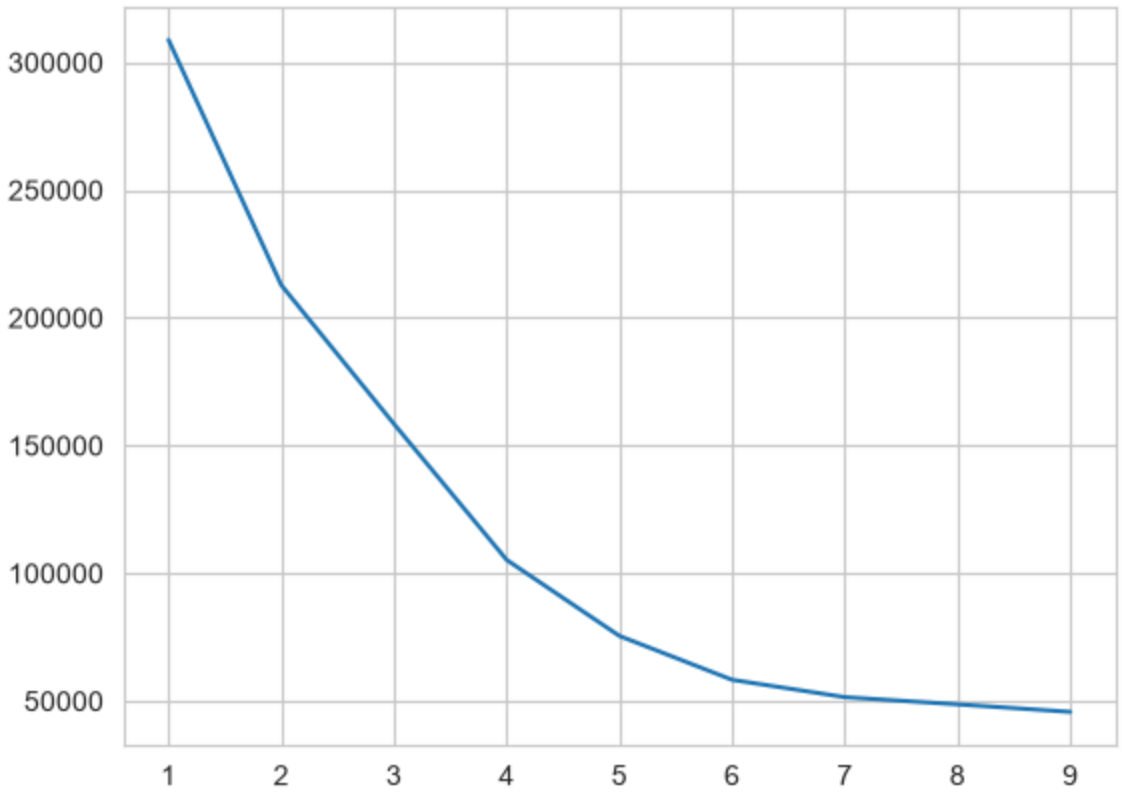
	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	0
1	15	81	1
2	16	6	0
3	16	77	1
4	17	40	0

In [14]:
```
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend();
plt.show()
```



In [16]:
```
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
 model=KMeans(n_clusters=i)
 model.fit(features_el)
 wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```

Out[16]: [<matplotlib.lines.Line2D at 0x2cb2ec063c0>]


300000
250000
200000
150000
100000
50000
1
2
3
4
5
6
7
8
9